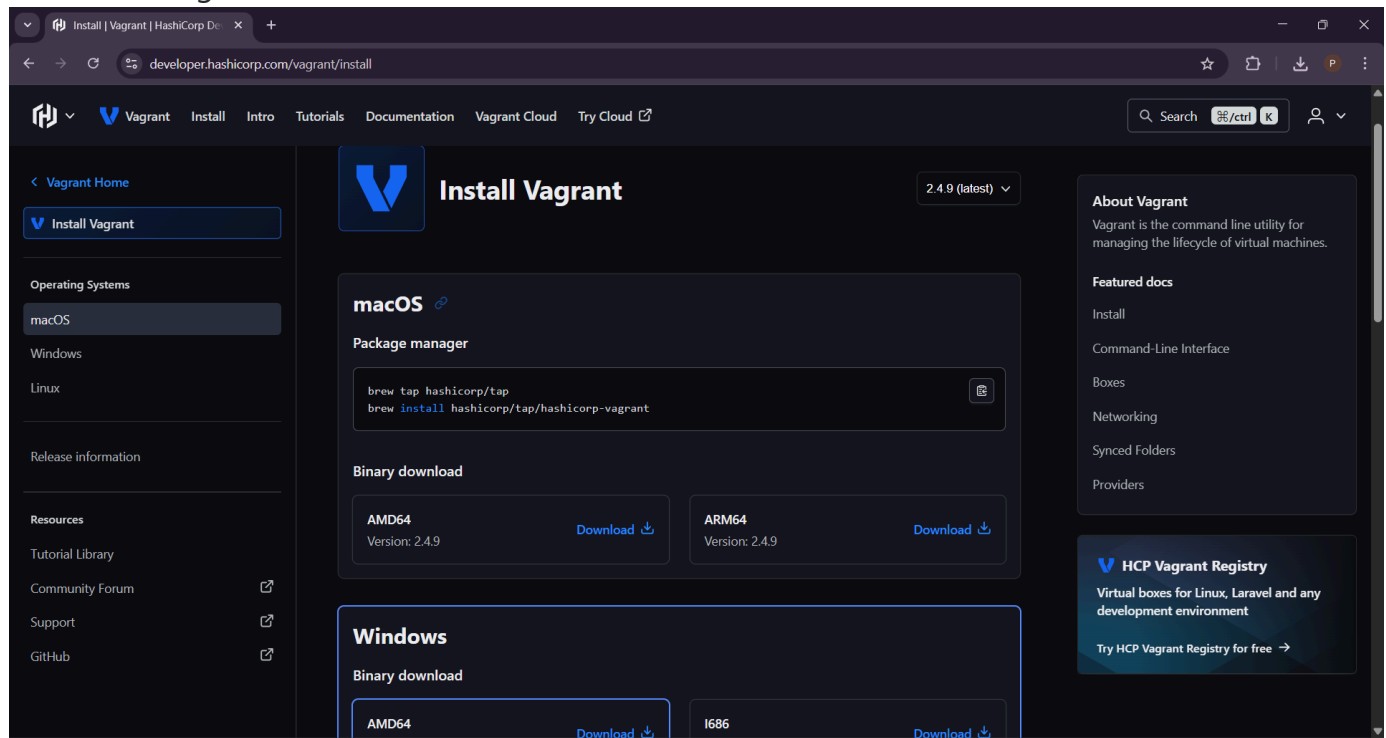


Cloud-containerization-and-DevOps

This is Experiment-1 Readme

Download Virtual Box from [here](#)

Download Vagrant from [here](#)



To verify the installation we will check the version via following command

```
vagrant --version
```

```
PS C:\Users\hp\Desktop\DEMO> vagrant -v
Vagrant 2.4.9
```

Initialize Vagrant with Ubuntu box:

```
vagrant init hashicorp/bionic64
```

```
Vagrant 2.4.9
PS C:\Users\hp\Desktop\DEMO> vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
```

Start the VM:

```
vagrant up
```

```
PS C:\Users\hp\Desktop\DEMO> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find and install...
default: Box Provider: virtualbox
default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualbox
default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/versions/1.0.282/providers/virtualbox/unknown/vagrant.box
Progress: 65% (Rate: 21.5M/s, Estimated time remaining: 0:00:10)
```

Access the VM:

```
vagrant ssh
```

```
Windows PowerShell
PS C:\Users\hp\Desktop\DEMO> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information disabled due to load higher than 1.0

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 packages can be updated.
0 updates are security updates.
```

Step 4: Install Nginx inside VM

```
sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
```

Verify Nginx

```
curl localhost
```

```
58261a837e072ada0319e55846b48ff294b3a02fba6e52bf7c3ec503b5cf2e98
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Utilization Matrix In Running State

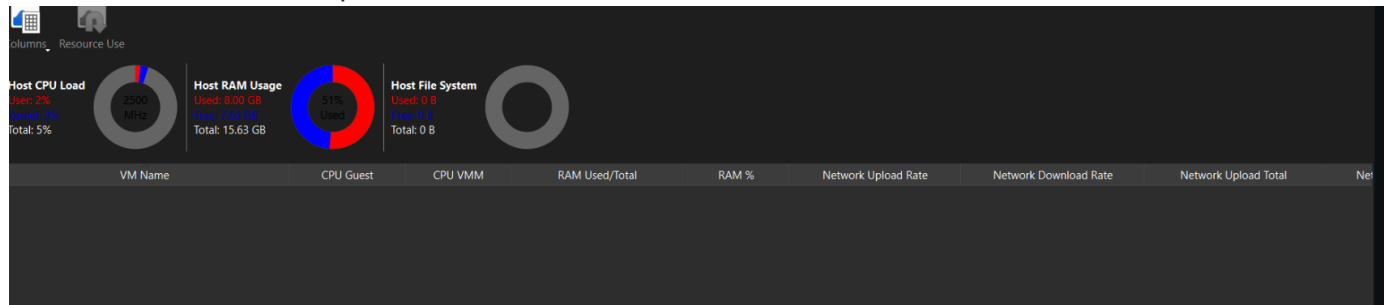
The screenshot displays two windows side-by-side. The left window is a Windows PowerShell terminal running the Vagrant command-line interface. It shows the default configuration for a VM named 'Palak16', including the operating system (Ubuntu 18.04.3 LTS), the host (C:\Users\hp\Desktop\DEMO), and the network configuration. It also displays system information for the host, such as system load, memory usage, and the number of processes. The right window is the Oracle VM VirtualBox Manager interface, showing the 'Palak16' VM in a 'Powered Off' state. The 'General' tab is selected, displaying the VM's name, operating system, and hardware configuration (4GB RAM, 4 processors, VBOXSVGA graphics controller).

Stop VM

vagrant halt

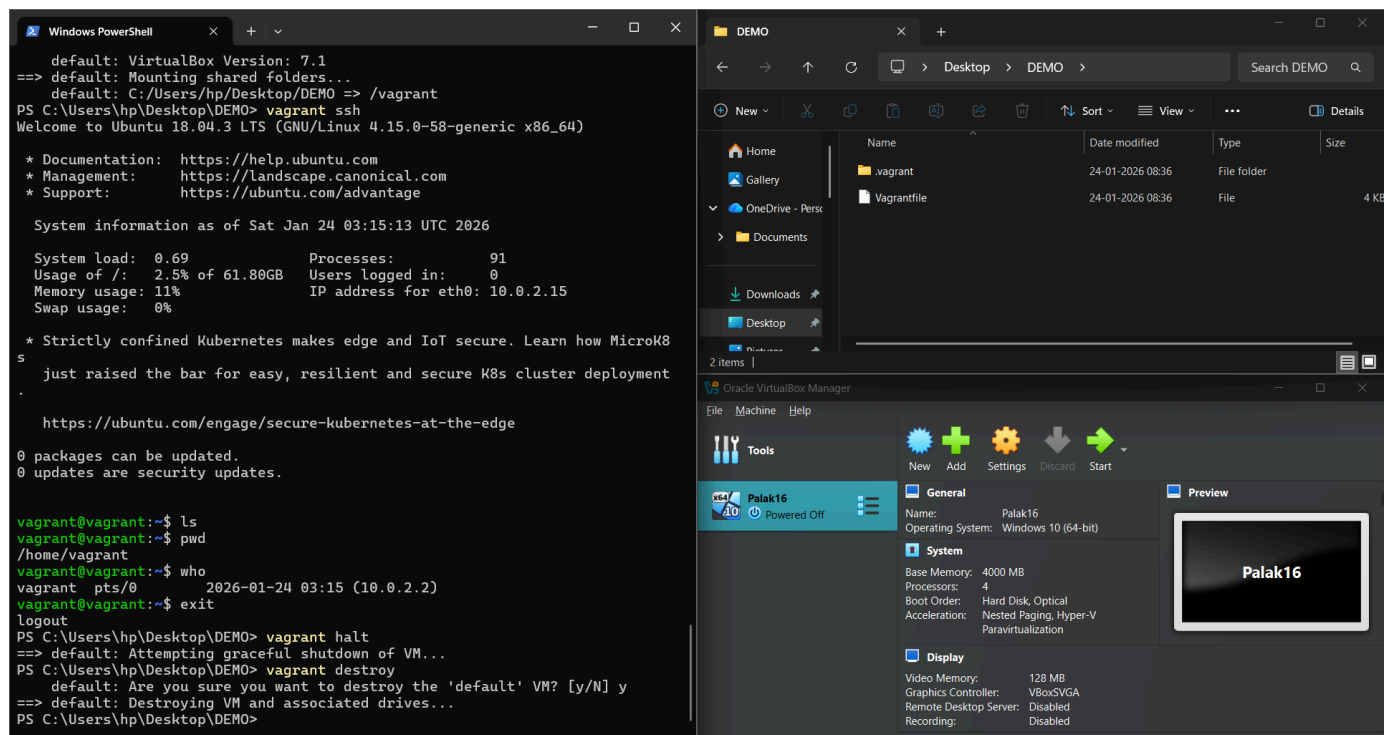
```
logout
PS C:\Users\hp\Desktop\DEMO> vagrant halt
==> default: Attempting graceful shutdown of VM...
PS C:\Users\hp\Desktop\DEMO> vagrant destroy
default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
PS C:\Users\hp\Desktop\DEMO>
```

Utilization Matrix In Stop State



Remove VM

```
vagrant destroy
```



PART-B:- Containers using WSL (Windows)

Step 1: Install WSL

```
wsl --install
```

Reboot the system after installation.

Step 2: Install Ubuntu on WSL

```
wsl --install -d Ubuntu
```



```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo --install -d Ubuntu
sudo: unrecognized option '--install'
usage: sudo -h | -K | -k | -V
usage: sudo -v [-ABkNnS] [-g group] [-h host] [-p prompt] [-u user]
usage: sudo -l [-ABkNnS] [-g group] [-h host] [-p prompt] [-U user]
        [-u user] [command [arg ...]]
usage: sudo [-ABbEHkNnPS] [-r role] [-t type] [-C num] [-D directory]
        [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
        [-u user] [VAR=value] [-i | -s] [command [arg ...]]
usage: sudo -e [-ABkNnS] [-r role] [-t type] [-C num] [-D directory]
        [-g group] [-h host] [-p prompt] [-R directory] [-T timeout]
        [-u user] file ...

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done

```

Step 3: Install Docker Engine inside WSL

```

sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo usermod -aG docker $USER

```

Logout and login again to apply group changes.

Step 4: Verify Docker Installation

```

docker --version

```

```

exit
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1

```

Step 5: Pull Ubuntu Image

```

docker pull ubuntu

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo apt install -y docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (28.2.2-0ubuntu1~24.04.1).
The following package was automatically installed and is no longer required:
  libllvm19
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 86 not upgraded.
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo systemctl start docker
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo usermod -aG docker $USER
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest

```

Step 6: Run Ubuntu Container with Nginx

```
docker run -d -p 8080:80 --name nginx-container nginx
```

```

Run 'docker run --help' for more information
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo docker run -d -p 8080:80 --name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
58261a837e072ada0319e55846b48ff294b3a02fba6e52bf7c3ec503b5cf2e98

```

Step 7: Verify Nginx in Container

```
curl localhost:8080
```

```

58261a837e072ada0319e55846b48ff294b3a02fba6e52bf7c3ec503b5cf2e98
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

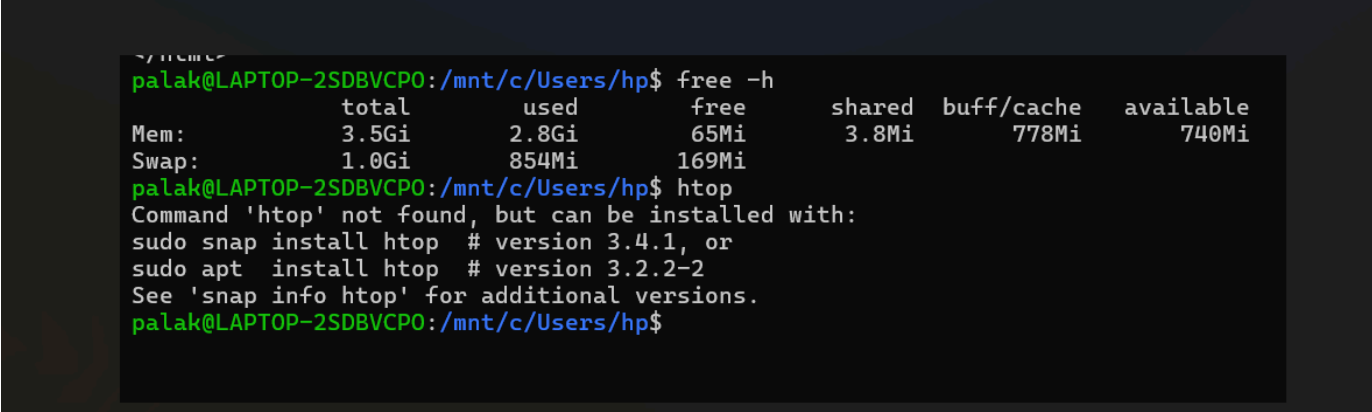
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Step 8: Container Observation Commands

```
docker stats  
free -h
```



```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ free -h  
              total        used        free      shared  buff/cache   available  
Mem:           3.5Gi        2.8Gi        65Mi       3.8Mi       778Mi       740Mi  
Swap:          1.0Gi         854Mi        169Mi  
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ htop  
Command 'htop' not found, but can be installed with:  
sudo snap install htop # version 3.4.1, or  
sudo apt install htop # version 3.2.2-2  
See 'snap info htop' for additional versions.  
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$
```

Parameters to Compare

Parameter	Virtual Machine	Container
Boot Time	High	Very Low
RAM Usage	High	Low
CPU Overhead	Higher	Minimal
Disk Usage	Larger	Smaller
Isolation	Strong	Moderate

Cloud-containerization-and-DevOps

Experiment 2:- Docker Installation, Configuration, and Running Images [↗](#)

Step 1: Pull Image

```
docker --version
```

```
exit
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

Step 2: Run Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

```
Run 'docker run --help' for more information
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo docker run -d -p 8080:80 --name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
58261a837e072ada0319e55846b48ff294b3a02fba6e52bf7c3ec503b5cf2e98
```

Step 3: Verify Running Containers

```
docker ps
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo docker ps
[sudo] password for palak:
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
d1d16711340f   nginx    "/docker-entrypoint..." 41 seconds ago Up 40 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp kind_sutherland
```

Step 4: Stop Container

```
docker stop <container_id>
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo docker stop d1d
d1d
```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp\$ sudo docker ps -a							
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES	
d1d16711340f	nginx	"/docker-entrypoint..."	4 minutes ago	Exited (0)	About a minute ago	kind_sutherland	

Step 5: Remove Container

```
docker rm <container_id>
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo docker rm d1d  
d1d
```

Step 6: Remove Image

```
docker rmi nginx
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo docker rmi nginx  
Untagged: nginx:latest  
Untagged: nginx@sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec  
Deleted: sha256:4af177a024eb8a1e43f4fb6c66735bb8260115cb5925a64f51673219bd97c144  
Deleted: sha256:74821a6b71dfac14eaeb80c7c9e48130391306557f75228278054afe105cde75  
Deleted: sha256:ae46c5037a8cc17f658f78b0e6523c9363ca800bf51f91f58cbc3aad8ffe53b5  
Deleted: sha256:ec81248cdbcb6b02bb24b1c32d4f59194a4a42aab3c81a42631efa1af7cba4f69  
Deleted: sha256:774cc13f29089d7d0ebaaa291c5ac2d12841f2d70ab07a21c470d3cbdc11c059  
Deleted: sha256:f1155703f3479bf6b295f5e17684156f82f815ca0798af8a2d08cf994411a055  
Deleted: sha256:a455653294af11c547d0b3d2d769fd184f357cd5c0decabd1f89ffe9b380985aa  
Deleted: sha256:e50a58335e1366e2581fe61794c1651afe2fe04e881e795aa166f24f4fc78d92
```

Result Docker images were successfully pulled, containers executed, and lifecycle commands performed.

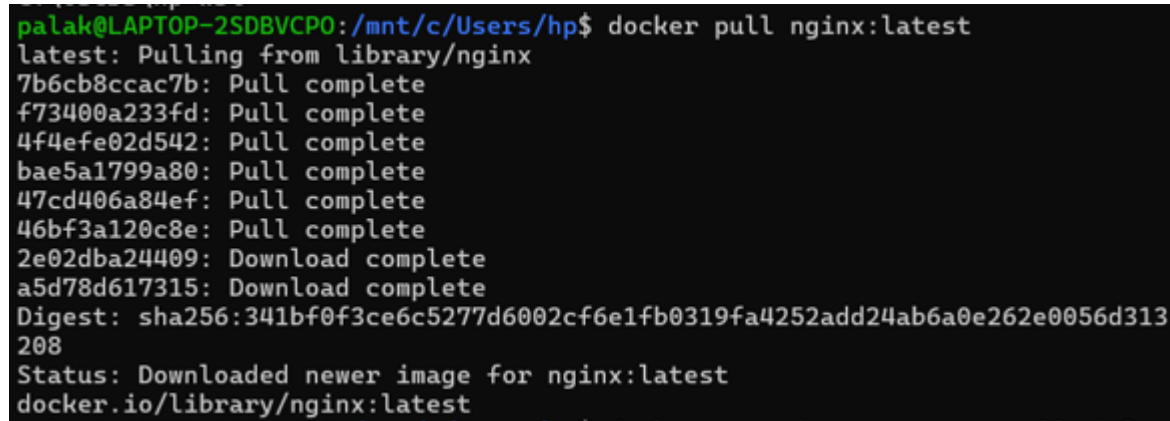
Cloud-containerization-and-DevOps

Experiment 3:- Docker Deploy NGINX Using Different Base Images and Comparing Image Layers

Part 1: Deploy NGINX Using Official Image (Recommended Approach)

Step 1: Pull the Image

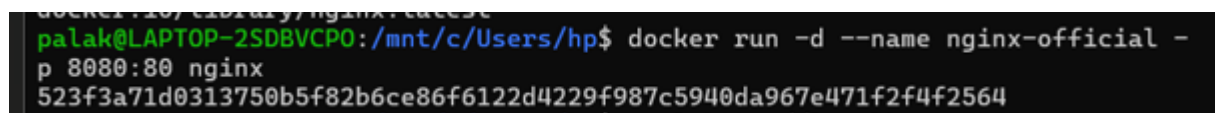
```
docker pull nginx:latest
```



```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker pull nginx:latest
latest: Pulling from library/nginx
7b6cb8ccac7b: Pull complete
f73400a233fd: Pull complete
4f4efe02d542: Pull complete
bae5a1799a80: Pull complete
47cd406a84ef: Pull complete
46bf3a120c8e: Pull complete
2e02dba24409: Download complete
a5d78d617315: Download complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Step 2: Run the Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```



```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d --name nginx-official -p 8080:80 nginx
523f3a71d0313750b5f82b6ce86f6122d4229f987c5940da967e471f2f4f2564
```

Step 3: Verify

```
curl http://localhost:8080
```

```

523f3a71d0313750b5f82b6ce86f6122d4229f987c5940da967e471f2f4f2564
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

```

You should see the NGINX welcome page.

Key Observations

docker images nginx

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker images nginx
REPOSITORY      TAG          IMAGE ID      CREATED      SIZE
nginx           latest       341bf0f3ce6c  2 days ago  240MB

```

- Image is **pre-optimized**
- Minimal configuration required
- Uses Debian-based OS internally

Part 2: Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

FROM ubuntu:22.04

RUN apt-get update && \
 apt-get install -y nginx && \
 apt-get clean && \
 rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/nginx-ubuntu$ docker build -t nginx-ubuntu .
[+] Building 95.0s (6/6) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 218B                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04  7.1s
=> [auth] library/ubuntu:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                 0.1s
=> => transferring context: 2B                                    0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2 21.4s
=> => resolve docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2a 0.1s
=> => sha256:6f4ebca3e823b18dac366f72e537b1772bc3 29.54MB / 29.54MB 16.8s
=> => extracting sha256:6f4ebca3e823b18dac366f72e537b1772bc3522a5c7a 4.4s
=> ERROR [2/2] RUN apt-get update && apt-get install -y nginx & 65.8s
-----
> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get c
lean && rm -rf /var/lib/apt/lists/*:
3.133 Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
3.322 Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129
kB]
4.984 Get:3 http://security.ubuntu.com/ubuntu jammy-security/universe amd64
Packages [1297 kB]
4.996 Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB
]
5.760 Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127
kB]
7.083 Get:6 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages
[266 kB]
```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/nginx_latest$ docker run --name nginx_latest_container -p 8081:80 -d nginx_latest_custom
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/nginx_latest$ |
```

Observations

```
docker images nginx-ubuntu
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker images nginx
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx latest 341bf0f3ce6c 2 days ago 240MB
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker build -t nginx-ubuntu .
```

- Much larger image size
- More layers
- Full OS utilities available

Part 3: Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

```
FROM alpine:latest

RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-alpine .
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/nginx_ubuntu/nginx_alpine$ docker build -t nginx_alpine_custom
[+] Building 11.4s (7/7) FINISHED          docker:default
=> [internal] load build definition from Dockerfile                                0.1s
=> => transferring dockerfile: 104B                                              0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                  1.4s
=> [internal] load .dockerignore                                                  0.1s
=> => transferring context: 2B                                                  0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb0            8.5s
=> => resolve docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb0            0.1s
=> => sha256:955a8478f9aceb66cbf2f579fa3c24e1af278d1 1.21kB / 1.21kB          1.4s
=> => sha256:6d397a54a185dd0b6638d1a3934b81daef7a140 1.40kB / 1.40kB          1.1s
=> => sha256:5e7756927bef33a266e1221356d5da8553139 20.24MB / 20.24MB          5.5s
=> => sha256:6b7b6c7061b76cdb8601e18722d12ae3232f0ddcfaf1 954B / 954B         1.4s
=> => sha256:399d0898a94e0084f81499a3e3c29824357118c7ce5 404B / 404B         0.7s
=> => sha256:3e2c181db1b0985ce357c7aaf48ac615f30f392cd15 626B / 626B         0.7s
=> => sha256:bca5d04786e112d958f100a66f8257b2aeefc14 1.86MB / 1.86MB          1.2s
=> => sha256:589002ba0eae121a1dbf42f6648f29e5be55d5 3.86MB / 3.86MB          1.9s
=> => extracting sha256:589002ba0eae121a1dbf42f6648f29e5be55d5c8a6e 0.9s
=> => extracting sha256:bca5d04786e112d958f100a66f8257b2aeefc14b64d8 0.9s
=> => extracting sha256:3e2c181db1b0985ce357c7aaf48ac615f30f392cd15d 0.1s
=> => extracting sha256:6b7b6c7061b76cdb8601e18722d12ae3232f0ddcfaf1d 0.0s
=> => extracting sha256:399d0898a94e0084f81499a3e3c29824357118c7ce55 0.0s
=> => extracting sha256:955a8478f9aceb66cbf2f579fa3c24e1af278d1fa3ff 0.0s
=> => extracting sha256:6d397a54a185dd0b6638d1a3934b81daef7a140741e1 0.1s
=> => extracting sha256:5e7756927bef33a266e1221356d5da8553139cb80bc5 2.1s
=> [internal] load build context                                                  0.1s
=> => transferring context: 77B                                                  0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html                      0.3s
=> => exporting to image                                                         0.7s
=> => exporting layers                                                            0.2s
=> => exporting manifest sha256:3f087e4537f72bfe7cdfe26db640adda44d9 0.0s
=> => exporting config sha256:a9554a547c9f0d96ff56d09998a8879f90a696 0.0s
=> => exporting attestation manifest sha256:c4e4e4f8a5e07f3df03e028a 0.1s
=> => exporting manifest list sha256:e1920b54b1d3050b3ccab60aa8786fd 0.0s
=> => naming to docker.io/library/nginx_alpine_custom:latest                  0.0s
=> => unpacking to docker.io/library/nginx_alpine_custom:latest                0.1s
```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/nginx_ubuntu/nginx_alpine$ docker run --name nginx_alpine_container -p 8083:80 -d nginx_alpine_custom
ddc95792741066e8482236b372793aa3bd9df37cccba6b38c225a893eb4e5087
```

Observations

```
docker images nginx-alpine
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-nginx-experiment/nginx_ubuntu/nginx_alpine$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
nginx_alpine_custom  latest             e1920b54b1d3       23 seconds ago     92.5MB
nginx_ubuntu_custom  latest             bd03b4cee3df       3 days ago         226MB
nginx_latest_custom  latest             60cfe9a9a0c0       3 days ago         237MB
myimage              1.0               6294b3435f74       9 days ago         183MB
mysql                latest             db32c8ec843c       10 days ago        1.27GB
nginx                latest             341bf0f3ce6c       11 days ago        240MB
ubuntu               latest             cd1dba651b30       4 weeks ago        119MB
```

- Extremely small image
- Fewer packages
- Faster pull and startup time

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-nginx-experiment/nginx_ubuntu/nginx_alpine$ docker images | grep nginx
nginx_alpine_custom  latest             e1920b54b1d3       5 minutes ago     92.5MB
nginx_ubuntu_custom  latest             bd03b4cee3df       3 days ago         226MB
nginx_latest_custom  latest             60cfe9a9a0c0       3 days ago         237MB
nginx                latest             341bf0f3ce6c       11 days ago        240MB
```

Typical result (approx):

Image Type	Size
nginx:latest	237 MB
nginx-ubuntu	226 MB
nginx-alpine	92 MB

Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/nginx_ubuntu/nginx_alpine$ docker history nginx_ubuntu_custom
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
bd03b4cee3df   3 days ago      CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      3 days ago      EXPOSE [80/tcp]                                0B        buildkit.dockerfile.v0
<missing>      3 days ago      COPY index.html /var/www/html/index.html # b... 20.5kB     buildkit.dockerfile.v0
<missing>      3 days ago      RUN /bin/sh -c apt-get update && apt-get ins... 65.9MB     buildkit.dockerfile.v0
<missing>      4 weeks ago     /bin/sh -c #(nop) CMD ["/bin/bash"]           0B
<missing>      4 weeks ago     /bin/sh -c #(nop) ADD file:3077ee44db3cc7d38... 87.6MB
<missing>      4 weeks ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B
<missing>      4 weeks ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B
<missing>      4 weeks ago     /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH    0B
<missing>      4 weeks ago     /bin/sh -c #(nop) ARG RELEASE                 0B
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/nginx_ubuntu/nginx_alpine$ docker history nginx_latest_custom
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
60cfe9a9a0c0   3 days ago      COPY index.html /usr/share/nginx/html/index... 24.6kB     buildkit.dockerfile.v0
<missing>      11 days ago     CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      11 days ago     STOPSIGNAL SIGQUIT                            0B        buildkit.dockerfile.v0
<missing>      11 days ago     EXPOSE map[80/tcp:{}]                        0B        buildkit.dockerfile.v0
<missing>      11 days ago     ENTRYPOINT ["/docker-entrypoint.sh"]         0B        buildkit.dockerfile.v0
<missing>      11 days ago     COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB     buildkit.dockerfile.v0
<missing>      11 days ago     COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB     buildkit.dockerfile.v0
<missing>      11 days ago     COPY 15-local-resolvers.envsh /docker-entryp... 12.3kB     buildkit.dockerfile.v0
<missing>      11 days ago     COPY 10-listen-on-ipv6-by-default.sh /docke... 12.3kB     buildkit.dockerfile.v0
<missing>      11 days ago     COPY docker-entrypoint.sh / # buildkit       8.19kB     buildkit.dockerfile.v0
<missing>      11 days ago     RUN /bin/sh -c set -x && groupadd --syst... 86.6MB     buildkit.dockerfile.v0
<missing>      11 days ago     ENV DYNPKG_RELEASE=1~trixie                  0B        buildkit.dockerfile.v0
<missing>      11 days ago     ENV PKG_RELEASE=1~trixie                    0B        buildkit.dockerfile.v0
<missing>      11 days ago     ENV ACME_VERSION=0.3.1                      0B        buildkit.dockerfile.v0
<missing>      11 days ago     ENV NJS_RELEASE=1~trixie                    0B        buildkit.dockerfile.v0
<missing>      11 days ago     ENV NJS_VERSION=0.9.5                       0B        buildkit.dockerfile.v0
<missing>      11 days ago     ENV NGINX_VERSION=1.29.5                    0B        buildkit.dockerfile.v0
<missing>      11 days ago     LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1... 87.4MB     debuerreotype 0.17
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/nginx_ubuntu/nginx_alpine$ docker history nginx
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
341bf0f3ce6c   11 days ago     CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>      11 days ago     STOPSIGNAL SIGQUIT                            0B        buildkit.dockerfile.v0
<missing>      11 days ago     EXPOSE map[80/tcp:{}]                        0B        buildkit.dockerfile.v0
<missing>      11 days ago     ENTRYPOINT ["/docker-entrypoint.sh"]         0B        buildkit.dockerfile.v0
<missing>      11 days ago     COPY 30-tune-worker-processes.sh /docker-ent... 16.4kB     buildkit.dockerfile.v0
<missing>      11 days ago     COPY 20-envsubst-on-templates.sh /docker-ent... 12.3kB     buildkit.dockerfile.v0

```

Observations:

- Ubuntu has many filesystem layers
- Alpine has minimal layers
- Official NGINX image is optimized but heavier than Alpine

Part 5: Functional Tasks Using NGINX

Task 1: Serve Custom HTML Page

```

mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment$ mkdir html
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment$ echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment$ nano Dockerfile
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment$ ls
Dockerfile  html  nginx_latest  nginx_ubuntu
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment$ cd html/
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/html$ ls
index.html
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-nginx-experiment/html$ |

```

Run:

```

docker run -d \
  -p 8083:80 \

```

```
-v $(pwd)/html:/usr/share/nginx/html \
nginx
```

Task 2: Reverse Proxy (Conceptual)

NGINX can:

- Forward traffic to backend services
- Load balance multiple containers
- Terminate SSL

Example use cases:

- Frontend for microservices
- API gateway
- Static file server

Part 6: Comparison Summary

Image Comparison Table

Feature	Official NGINX	Ubuntu + NGINX	Alpine + NGINX
Image Size	Medium	Large	Very Small
Ease of Use	Very Easy	Medium	Medium
Startup Time	Fast	Slow	Very Fast
Debugging Tools	Limited	Excellent	Minimal
Security Surface	Medium	Large	Small
Production Ready	Yes	Rarely	Yes

Part 7: When to Use What

Use Official NGINX Image When:

- Production deployment
- Standard web hosting
- Reverse proxy / load balancer

Use Ubuntu-Based Image When:

- Learning Linux + NGINX internals
- Heavy debugging needed
- Custom system-level dependencies

Use Alpine-Based Image When:

- Microservices
- CI/CD pipelines
- Cloud and Kubernetes workloads

Cloud-containerization-and-DevOps

Experiment 4: Docker Essentials

Dockerfile .dockerignore tagging publishing

Containerizing Applications with Dockerfile

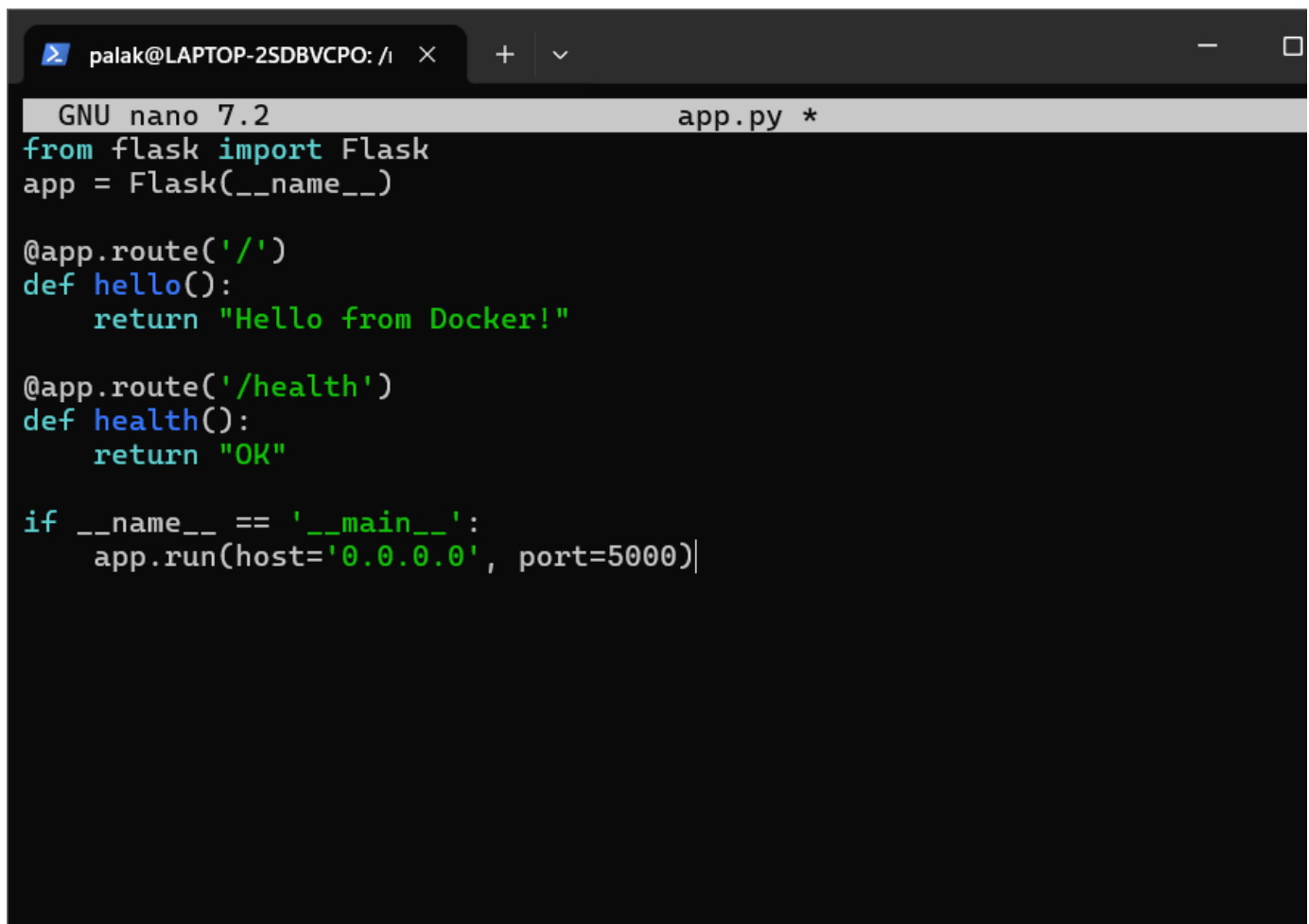
Step-1: Create a Directory for Python application

```
mkdir my-flask-app  
cd my-flask-app
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4$ mkdir my-flask-app  
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4$ cd my-flask-app  
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$
```

Step-2:- Create an **** app.py ****

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Docker!"  
  
@app.route('/health')  
def health():  
    return "OK"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```



```
palak@LAPTOP-2SDBVCPO: /1 × + v
GNU nano 7.2 app.py *
from flask import Flask
app = Flask(__name__)

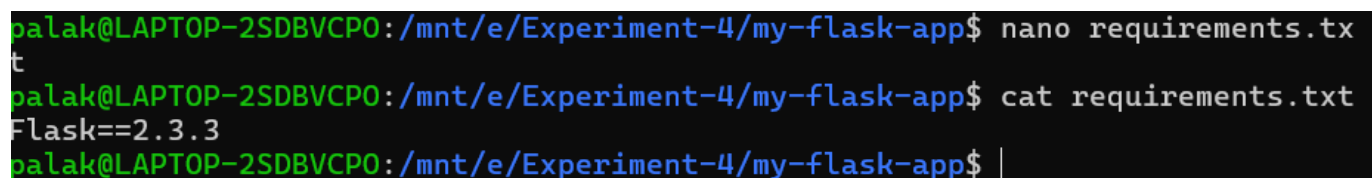
@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step-3:- Create **requirements.txt** :

```
Flask==2.3.3
```



```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ nano requirements.tx
t
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ cat requirements.txt
Flask==2.3.3
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-4: Create **Dockerfile**

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
```



```
# Copy application code
```

```
COPY app.py .
```

```
# Expose port
```

```
EXPOSE 5000
```

```
# Run the application
```

```
CMD ["python", "app.py"]
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ nano Dockerfile
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ cat Dockerfile
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-5:- Create a **** .dockerignore ****

```
# Python files
```

```
__pycache__/
```

```
*.pyc
```

```
*.pyo
```

```
*.pyd
```

```
# Environment files
```

```
.env
```

```
.venv
```

```
env/
```

```
venv/
```

```
# IDE files
```

```
.vscode/
```

```
.idea/
```

```
# Git files
```

```
.git/
```

```
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ nano .dockerignore
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ cat .dockerignore
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ |
```

Why .dockerignore is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Step-6: Build image from Dockerfile

```
docker build -t my-flask-app .
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4$ cd my-flask-app
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker build -t my-flask-app .
[+] Building 20.3s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 361B                                0.1s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.9s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 273B                                    0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                                  0.1s
=> => transferring context: 321B                                    0.1s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> [3/5] COPY requirements.txt .                                    0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt       13.2s
=> [5/5] COPY app.py .                                            0.2s
=> exporting to image                                              4.0s
=> => exporting layers                                             2.2s
=> => exporting manifest sha256:ce68559591e7e98f3f73f9a860ce03fa3978 0.0s
=> => exporting config sha256:f314acacc3728e7e2ee0f4454eee1d97176128 0.0s
=> => exporting attestation manifest sha256:d5af0b7ca2d416dcc86c7d82 0.1s
=> => exporting manifest list sha256:4031a16a4a75b78cba7b150fabf3628 0.0s
=> => naming to docker.io/library/my-flask-app:latest             0.1s
=> => unpacking to docker.io/library/my-flask-app:latest          1.3s
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-7:- Verify Image

```
docker images
```

```
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
my-flask-app        latest      4031a16a4a75  31 seconds ago 200MB
nginx_alpine_custom latest      e1920b54b1d3  7 days ago    92.5MB
nginx_ubuntu_custom latest      bd03b4cee3df  10 days ago   226MB
nginx_latest_custom latest      60cfe9a9a0c0  10 days ago   237MB
myimage             1.0        6294b3435f74  2 weeks ago   183MB
palakagarwal16/java-app 1.0        6294b3435f74  2 weeks ago   183MB
mysql               latest      db32c8ec843c  2 weeks ago   1.27GB
nginx               latest      341bf0f3ce6c  2 weeks ago   240MB
alpine              latest      25109184c71b  3 weeks ago   13.1MB
ubuntu              latest      cd1dba651b30  5 weeks ago   119MB
busybox             latest      b3255e7dfbcd  17 months ago 6.77MB
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$
```

Step-8: Add Tag in Image

```
docker build -t my-flask-app:1.0 .
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker build -t my-flask-app:1.0 .
[+] Building 2.3s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.4s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 273B                                    0.1s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                                   0.1s
=> => transferring context: 63B                                     0.1s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                     0.0s
=> exporting to image                                              0.3s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:ce68559591e7e98f3f73f9a860ce03fa3978 0.0s
=> => exporting config sha256:f314acacc3728e7e2ee0f4454eee1d97176128 0.0s
=> => exporting attestation manifest sha256:f813f338c370437fbb6214d0 0.1s
=> => exporting manifest list sha256:84617a456e6cd533d724e729969c72a 0.0s
=> => naming to docker.io/library/my-flask-app:1.0                0.0s
=> => unpacking to docker.io/library/my-flask-app:1.0              0.0s
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$
```

Step-9:- Add multiple tags

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

```

palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker build -t my-flask-app:latest -t my-flask-app:1.0 .
[+] Building 3.2s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile               0.1s
=> => transferring dockerfile: 361B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.2s
=> [auth] library/python:pull token for registry-1.docker.io     0.0s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 273B                                   0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                                  0.1s
=> => transferring context: 63B                                    0.0s
=> CACHED [2/5] WORKDIR /app                                     0.0s
=> CACHED [3/5] COPY requirements.txt .                           0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                    0.0s
=> exporting to image                                           0.4s
=> => exporting layers                                           0.0s
=> => exporting manifest sha256:ce68559591e7e98f3f73f9a860ce03fa3978 0.0s
=> => exporting config sha256:f314acacc3728e7e2ee0f4454eee1d97176128 0.0s
=> => exporting attestation manifest sha256:54e382ef93033e118393d901 0.1s
=> => exporting manifest list sha256:fcabc2ae649ca59f6706f37a273534df 0.0s
=> => naming to docker.io/library/my-flask-app:latest            0.0s
=> => unpacking to docker.io/library/my-flask-app:latest         0.0s
=> => naming to docker.io/library/my-flask-app:1.0               0.0s
=> => unpacking to docker.io/library/my-flask-app:1.0           0.0s
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$

```

Step-10:- Add Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker build -t user
name/my-flask-app:1.0 .
[+] Building 2.3s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.3s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 273B                                    0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                                  0.1s
=> => transferring context: 63B                                    0.1s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                            0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                     0.0s
=> exporting to image                                              0.3s
=> => exporting layers                                            0.0s
=> => exporting manifest sha256:ce68559591e7e98f3f73f9a860ce03fa3978 0.0s
=> => exporting config sha256:f314acacc3728e7e2ee0f4454eee1d97176128 0.0s
=> => exporting attestation manifest sha256:5f03f32cfe4102b86a375eaa 0.1s
=> => exporting manifest list sha256:a275c7525e0dd5d6a7b26b5494f14ac 0.0s
=> => naming to docker.io/username/my-flask-app:1.0              0.0s
=> => unpacking to docker.io/username/my-flask-app:1.0           0.0s
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$

```

Step-11:- Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker tag my-flask-
app:latest my-flask-app:v1.0

```

Step-12: View Image Details

```
docker images
```

```
app: latest my-flask-app.v1.0
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
username/my-flask-app 1.0            a275c7525e0d   3 minutes ago  200MB
my-flask-app         latest         fcbc2ae649ca   3 minutes ago  200MB
my-flask-app         v1.0          fcbc2ae649ca   3 minutes ago  200MB
my-flask-app         1.0           84617a456e6c   3 minutes ago  200MB
nginx_alpine_custom  latest        e1920b54b1d3   7 days ago     92.5MB
nginx_ubuntu_custom latest        bd03b4cee3df   10 days ago    226MB
nginx_latest_custom latest        60cfe9a9a0c0   10 days ago    237MB
myimage             1.0           6294b3435f74   2 weeks ago    183MB
palakagarwal16/java-app 1.0          6294b3435f74   2 weeks ago    183MB
mysql               latest        db32c8ec843c   2 weeks ago    1.27GB
nginx               latest        341bf0f3ce6c   2 weeks ago    240MB
alpine              latest        25109184c71b   3 weeks ago    13.1MB
ubuntu              latest        cd1dba651b30   5 weeks ago    119MB
busybox             latest        b3255e7dfbcd   17 months ago  6.77MB
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$
```

Step-13:- Show image history

```
docker history my-flask-app
```



```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker history my-fl
ask-app
IMAGE          CREATED          CREATED BY
SIZE          COMMENT
fcbc2ae649ca   4 minutes ago   CMD ["python" "app.py"]
0B            buildkit.dockerfile.v0
<missing>      4 minutes ago   EXPOSE [5000/tcp]
0B            buildkit.dockerfile.v0
<missing>      4 minutes ago   COPY app.py . # buildkit
12.3kB        buildkit.dockerfile.v0
<missing>      4 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r...
13.4MB        buildkit.dockerfile.v0
<missing>      4 minutes ago   COPY requirements.txt . # buildkit
12.3kB        buildkit.dockerfile.v0
<missing>      7 minutes ago   WORKDIR /app
8.19kB        buildkit.dockerfile.v0
<missing>      3 months ago   CMD ["python3"]
0B            buildkit.dockerfile.v0
<missing>      3 months ago   RUN /bin/sh -c set -eux;  for src in idle3 p...
16.4kB        buildkit.dockerfile.v0
<missing>      3 months ago   RUN /bin/sh -c set -eux;  savedAptMark="$(a...
45.7MB        buildkit.dockerfile.v0
<missing>      3 months ago   ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8...
0B            buildkit.dockerfile.v0
<missing>      3 months ago   ENV PYTHON_VERSION=3.9.25
0B            buildkit.dockerfile.v0
<missing>      3 months ago   ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3...
0B            buildkit.dockerfile.v0
<missing>      3 months ago   RUN /bin/sh -c set -eux;  apt-get update; a...
4.94MB        buildkit.dockerfile.v0
<missing>      3 months ago   ENV LANG=C.UTF-8
0B            buildkit.dockerfile.v0
<missing>      3 months ago   ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...
0B            buildkit.dockerfile.v0
<missing>      4 months ago   # debian.sh --arch 'amd64' out/ 'trixie' '@1...
87.4MB        debuerreotype 0.16
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$

```

Step-14:- Inspect image details

```
docker inspect my-flask-app
```



```

palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker inspect my-flask-app
[
  {
    "Architecture": "amd64",
    "Author": "",
    "Comment": "buildkit.dockerfile.v0",
    "Config": {
      "ArgsEscaped": true,
      "Cmd": [
        "python",
        "app.py"
      ],
      "Entrypoint": null,
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Labels": null,
      "OnBuild": null,
      "User": "",
      "Volumes": null,
      "WorkingDir": "/app"
    },
    "Created": "2026-02-23T15:02:20.589283809Z",
    "Descriptor": {
      "digest": "sha256:fcabc2ae649ca59f6706f37a273534dffe2b22d437a94ef32246581daedc4295d",
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "size": 856
    },
    "DockerVersion": ""
  }
]

```

Step-15: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

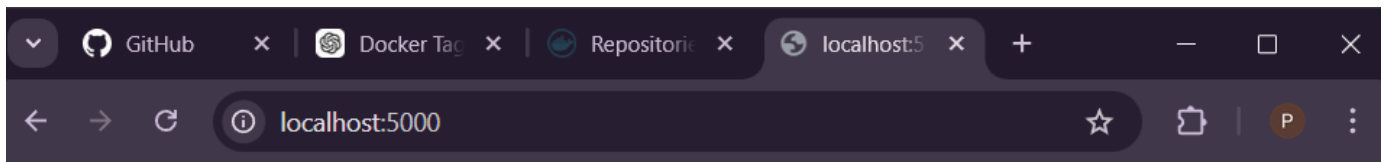
```

palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker run -d -p 5000:5000 --name flask-container my-flask-app
3e08cb4fd98b2e66f83248529a0451ba445d77944bcc06140d1598d880b14882

```

Step-16:- Test the application

```
curl http://localhost:5000
```



Hello from Docker!

Step-17:- View running containers

```
docker ps
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
3e08cb4fd98b   my-flask-app   "python app.py"         2 minutes ago Up 2 minutes
0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Step-18:- View container logs

```
docker logs flask-container
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$
```

Step-19:- Stop container

```
docker stop flask-container
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker stop flask-container
flask-container
```

Step-20:- Start stopped container

```
docker start flask-container
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker start flask-container
flask-container
```

Step-21:- Remove container

```
docker rm flask-container
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker rm -f flask-container
flask-container
```

Step-22:- If working then Remove container forcefully

Since Container is in Running State, so we will remove it forcefully...

```
docker rm -f flask-container
```

Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step-23: Create a Simple **** Dockerfile.multistage ****

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .
```

```
# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ nano Dockerfile.multistage
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ cat Dockerfile.multistage
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]

palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$
```

Step-24:- Build regular image

```
docker build -t flask-regular .
```

```

palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker build -t flask-regular .
[+] Building 3.3s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 361B                             0.1s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.3s
=> [auth] library/python:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 273B                                  0.1s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                                0.1s
=> => transferring context: 63B                                    0.1s
=> CACHED [2/5] WORKDIR /app                                    0.0s
=> CACHED [3/5] COPY requirements.txt .                          0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                    0.0s
=> exporting to image                                           0.4s
=> => exporting layers                                           0.0s
=> => exporting manifest sha256:ce68559591e7e98f3f73f9a860ce03fa3978 0.0s
=> => exporting config sha256:f314acacc3728e7e2ee0f4454eee1d97176128 0.0s
=> => exporting attestation manifest sha256:7997fcce865f6b68dbbe90e6 0.1s
=> => exporting manifest list sha256:57183d2afc77e6e366a369d08f71d65 0.0s
=> => naming to docker.io/library/flask-regular:latest          0.0s
=> => unpacking to docker.io/library/flask-regular:latest       0.0s
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |

```

Step-25:- Build multi-stage image

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 33.0s (13/13) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile.multistage    0.1s
=> => transferring dockerfile: 676B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.3s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 273B                                    0.0s
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.1s
=> [internal] load build context                                  0.1s
=> => transferring context: 63B                                    0.0s
=> CACHED [builder 2/5] WORKDIR /app                              0.0s
=> CACHED [builder 3/5] COPY requirements.txt .                    0.0s
=> [builder 4/5] RUN python -m venv /opt/venv                      13.7s
=> [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt 7.1s
=> [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv          1.8s
=> [stage-1 4/5] COPY app.py .                                    0.2s
=> [stage-1 5/5] RUN useradd -m -u 1000 appuser                    0.9s
=> exporting to image                                             5.6s
=> => exporting layers                                             2.8s
=> => exporting manifest sha256:7442dda76c8f647bb5e53cbbab437633d406 0.0s
=> => exporting config sha256:b3eff3e39a8f47fb3dbc3663d673b5f99f41e6 0.0s
=> => exporting attestation manifest sha256:61673a525f9412de21facf33 0.1s
=> => exporting manifest list sha256:6cab8b39f6da3682b2f0fb3e7b9d511 0.0s
=> => naming to docker.io/library/flask-multistage:latest         0.0s
=> => unpacking to docker.io/library/flask-multistage:latest      2.5s
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ |

```

Step-26:- Compare sizes

```
docker images | grep flask-
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ docker images | grep flask-
flask-multistage          latest      6cab8b39f6da  40 seconds ago  219MB
my-flask-app              latest      fcbc2ae649ca  15 minutes ago  200MB
my-flask-app              v1.0       fcbc2ae649ca  15 minutes ago  200MB
my-flask-app              1.0        84617a456e6c  15 minutes ago  200MB
username/my-flask-app     1.0        a275c7525e0d  15 minutes ago  200MB
flask-regular             latest      57183d2afc77  15 minutes ago  200MB
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app$ |

```

Publishing to Docker Hub

Step-27:- Login to Docker Hub

```
docker login
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker login
Authenticating with existing credentials... [Username: palakagarwal16]

i Info → To login with a different account, run 'docker logout' followed by
'docker login'

Login Succeeded
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-28:- Tag image for Docker Hub

```
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker tag my-flask-
app:latest username/my-flask-app:1.0
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker tag my-flask-
app:latest username/my-flask-app:latest
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-29:- Push to Docker Hub

```
docker push username/my-flask-app:1.0
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker push palakaga
rwal16/my-flask-app:1.0
The push refers to repository [docker.io/palakagarwal16/my-flask-app]
845db20e1195: Pushed
fed5e4f9175c: Pushed
fc7443084902: Pushed
729f71acce10: Pushed
0d7dd57d7152: Pushed
b3ec39b36ae8: Pushed
38513bd72563: Pushed
ea56f685404a: Pushed
b7575848b527: Pushed
1.0: digest: sha256:84617a456e6cd533d724e729969c72ab648446e6b758de599712fa91
1243cc2d size: 856
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-30:- Push Latest Image

```
docker push username/my-flask-app:latest
```



```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker push palakagarwal16/my-flask-app:latest
The push refers to repository [docker.io/palakagarwal16/my-flask-app]
03aa8506aca6: Pushed
fc7443084902: Layer already exists
ea56f685404a: Layer already exists
38513bd72563: Layer already exists
729f71acce10: Layer already exists
b3ec39b36ae8: Layer already exists
0d7dd57d7152: Layer already exists
b7575848b527: Layer already exists
fed5e4f9175c: Layer already exists
latest: digest: sha256:fcabc2ae649ca59f6706f37a273534dffeeb22d437a94ef32246581daedc4295d size: 856
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-31:- Pull from Docker Hub (on another machine)

```
docker pull username/my-flask-app:latest
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker pull palakagarwal16/my-flask-app:latest
latest: Pulling from palakagarwal16/my-flask-app
Digest: sha256:fcabc2ae649ca59f6706f37a273534dffeeb22d437a94ef32246581daedc4295d
Status: Image is up to date for palakagarwal16/my-flask-app:latest
docker.io/palakagarwal16/my-flask-app:latest
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |
```

Step-32:- Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ docker run -d -p 5000:5000 palakagarwal16/my-flask-app:latest
a93d12e29be1340d574d06be35aad4bb5689ae7d1827ffc7aa2591639284071a
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ |
```

Node.js Application Example (Quick Version)**

Step-33:- Create a directory Node.js Application

```
mkdir my-node-app
cd my-node-app
```

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ mkdir my-node-app
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app$ cd my-node-app
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app/my-node-app$ |
```

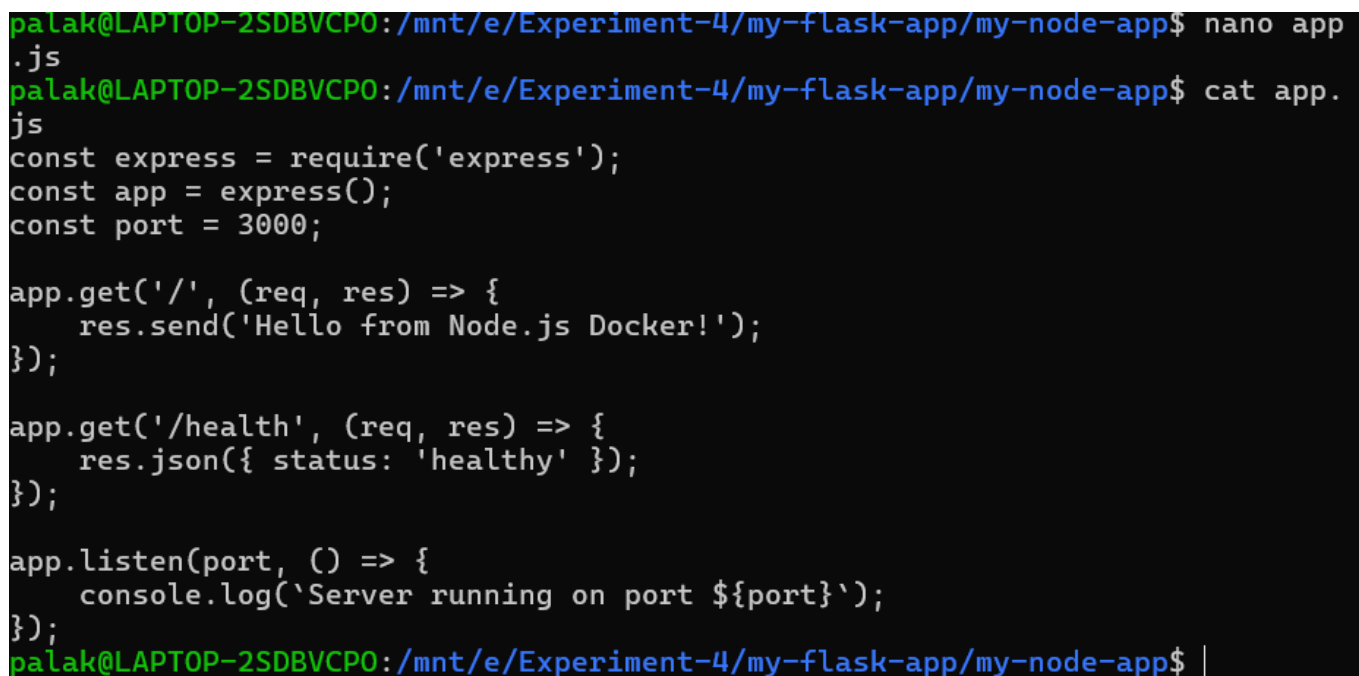
Step-34:- Create a ** app.js **:~

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

A terminal window with a dark background and light green text. The prompt is 'palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app/my-node-app\$'. The user enters 'nano app' and then '.js'. The prompt changes to 'palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app/my-node-app\$'. The user enters 'cat app.js'. The terminal displays the contents of the file: a Node.js Express server script that listens on port 3000, has a root route returning 'Hello from Node.js Docker!', and a /health route returning a JSON object with status 'healthy'. The prompt returns to 'palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app/my-node-app\$' with a cursor at the end.

```
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app/my-node-app$ nano app
.js
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app/my-node-app$ cat app.
js
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
palak@LAPTOP-2SDBVCP0:/mnt/e/Experiment-4/my-flask-app/my-node-app$ |
```

Step-35:- Create a **** package.json ****

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ nano package.json
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ cat package.json
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ |

```

Step-36:- Create a Node.js Dockerfile

```

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]

```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ nano Dockerfile
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ cat Dockerfile
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ |

```

Step-37:- Build image

```
docker build -t my-node-app .
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ docker build -t my-node-app .
[+] Building 38.2s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 183B                                0.1s
=> [internal] load metadata for docker.io/library/node:18-alpine  4.5s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                      0.0s
=> [internal] load build context                                   0.2s
=> => transferring context: 514B                                    0.1s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c 21.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c2 0.2s
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda 1.26MB / 1.26MB 1.0s
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab2 446B / 446B 1.1s
=> => sha256:f18232174bc91741fdf3da96d85011092101a03 3.64MB / 3.64MB 2.1s
=> => sha256:dd71dde834b5c203d162902e6b8994cb2309 40.01MB / 40.01MB 16.2s
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a 0.6s
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0e 4.1s
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837 0.2s
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab22 0.0s
=> [2/5] WORKDIR /app                                              0.1s
=> [3/5] COPY package*.json ./                                     0.1s
=> [4/5] RUN npm install --only=production                         8.0s
=> [5/5] COPY app.js .                                             0.2s
=> exporting to image                                              3.2s
=> => exporting layers                                             1.0s
=> => exporting manifest sha256:d49f2efe10e16a37ff48961869858427a0d6 0.0s
=> => exporting config sha256:d2c9bddc691ebde927ab6a7df3b3f2f89c5ebe 0.0s
=> => exporting attestation manifest sha256:24d6d5f8f7d6a42e524e1e24 0.1s
=> => exporting manifest list sha256:9db77d28d3a2482ce1ff48319f76dc1 0.0s
=> => naming to docker.io/library/my-node-app:latest              0.0s
=> => unpacking to docker.io/library/my-node-app:latest           1.8s
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ |

```

Step-38:- Run container

```
docker run -d -p 3000:3000 --name node-container my-node-app
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ docker run -d -p 3000:3000 --name node-container my-node-app
dce02afcd688fc1732588c59271ff73acf78de3ab469ecfe884c5e8d2a82bc8f
palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ |

```

Step-39:- Test on Terminal

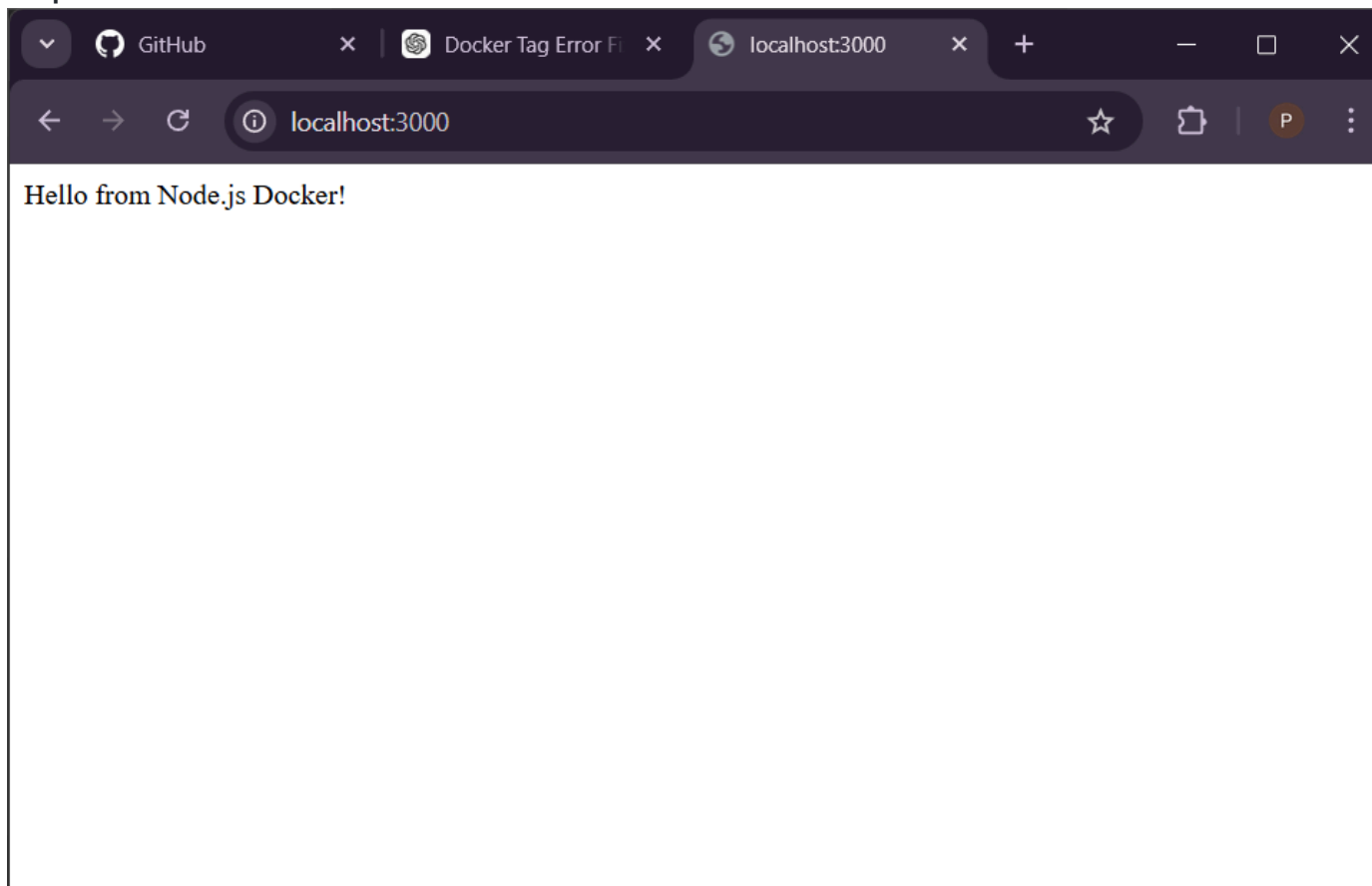
```
curl http://localhost:3000
```

```

palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ curl http://localhost:3000
Hello from Node.js Docker!palak@LAPTOP-2SDBVCPO:/mnt/e/Experiment-4/my-flask-app/my-node-app$ |

```

Step-40:- Test on Browser



DockerHub

View Pushed Image on [DockerHub](#)

Name	Last Pushed ↑	Contains	Visibility
palakagarwal16/my-node-app	less than a minute ago	IMAGE	Public
palakagarwal16/my-flask-app	17 minutes ago	IMAGE	Public

Conclusion

Essential Docker Commands Cheatsheet

Command	Purpose	Example
<code>docker build</code>	Build image	<code>docker build -t myapp .</code>

Command	Purpose	Example
<code>docker run</code>	Run container	<code>docker run -p 3000:3000 myapp</code>
<code>docker ps</code>	List containers	<code>docker ps -a</code>
<code>docker images</code>	List images	<code>docker images</code>
<code>docker tag</code>	Tag image	<code>docker tag myapp:latest myapp:v1</code>
<code>docker login</code>	Login to Dockerhub using username and password or token	<code>echo "token" docker login -u username --password-stdin</code>
<code>docker push</code>	Push to registry	<code>docker push username/myapp</code>
<code>docker pull</code>	Pull from registry	<code>docker pull username/myapp</code>
<code>docker rm</code>	Remove container	<code>docker rm container-name</code>
<code>docker rmi</code>	Remove image	<code>docker rmi image-name</code>
<code>docker logs</code>	View logs	<code>docker logs container-name</code>
<code>docker exec</code>	Execute command	<code>docker exec -it container-name bash</code>

Common Workflow Summary

Development Workflow:

```
# 1. Create Dockerfile and .dockerignore
# 2. Build image
docker build -t myapp .

# 3. Test Locally
docker run -p 8080:8080 myapp

# 4. Tag for production
docker tag myapp:latest myapp:v1.0

# 5. Push to registry
docker push myapp:v1.0
```

Production Workflow:

```
# 1. Pull from registry
docker pull myapp:v1.0

# 2. Run in production
docker run -d -p 80:8080 --name prod-app myapp:v1.0

# 3. Monitor
docker logs -f prod-app
```

Key Takeaways

1. **Dockerfile** defines how to build your image
2. **.dockerignore** excludes unnecessary files (important for security and performance)
3. **Tagging** helps version and organize images
4. **Multi-stage builds** create smaller, more secure production images
5. **Docker Hub** allows sharing and distributing images
6. **Always test** images locally before publishing

Cleanup

```
# Remove all stopped containers
docker container prune

# Remove unused images
docker image prune

# Remove everything unused
docker system prune -a
```

</div>

Cloud-containerization-and-DevOps

Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
# Create a container that writes data
docker run -it --name test-container ubuntu /bin/bash

# Inside container:
echo "Hello World" > /data/message.txt
cat /data/message.txt # Shows "Hello World"
exit

# Restart container
docker start test-container
docker exec test-container cat /data/message.txt
# ERROR: File doesn't exist! Data was lost.
```

Solution: Docker Volumes

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -it --name test-container
ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
e4a1e8de092c: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072fee
f0c
Status: Downloaded newer image for ubuntu:latest
root@15da38de8f32:/# |

root@15da38de8f32:/# mkdir /data
root@15da38de8f32:/# echo "Hello World" > /data/message.txt
root@15da38de8f32:/# cat /data/message.txt
Hello World
root@15da38de8f32:/# exit
exit
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$
```



```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker start test-container
test-container
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker exec test-container cat /data/
message.txt
Hello World
```

Lab 2: Volume Types

1. Anonymous Volumes

Create anonymous volume (auto-generated name)

```
docker run -d -v /app/data --name web1 nginx
```

Check volume

```
docker volume ls
```

Shows: anonymous volume with random hash

Inspect container to see volume mount

```
docker inspect web1 | grep -A 5 Mounts
```

2. Named Volumes

Create named volume

```
docker volume create mydata
```

Use named volume

```
docker run -d -v mydata:/app/data --name web2 nginx
```

List volumes

```
docker volume ls
```

Shows: mydata

Inspect volume

```
docker volume inspect mydata
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume create mydata
mydata
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
--name web2 \
-v mydata:/app/data \
nginx
32b1e567b876c41ebda73f837bad5f427250cf6516033bcd9c8a1ac193898ce3
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume ls
DRIVER      VOLUME NAME
local       00cbe078c61bece7062e787243a4f79685e2d22b86462fd9a0d76d8630c777f8
local       6fc2dbc37ecbadeab75df0d357f7c75376d351ac8427a8dfc4767871f080be9d
local       38d08c83a4e91a9437770b43beae89a8c339464e98cd4ed4f0d064fcf860473b
local       332eec9471b31c0c62578961487f38ae027142e49c286c9bbd02228ba085e0b5
local       assignment-1_pgdata
local       dbdata
local       k3d-mycluster-images
local       mydata
local       myvolume
local       test

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume inspect mydata
[
  {
    "CreatedAt": "2026-03-22T15:29:37Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]

```

3. Bind Mounts (Host Directory)

Create directory on host

```
mkdir ~/myapp-data
```

Mount host directory to container

```
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
```

Add file on host

```
echo "From Host" > ~/myapp-data/host-file.txt
```

Check in container

```
docker exec web3 cat /app/data/host-file.txt
```

Shows: From Host

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ mkdir ~/myapp-data
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
--name web3 \
-v ~/myapp-data:/app/data \
nginx
e997985e0420e327087669998fbf9e8918e698dfeaf9ebe25e032d1fb24ee648
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ echo "From Host" > ~/myapp-data/test.
txt
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker exec web3 cat /app/data/test.t
xt
From Host

```

Lab 3: Practical Volume Examples

Example 1: Database with Persistent Storage

MySQL with named volume

```
docker run -d \  
  --name mysql-db \  
  -v mysql-data:/var/lib/mysql \  
  -e MYSQL_ROOT_PASSWORD=secret \  
  mysql:8.0
```

Check data persists

```
docker stop mysql-db  
docker rm mysql-db
```

New container with same volume

```
docker run -d \  
  --name new-mysql \  
  -v mysql-data:/var/lib/mysql \  
  -e MYSQL_ROOT_PASSWORD=secret \  
  mysql:8.0
```

Data is preserved!

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
d4c7048d1cf1: Pull complete
771773a1fe7a: Pull complete
6375ec02106f: Pull complete
5915b0e5580c: Pull complete
60c3210ab1f6: Pull complete
c3b8cfb10b41: Pull complete
440208b3f29f: Pull complete
1378718ee973: Pull complete
10ee8ef3b9fc: Pull complete
cd8097e7c26b: Pull complete
0940ecf5daec: Pull complete
de0490608ca2: Download complete
2f1dcce71f1f: Download complete
Digest: sha256:64756cc92f707eb504496d774353990bcb0f6999ddf598b6ad188f2da66bd000
Status: Downloaded newer image for mysql:8.0
d3becb7d728089f8193afa53f60499054c946e19ee17f0c28f5ee6b7c1a24685
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
d4c7048d1cf1: Pull complete
771773a1fe7a: Pull complete
6375ec02106f: Pull complete
5915b0e5580c: Pull complete
60c3210ab1f6: Pull complete
c3b8cfb10b41: Pull complete
440208b3f29f: Pull complete
1378718ee973: Pull complete
10ee8ef3b9fc: Pull complete
cd8097e7c26b: Pull complete
0940ecf5daec: Pull complete
de0490608ca2: Download complete
2f1dcce71f1f: Download complete
Digest: sha256:64756cc92f707eb504496d774353990bcb0f6999ddf598b6ad188f2da66bd000
Status: Downloaded newer image for mysql:8.0
d3becb7d728089f8193afa53f60499054c946e19ee17f0c28f5ee6b7c1a24685
```

Example 2: Web App with Configuration Files

```
# Create config directory
mkdir ~/nginx-config

# Create nginx config file
echo 'server {
    listen 80;
    server_name localhost;
    location / {
```

```

        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf

# Run nginx with config bind mount
docker run -d \
    --name nginx-custom \
    -p 8080:80 \
    -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
    nginx

# Test
curl http://localhost:8080

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker stop mysql-db
mysql-db
mysql-db
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
    --name new-mysql \
    -v mysql-data:/var/lib/mysql \
    -e MYSQL_ROOT_PASSWORD=secret \
    mysql:8.0
3c5cf44a9167df443a4b0c6939e577b1417f3e93ba2cfe68c1926a96201848f5

```

Lab 4: Volume Management Commands

```

# List all volumes
docker volume ls

# Create a volume
docker volume create app-volume

# Inspect volume details
docker volume inspect app-volume

# Remove unused volumes
docker volume prune

# Remove specific volume
docker volume rm volume-name

# Copy files to/from volume
docker cp local-file.txt container-name:/path/in/volume

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume ls
DRIVER      VOLUME NAME
local       00cbe078c61bece7062e787243a4f79685e2d22b86462fd9a0d76d8630c777f8
local       6fc2dbc37ecbadeab75df0d357f7c75376d351ac8427a8dfc4767871f080be9d
local       38d08c83a4e91a9437770b43beae89a8c339464e98cd4ed4f0d064fcf860473b
local       332eec9471b31c0c62578961487f38ae027142e49c286c9bbd02228ba085e0b5
local       app-volume
local       assignment-1_pgdata
local       dbdata
local       k3d-mycluster-images
local       mydata
local       mysql-data
local       myvolume
local       test
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume create app-volume
app-volume
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-03-27T06:07:18Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one c
ontainer.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume rm volume-name
Error response from daemon: volume volume-name not found
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker volume rm app-volume
app-volume

```

Part 2: Environment Variables

Lab 1: Setting Environment Variables

Method 1: Using -e flag

```

# Single variable
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  my-node-app

```

```

# Multiple variables

```

```
docker run -d \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  my-app
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  -p 3000:3000 \
  my-node-app
8d280d63827382034831695d80f7e1ce56873e6c3f97c45daa7ff0207ad48aca
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  my-node-app
59c09dd25b9943f21dd4a516b9052c6888472f77bb1e343c67bcbefd394bb73a
```

Lab 2: Environment Variables in Applications

Python Flask Example

```
# app.py
import os
from flask import Flask

app = Flask(__name__)

# Read environment variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-env-app$ cat app.py
import os
from flask import Flask

app = Flask(__name__)

# Read environment variables
db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

Dockerfile with Environment Variables

```
FROM python:3.9-slim

# Set environment variables at build time
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

# Default runtime environment variables
ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
```



```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-env-app$ cat Dockerfile
FROM python:3.9-slim

ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
```

Lab 3: Test Environment Variables

Run with custom env vars

```
docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e PORT="8080" \
  flask-app
```

Check environment in running container

```
docker exec flask-app env
docker exec flask-app printenv DATABASE_HOST
```

Test the endpoint

```
curl http://localhost:5000/config
```

```

flask-app
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e PORT="5000" \
  flask-app
8fa6b865af4ca513e56656431242975381e7f0e38f887826590353febc1bb0b6
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker exec flask-app env
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=149d273a6168
LANG=C.UTF-8
GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568
PYTHON_VERSION=3.9.25
PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834
HOME=/root
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker exec flask-app printenv DATABASE_HOST

```

Part 3: Docker Monitoring

Lab 1: Basic Monitoring Commands

docker stats - Real-time Container Metrics

Live stats for all containers

```
docker stats
```

Live stats for specific containers

```
docker stats container1 container2
```

Specific format output

```
docker stats --format "table \t\t\t"
```

No-stream (single snapshot)

```
docker stats --no-stream
```

All containers (including stopped)

```
docker stats --all
```

```

CONTAINER ID   NAME                CPU %      MEM USAGE / LIMIT   MEM
%      NET I/O       BLOCK I/O  PIDS
149d273a6168   flask-app           0.02%      20.5MiB / 3.489GiB   0.57
%      1.17kB / 126B   5.22MB / 127kB    1
798f3207dfb2   k3d-mycluster-serverlb 0.01%      1.637MiB / 3.489GiB   0.05
%      1.17kB / 126B   1.04MB / 8.19kB   3

```

Useful Format Options:

Custom format

```
docker stats --format "Container: | CPU: | Memory: "
```

JSON output

```
docker stats --format json --no-stream
```

Wide output

```
docker stats --no-stream --no-trunc
```

```

PERC}}\{{{.MemUsage}}\{{{.NetIO}}
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker stats --no-stream
--no-trunc
CONTAINER ID           NAME                CPU %      MEM USAGE / LIMIT   MEM %      NET I/O       BL
OCK I/O               PIDS
149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857   flask-app
0.02%      20.17MiB / 3.489GiB   0.56%      1.17kB / 126B   5.
22MB / 127kB    1
798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76   k3d-myclu
ster-serverlb 0.01%      2.219MiB / 3.489GiB   0.06%      1.04kB / 126B   1.
63MB / 4.1kB    3
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ |

```

Lab 2: docker top - Process Monitoring

View processes in container

```
docker top container-name
```

View with full command line

```
docker top container-name -ef
```

Compare with host processes

```
ps aux | grep docker
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker top flask-app
UID                PID                PPID                C
  STIME            TTY                TIME               CMD
root               2683              2659                0
  03:36            ?                  00:00:00           python app.p
y
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker top flask-app
UID                PID                PPID                C
  STIME            TTY                TIME               CMD
root               2683              2659                0
  03:36            ?                  00:00:00           python app.p
y
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ ps aux | grep docker
root        6590   0.6   0.4 1254164 16964 pts/4    Ssl+ 03:32   0:03 /mnt/wsl/do
cker-desktop/docker-desktop-user-distro proxy --distro-name Ubuntu --docker-
desktop-root /mnt/wsl/docker-desktop C:\Program Files\Docker\Docker\resource
s
palak      13015   0.2   0.8 2144072 30464 pts/0    TL   03:38   0:00 docker stat
s
palak      15403   0.2   0.7 2144072 29184 pts/0    TL   03:40   0:00 docker stat
s --format table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}
palak      18574   0.0   0.0   4088   1920 pts/0    S+   03:42   0:00 grep --colo
r=auto docker

```

Lab 3: docker logs - Application Logs

View Logs

```
docker logs container-name
```

Follow Logs (like tail -f)

```
docker logs -f container-name
```

Last N Lines

```
docker logs --tail 100 container-name
```

Logs with timestamps

```
docker logs -t container-name
```

Logs since specific time

```
docker logs --since 2024-01-15 container-name
```

Combine options

```
docker logs -f --tail 50 -t container-name
```

```

palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-docker$ docker logs flask-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-docker$ docker logs -f flask-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-docker$ docker logs --tail 100 flask-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-docker$ docker logs -t flask-app
2026-04-04T03:35:44.826146581Z * Serving Flask app 'app'
2026-04-04T03:35:44.826242078Z * Debug mode: off
2026-04-04T03:35:44.836472636Z WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2026-04-04T03:35:44.836533518Z * Running on all addresses (0.0.0.0)
2026-04-04T03:35:44.836542895Z * Running on http://127.0.0.1:5000
2026-04-04T03:35:44.836550416Z * Running on http://172.17.0.2:5000
2026-04-04T03:35:44.836557415Z Press CTRL+C to quit
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/flask-docker$ docker logs -f --tail 50 -t flask-app
2026-04-04T03:35:44.826146581Z * Serving Flask app 'app'
2026-04-04T03:35:44.826242078Z * Debug mode: off

```

Lab 4: Container Inspection

Detailed container info

```
docker inspect container-name
```

Specific information

```
docker inspect --format='{{.Name}}' container-name
```

```
docker inspect --format='{{.Image}}' container-name
```

```
docker inspect --format='{{.Labels}}' container-name
```

Resource Limits

```
docker inspect --format='{{.MemLimit}}' container-name
```

```
docker inspect --format='{{.CpuShares}}' container-name
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker inspect flask-app
[
  {
    "Id": "149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857",
    "Created": "2026-04-04T03:35:43.535816539Z",
    "Path": "python",
    "Args": [
      "app.py"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 2683,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-04-04T03:35:43.859126868Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:3efb7b4a43b0a7fce5522c0986f9f40c14bded401f2d89e7f331620a38b19f1e",
    "ResolvConfPath": "/var/lib/docker/containers/149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857/hostname",
    "HostsPath": "/var/lib/docker/containers/149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857/hosts",
    "LogPath": "/var/lib/docker/containers/149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857/149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857-json.log",
    "Name": "/flask-app",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": ""
  }
]

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker inspect --format='{{.State.Status}}' flask-app
running
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker inspect --format='{{.NetworkSettings.IPAddress}}' flask-app

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker inspect --format='{{.Config.Env}}' flask-app
[PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin LANG=C.UTF-8 GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568 PYTHON_VERSION=3.9.25 PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834]
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker inspect --format='{{.HostConfig.Memory}}' flask-app
0
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker inspect --format='{{.HostConfig.NanoCpus}}' flask-app
0
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$

```

Lab 5: Events Monitoring

Monitor Docker events in real-time

```
docker events
```

Filter events

```
docker events --filter 'type=container'
```

```
docker events --filter 'event=start'
```

```
docker events --filter 'event=die'
```

Since specific time

```
docker events --since '2024-01-15'
```

Format output

```
docker events --format ' '
```



```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker events
2026-04-04T03:51:08.622542688Z network disconnect b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, name=k3d-mycluster, type=bridge)
2026-04-04T03:51:08.634230132Z volume unmount k3d-mycluster-images (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, driver=local)
2026-04-04T03:51:08.647080000Z container die 798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76 (app=k3d, desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/6443/tcp=0.0.0.0:42837, desktop.docker.io/wsl-distro=Ubuntu, execDuration=9, exitCode=1, image=ghcr.io/k3d-io/k3d-proxy:5.8.3, k3d.cluster=mycluster, k3d.cluster.imageVolume=k3d-mycluster-images, k3d.cluster.network=k3d-mycluster, k3d.cluster.network.external=false, k3d.cluster.network.id=b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c, k3d.cluster.network.iprange=172.20.0.0/16, k3d.cluster.token=QCDEqfpSBJarwrLpYFUu, k3d.cluster.url=https://k3d-mycluster-server-0:6443, k3d.role=loadbalancer, k3d.server.loadbalancer=k3d-mycluster-serverlb, k3d.version=v5.8.3, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=k3d-mycluster-serverlb, org.opencontainers.image.created=2025-02-15T19:07:40.209Z, org.opencontainers.image.description=Little helper to run CNCF's k3s in Docker, org.opencontainers.image.licenses=MIT, org.opencontainers.image.revision=201b614ea6798f0ee9e75c63360eef891b36bb19, org.opencontainers.image.source=https://github.com/k3d-io/k3d, org.opencontainers.image.title=k3d, org.opencontainers.image.url=https://github.com/k3d-io/k3d, org.opencontainers.image.version=5)
2026-04-04T03:51:08.680222988Z volume mount k3d-mycluster-images (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, destination=/k3d/images, driver=local, propagation=, read/write=true)
2026-04-04T03:51:09.063259573Z network connect b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, name=k3d-mycluster, type=bridge)
2026-04-04T03:51:09.115199700Z container start 798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76 (app=k3d, desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/6443/tcp=0.0.0.0:42837, desktop.docker.io/wsl-distro=Ubuntu, image=ghcr.io/k3d-io/k3d-proxy:5.8.3, k3d.cluster=mycluster, k3d.cluster.imageVolume=k3d-mycluster-images, k3d.cluster.network=k3d-mycluster, k3d.cluster.network.external=false, k3d.cluster.network.id=b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c, k3d.cluster.network.iprange=172.20.0.0/16, k3d.cluster.token=QCDEqfpSBJarwrLpYFUu, k3d.cluster.url=https://k3d-mycluster-server-0:6443, k3d.role=loadbalancer, k3d.server.loadbalancer=k3d-mycluster-serverlb, k3d.version=v5.8.3, maintainer=NGINX Docker
```



```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker events --filter 'type=co
ntainer'
2026-04-04T03:51:32.256826600Z container die 798f3207dfb26bde1b64437f908cfd69820a5b
372911cbe79bd09dd6bf5f8a76 (app=k3d, desktop.docker.io/ports.scheme=v2, desktop.doc
ker.io/ports/6443/tcp=0.0.0.0:42837, desktop.docker.io/wsl-distro=Ubuntu, execDurat
ion=11, exitCode=1, image=ghcr.io/k3d-io/k3d-proxy:5.8.3, k3d.cluster=mycluster, k3d
.cluster.imageVolume=k3d-mycluster-images, k3d.cluster.network=k3d-mycluster, k3d.c
luster.network.external=false, k3d.cluster.network.id=b128b4abd3e5e015af20182a06dc
c3adaea02013c25d93652459ea60065bf96c, k3d.cluster.network.iprange=172.20.0.0/16, k3
d.cluster.token=QCDEqfpSBJarwrLpYFUu, k3d.cluster.url=https://k3d-mycluster-server-
0:6443, k3d.role=loadbalancer, k3d.server.loadbalancer=k3d-mycluster-serverlb, k3d.
version=v5.8.3, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=
k3d-mycluster-serverlb, org.opencontainers.image.created=2025-02-15T19:07:40.209Z,
org.opencontainers.image.description=Little helper to run CNCF's k3s in Docker, org
.opencontainers.image.licenses=MIT, org.opencontainers.image.revision=201b614ea6798
f0ee9e75c63360eef891b36bb19, org.opencontainers.image.source=https://github.com/k3d
-io/k3d, org.opencontainers.image.title=k3d, org.opencontainers.image.url=https://g
ithub.com/k3d-io/k3d, org.opencontainers.image.version=5)
2026-04-04T03:51:32.740611000Z container start 798f3207dfb26bde1b64437f908cfd69820a
5b372911cbe79bd09dd6bf5f8a76 (app=k3d, desktop.docker.io/ports.scheme=v2, desktop.d
ocker.io/ports/6443/tcp=0.0.0.0:42837, desktop.docker.io/wsl-distro=Ubuntu, image=g
hcr.io/k3d-io/k3d-proxy:5.8.3, k3d.cluster=mycluster, k3d.cluster.imageVolume=k3d-m
ycluster-images, k3d.cluster.network=k3d-mycluster, k3d.cluster.network.external=fa
lse, k3d.cluster.network.id=b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea6
0065bf96c, k3d.cluster.network.iprange=172.20.0.0/16, k3d.cluster.token=QCDEqfpSBJa
rwrLpYFUu, k3d.cluster.url=https://k3d-mycluster-server-0:6443, k3d.role=loadbalanc
er, k3d.server.loadbalancer=k3d-mycluster-serverlb, k3d.version=v5.8.3, maintainer=
NGINX Docker Maintainers <docker-maint@nginx.com>, name=k3d-mycluster-serverlb, org
.opencontainers.image.created=2025-02-15T19:07:40.209Z, org.opencontainers.image.de
scription=Little helper to run CNCF's k3s in Docker, org.opencontainers.image.licen
ses=MIT, org.opencontainers.image.revision=201b614ea6798f0ee9e75c63360eef891b36bb19
, org.opencontainers.image.source=https://github.com/k3d-io/k3d, org.opencontainers
.image.title=k3d, org.opencontainers.image.url=https://github.com/k3d-io/k3d, org.o
pencontainers.image.version=5)
2026-04-04T03:51:41.777870800Z container die 798f3207dfb26bde1b64437f908cfd69820a5b
372911cbe79bd09dd6bf5f8a76 (app=k3d, desktop.docker.io/ports.scheme=v2, desktop.doc
ker.io/ports/6443/tcp=0.0.0.0:42837, desktop.docker.io/wsl-distro=Ubuntu, execDurat
ion=9, exitCode=1, image=ghcr.io/k3d-io/k3d-proxy:5.8.3, k3d.cluster=mycluster, k3d
.cluster.imageVolume=k3d-mycluster-images, k3d.cluster.network=k3d-mycluster, k3d.c
luster.network.external=false, k3d.cluster.network.id=b128b4abd3e5e015af20182a06dc
c3adaea02013c25d93652459ea60065bf96c, k3d.cluster.network.iprange=172.20.0.0/16, k3d
.cluster.token=QCDEqfpSBJarwrLpYFUu, k3d.cluster.url=https://k3d-mycluster-server-0
:6443, k3d.role=loadbalancer, k3d.server.loadbalancer=k3d-mycluster-serverlb, k3d.v
ersion=v5.8.3, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=k
3d-mycluster-serverlb, org.opencontainers.image.created=2025-02-15T19:07:40.209Z, o
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker events --since '2024-01-15'
2026-04-04T03:45:55.252370700Z container die 798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76 (app=k3d, desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/6443/tcp=0.0.0.0:42837, desktop.docker.io/wsl-distro=Ubuntu, execDuration=9, exitCode=1, image=ghcr.io/k3d-io/k3d-proxy:5.8.3, k3d.cluster=mycluster, k3d.cluster.imageVolume=k3d-mycluster-images, k3d.cluster.network=k3d-mycluster, k3d.cluster.network.external=false, k3d.cluster.network.id=b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c, k3d.cluster.network.iprange=172.20.0.0/16, k3d.cluster.token=QCDEqfpSBJarwrLpYFUu, k3d.cluster.url=https://k3d-mycluster-server-0:6443, k3d.role=loadbalancer, k3d.server.loadbalancer=k3d-mycluster-serverlb, k3d.version=v5.8.3, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=k3d-mycluster-serverlb, org.opencontainers.image.created=2025-02-15T19:07:40.209Z, org.opencontainers.image.description=Little helper to run CNCF's k3s in Docker, org.opencontainers.image.licenses=MIT, org.opencontainers.image.revision=201b614ea6798f0ee9e75c63360eef891b36bb19, org.opencontainers.image.source=https://github.com/k3d-io/k3d, org.opencontainers.image.title=k3d, org.opencontainers.image.url=https://github.com/k3d-io/k3d, org.opencontainers.image.version=5)
2026-04-04T03:45:55.319173168Z volume mount k3d-mycluster-images (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, destination=/k3d/images, driver=local, propagation=, read/write=true)
2026-04-04T03:45:55.725154612Z network connect b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, name=k3d-mycluster, type=bridge)
2026-04-04T03:45:55.774844000Z container start 798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76 (app=k3d, desktop.docker.io/ports.scheme=v2, desktop.docker.io/ports/6443/tcp=0.0.0.0:42837, desktop.docker.io/wsl-distro=Ubuntu, image=ghcr.io/k3d-io/k3d-proxy:5.8.3, k3d.cluster=mycluster, k3d.cluster.imageVolume=k3d-mycluster-images, k3d.cluster.network=k3d-mycluster, k3d.cluster.network.external=false, k3d.cluster.network.id=b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c, k3d.cluster.network.iprange=172.20.0.0/16, k3d.cluster.token=QCDEqfpSBJarwrLpYFUu, k3d.cluster.url=https://k3d-mycluster-server-0:6443, k3d.role=loadbalancer, k3d.server.loadbalancer=k3d-mycluster-serverlb, k3d.version=v5.8.3, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=k3d-mycluster-serverlb, org.opencontainers.image.created=2025-02-15T19:07:40.209Z, org.opencontainers.image.description=Little helper to run CNCF's k3s in Docker, org.opencontainers.image.licenses=MIT, org.opencontainers.image.revision=201b614ea6798f0ee9e75c63360eef891b36bb19, org.opencontainers.image.source=https://github.com/k3d-io/k3d, org.opencontainers.image.title=k3d, org.opencontainers.image.url=https://github.com/k3d-io/k3d, org.opencontainers.image.version=5)
2026-04-04T03:46:07.115518516Z network disconnect b128b4abd3e5e015af20182a06dcc3adaea02013c25d93652459ea60065bf96c (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, name=k3d-mycluster, type=bridge)
2026-04-04T03:46:07.125964863Z volume unmount k3d-mycluster-images (container=798f3207dfb26bde1b64437f908cfd69820a5b372911cbe79bd09dd6bf5f8a76, destination=/k3d/images, driver=local, propagation=, read/write=true)
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker events --format '{{.Type}} {{.Action}} {{.Actor.Attributes.name}}'
network connect k3d-mycluster
container start k3d-mycluster-serverlb

```

Lab 6: Practical Monitoring Script

```

#!/bin/bash
# monitor.sh - Simple Docker monitoring

echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)"
echo

echo "1. Running Containers:"
docker ps --format "table \t\t"
echo

```

```

echo "2. Resource Usage:"
docker stats --no-stream --format "table \t\t\t\t\t"
echo

echo "3. Recent Events:"
docker events --since '5m' --until '0s' --format ' ' | tail -5
echo

echo "4. System Info:"
docker system df

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ #!/bin/bash
# monitor.sh - Simple Docker monitoring

echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)"
echo
=== Docker Monitoring Dashboard ===
Time: Sat Apr  4 03:54:05 UTC 2026

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ echo "1. Running Containers:"
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
echo
1. Running Containers:
NAMES                STATUS                PORTS
flask-app             Up 18 minutes        0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
k3d-mycluster-serverlb Restarting (1) 2 seconds ago

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ echo "2. Resource Usage:"
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}\t{{.BlockIO}}"
echo
2. Resource Usage:
NAME                CPU %    MEM USAGE / LIMIT    NET I/O    BLOCK I/O
flask-app           0.02%    19.48MiB / 3.489GiB   1.17kB / 126B  5.52MB / 127kB
k3d-mycluster-serverlb 0.01%    1.445MiB / 3.489GiB   690B / 84B    721kB / 4.1kB

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ echo "3. Recent Events:"
docker events --since '5m' --until '0s' --format '{{.Time}} {{.Type}} {{.Action}}' | tail -5
echo
3. Recent Events:
1775274864 volume unmount
1775274864 container die
1775274864 volume mount
1775274865 network connect
1775274865 container start

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ echo "4. System Info:"
docker system df
4. System Info:
TYPE                TOTAL        ACTIVE        SIZE        RECLAIMABLE
Images              14           12           10.81GB     5.219GB (48%)

```

Part 4: Docker Networks

Lab 1: Understanding Docker Network Types

List Networks

```

# Default networks
docker network ls

# Output:
# NETWORK ID        NAME          DRIVER          SCOPE
# abc123             bridge        bridge          Local

```

```
# def456      host      host      Local
# ghi789      none     null     Local
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
c3d63f648680        bridge              bridge              local
9e97d994ffc9        docker_gwbridge     bridge              local
4e98a639393a        host                host                local
bbdnr2mkx4ps        ingress             overlay             swarm
b128b4abd3e5        k3d-mycluster       bridge              local
cab046070ae6        macvlan_net         macvlan             local
fcce7893acdb        my_bridge           bridge              local
45b63b4f356d        none                null                local
```

Lab 2: Network Types Explained

1. Bridge Network (Default)

```
# Containers on bridge network can communicate
# Each container gets own IP, isolated from host
```

```
# Create custom bridge network
docker network create my-network
```

```
# Inspect network
docker network inspect my-network
```

```
# Run containers on custom network
docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx
```

```
# Containers can communicate using container names
docker exec web1 curl http://web2
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
c3d63f648680        bridge              bridge              local
9e97d994ffc9        docker_gwbridge     bridge              local
4e98a639393a        host                host                local
bbdnr2mkx4ps        ingress             overlay             swarm
b128b4abd3e5        k3d-mycluster       bridge              local
cab046070ae6        macvlan_net         macvlan             local
fcce7893acdb        my_bridge           bridge              local
45b63b4f356d        none                null                local
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker network create my-network
f65c99f04f7d5cc14db622d65ad345d51759ab718696d8b6cf51600d44da79e5
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker network inspect my-network
[
  {
    "Name": "my-network",
    "Id": "f65c99f04f7d5cc14db622d65ad345d51759ab718696d8b6cf51600d44da79e5",
    "Created": "2026-04-04T03:55:19.257630365Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.21.0.0/16",
          "Gateway": "172.21.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {}
  }
]

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker run -d --name web1 --network my-network nginx
6d163672a6bcfd9d632acf5350670547ca220c3977028fa91b2d5158ad3777
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/flask-docker$ docker run -d --name web2 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web2" is already in use by container "32b1e567b876c41ebda73f837bad5f427250cf6516033bcd9c8a1ac193898ce3". You have to remove (or rename) that container to be able to reuse that name.

```

2. Host Network

```

# Container uses host's network directly
# No network isolation, shares host's IP

```

```
docker run -d --name host-app --network host nginx
```

```

# Access directly on host port 80
curl http://localhost

```


3. None Network

No network access

```
docker run -d --name isolated-app --network none alpine sleep 3600
```

Test - no network interfaces

```
docker exec isolated-app ifconfig
```

Only Loopback interface

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d --name isolated-app --network none alpine sleep 3600
docker: Error response from daemon: Conflict. The container name "/isolated-app" is already in use by container "fc1b2cd
b298b9f87406a6d614615a609cc167c7de27b370d50258d76aad6379f". You have to remove (or rename) that container to be able to
reuse that name.

Run 'docker run --help' for more information
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker exec isolated-app ifconfig
Error response from daemon: container fc1b2cdb298b9f87406a6d614615a609cc167c7de27b370d50258d76aad6379f is not running
```

4. Overlay Network (Swarm)

For Docker Swarm - multi-host networking

```
docker network create --driver overlay my-overlay
```

```
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker network create --driver overlay my-overlay
Error response from daemon: network with name my-overlay already exists
```

Lab 3: Network Management Commands

Create network

```
docker network create app-network
```

```
docker network create --driver bridge --subnet 172.20.0.0/16 --gateway 172.20.0.1 my-subne
```

Connect container to network

```
docker network connect app-network existing-container
```

Disconnect container from network

```
docker network disconnect app-network container-name
```

Remove network

```
docker network rm network-name
```

Prune unused networks

```
docker network prune
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network create app-network
Error response from daemon: network with name app-network already exists
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network create --driver bridge --subnet 172.20.0.0/16 --gateway 172.20.0.1
my-subnet
Error response from daemon: invalid pool request: Pool overlaps with other one on this address space
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network connect app-network existing-container
Error response from daemon: No such container: existing-container
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network disconnect app-network container-name
Error response from daemon: No such container: container-name
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network disconnect app-network flask-app
Error response from daemon: container 149d273a61682453a3926073aad6622c0bf054dd80250082926f4302be2d3857 is not connected
to network app-network
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network rm app-network
app-network
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network prune
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
my-overlay
macvlan_net
my_bridge
app-networ

```

Lab 4: Multi-Container Application Example

Web App + Database Communication

Create network

```
docker network create app-network
```

Start database

```

docker run -d \
  --name postgres-db \
  --network app-network \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15

```

Start web application

```

docker run -d \
  --name web-app \
  --network app-network \
  -p 8080:3000 \
  -e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
  -e DATABASE_HOST="postgres-db" \
  node-app

```

Web app can connect to database using "postgres-db" hostname

```

palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker network create app-network
b335ac149a864da16a1b1354a03aaa94df70565a44b395a8b2b6ff2259fe65d6
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  --name postgres-db \
  --network app-network \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
a8a0725a32a6: Pull complete
9328c154beae: Pull complete
af0c3ea2fe5e: Pull complete
5c152148754e: Pull complete
8ecd9bd190da6: Pull complete
b4765ac2e7e6: Pull complete
f7049a93e93f: Pull complete
c45b346571fc: Pull complete
5ca81a59b0a4: Pull complete
00f33d580dc3: Pull complete
2c38b75ffca2: Pull complete
5435b2dcdf5c: Pull complete
af2a6e5f285c: Pull complete
71a32c7571f4: Pull complete
b9015a4f6b87: Download complete
cebcba07837b: Download complete
Digest: sha256:31224c60946ea75ee9522e0f4d02eb202e74754adea4294fdefe9f93c875b657
Status: Downloaded newer image for postgres:15
193122a402852fdcec5366451a31342763c1f2b1120f66b05ef3fada5b9b9514
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$

```

```

palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
  --name web-app \
  --network app-network \
  -p 8080:3000 \
  -e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
  -e DATABASE_HOST="postgres-db" \
  my-node-app
e34b3bc96974c2f68066f471e27f630e54d9707bf03e212d9a1506654ba4d83f

```

Lab 5: Network Inspection & Debugging

Inspect network

```
docker network inspect bridge
```

Check container IP

```
docker inspect --format='{{.IPAddr}}' container-name
```

DNS resolution test

```
docker exec container-name nslookup another-container
```

Network connectivity test

```
docker exec container-name ping -c 4 google.com
```

```
docker exec container-name curl -I http://another-container
```

View network ports

```
docker port container-name
```



```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network inspect bridge
[
  {
    "Name": "bridge",
    "Id": "74c464e2a5450f765d125cbaccff07bb9618be260c8a87b57a090c49606a0b42",
    "Created": "2026-04-10T06:41:19.330333488Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",
      "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
      "com.docker.network.bridge.name": "docker0",
      "com.docker.network.driver.mtu": "1500"
    },
    "Labels": {},
    "Containers": {}
  }
]

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web-app

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker exec web-app nslookup postgres
-db
Server:                127.0.0.11
Address:                127.0.0.11#53

Non-authoritative answer:
Name:   postgres-db
Address: 172.18.0.2

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker exec web-app ping -c 4 google.
com
PING google.com (142.250.207.206) 56(84) bytes of data.
64 bytes from 142.250.207.206: icmp_seq=1 ttl=63 time=95.5 ms
64 bytes from 142.250.207.206: icmp_seq=2 ttl=63 time=226 ms
64 bytes from 142.250.207.206: icmp_seq=3 ttl=63 time=108 ms
64 bytes from 142.250.207.206: icmp_seq=4 ttl=63 time=87.5 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3095ms
rtt min/avg/max/mdev = 87.514/129.244/225.787/56.223 ms

```

Lab 6: Port Publishing vs Exposing

PORT PUBLISHING (host:container)

```
docker run -d -p 80:8080 --name app1 nginx
```

Host port 80 → Container port 8080

Dynamic port publishing

```
docker run -d -p 8080 --name app2 nginx
```

Docker assigns random host port

Multiple ports

```
docker run -d -p 80:80 -p 443:443 --name app3 nginx
```

Specific host IP

```
docker run -d -p 127.0.0.1:8080:80 --name app4 nginx
```

EXPOSE in Dockerfile (metadata only)

Dockerfile: EXPOSE 80

Still need -p to publish

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d -p 80:8080 --name app1
nginx
e7af83364972bb5cb97d148c0428a24ea86ef9b98ab83a1a8f8d1cf0a1872f91

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d -p 8080 --name app2 ngi
nx
673c2bbbc787ef77b5328031fa520bb823f942986952c71d9755f146b293bef

```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d -p 8080:80 -p 8443:443
--name app3 nginx
6aa4a3dd3067b7010bbd327a2bdd20161e0d45be162e370a7cec60e318580347
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d -p 127.0.0.1:8080:80 --
name app4 nginx
55c791965439e08240fdcfd4ce9d22a978f54145fabb19df3fdc83d59657f45a

```

Part 5: Complete Real-World Example

Application Architecture:

- Flask Web App (port 5000)
- PostgreSQL Database (port 5432)
- Redis Cache (port 6379)
- All connected via custom network

Implementation:

1. Create network

```
docker network create myapp-network
```

2. Start database with volume

```
docker run -d \  
  --name postgres \  
  --network myapp-network \  
  -e POSTGRES_PASSWORD=mysecretpassword \  
  -e POSTGRES_DB=mydatabase \  
  -v postgres-data:/var/lib/postgresql/data \  
  postgres:15
```

3. Start Redis

```
docker run -d \  
  --name redis \  
  --network myapp-network \  
  -v redis-data:/data \  
  redis:7-alpine
```

4. Start Flask app with all configurations

```
docker run -d \  
  --name flask-app \  
  --network myapp-network \  
  -p 5000:5000 \  
  -v $(pwd)/app:/app \  
  -v app-logs:/var/log/app \  
  -e DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \  
  -e REDIS_URL="redis://redis:6379" \  
  -e DEBUG="false" \  
  -e LOG_LEVEL="INFO" \  
  --env-file .env.production \  
  flask-app:latest
```

```

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network create myapp-network
f4117bb3d2bf69f07f6eb7b37ad507c36c1e3486ab7632c5270dd962fe38d969
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
--name postgres \
--network myapp-network \
-e POSTGRES_PASSWORD=mysecretpassword \
-e POSTGRES_DB=mydatabase \
-v postgres-data:/var/lib/postgresql/data \
postgres:15
1a2cb50be907687f3f716042218847b547139743b69e42667e615ddc7a575ede

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d \
--name redis \
--network myapp-network \
-v redis-data:/data \
redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
4f4fb700ef54: Pull complete
09a5a0c32a23: Pull complete
9fe3744a2eac: Pull complete
7ad54b3c4cef: Pull complete
bd53938b1271: Pull complete
2a97533d89ca: Pull complete
bc1da058f299: Pull complete
1c2c8d1ee428: Pull complete
06cd2b88db40: Download complete
9bbcf5a5d5580: Download complete
Digest: sha256:8b81dd37ff027bec4e516d41acfbef9fe2460070dc6d4a4570a2ac5b9d59df
065
Status: Downloaded newer image for redis:7-alpine
9f0030ded1f8aa2d805c6ed204175992799747f754d9a87c5dd32490a9454ffa

palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker run -d --name flask-app --
network myapp-network -p 5000:5000 -e DATABASE_URL="postgresql://postgre
s:mysecretpassword@postgres:5432/mydatabase" -e REDIS_URL="redis://redis:6
379" flask-app:latest
e05cefb08b58f9ae1f0e87694154e32d21fc74f5257f0e4d5a3e521e3bc42550

```

Monitoring Commands:

Check all components

```
docker ps
```

Monitor resources

```
docker stats postgres redis flask-app
```

Check Logs

```
docker logs -f flask-app
```

Network connectivity test

```
docker exec flask-app ping -c 2 postgres
```

```
docker exec flask-app ping -c 2 redis
```

View network details

```
docker network inspect myapp-network
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O
36b682f0cb98	postgres	0.04%	17.09MiB / 3.489GiB	0.48%	1.95kB
/ 126B	139MB / 68.1MB	6			
6b8f720e828b	redis	0.69%	3.523MiB / 3.489GiB	0.10%	998B /
126B	32.6MB / 164kB	6			
489486dd1a3d	flask-app	0.02%	19.55MiB / 3.489GiB	0.55%	872B /
126B	29.6MB / 127kB	1			

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker logs -f flask-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.22.0.2:5000
Press CTRL+C to quit
```

```
palak@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ docker network inspect myapp-network
[
  {
    "Name": "myapp-network",
    "Id": "b7b7ecc95f198db40aa23ff8a3e3b8756a21cdb6e203b07433c69fa9334e7d44",
    "Created": "2026-04-10T17:03:07.49274786Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.22.0.0/16",
          "Gateway": "172.22.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    }
  }
]
```

conclusion

1. **Volumes** persist data beyond container lifecycle
2. **Environment variables** configure containers dynamically
3. **Monitoring commands** help debug and optimize containers
4. **Networks** enable secure container communication
5. **Always use named volumes** for production data

6. **Custom networks** provide better isolation and DNS
7. **Monitor resource usage** to prevent issues
8. **Use .env files** for sensitive configuration

Cloud-containerization-and-DevOps

Experiment 6 A

Title: Comparison of Docker Run and Docker Compose

PART A – THEORY

1. Objective

To understand the relationship between `docker run` and Docker Compose, and to compare their configuration syntax and use cases.

Example:

```
docker run -d \  
  --name my-nginx \  
  -p 8080:80 \  
  -v ./html:/usr/share/nginx/html \  
  -e NGINX_HOST=localhost \  
  --restart unless-stopped \  
  nginx:alpine
```



```

palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker run -d \
--name my-nginx \
-p 8080:80 \
-v ./html:/usr/share/nginx/html \
-e NGINX_HOST=localhost \
--restart unless-stopped \
nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
15e759724ff6: Pull complete
ff9f59a6a62e: Pull complete
f03becc8ac15: Pull complete
c8a2fa3a88d2: Pull complete
1165b869c51a: Pull complete
34dfdd2ef1f9: Pull complete
a71873b303e8: Pull complete
6057002906d9: Download complete
1116431dccff: Download complete
Digest: sha256:582c496ccf79d8aa6f8203a79d32aaf7fffd8b13362c60a701a2f9ac64886c93d
Status: Downloaded newer image for nginx:alpine
d915dfd48facdea7e2bfff62de860d7439373ca441c5e14c2e2741138544a9958

```

2.2 Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes in a structured format.

```
docker compose up -d
```

```

palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker compose up -d
WARN[0001] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
  ✓ Network docker-lab_default Created 37.8s
  ✓ Container lab-nginx Started 12.6s
palak@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker compose ps
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

```

NAME	IMAGE	PORTS	COMMAND	SERVICE	CREATED
lab-nginx	nginx:alpine	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	"/docker-entrypoint..."	nginx	19 seconds ago

Compose is **declarative**, meaning you define the desired state of the application.

Equivalent Compose file:

```
version: '3.8'
```

```
services:
```

```
nginx:
  image: nginx:alpine
  container_name: my-nginx
  ports:
    - "8080:80"
  volumes:
    - ./html:/usr/share/nginx/html
  environment:
    NGINX_HOST: localhost
  restart: unless-stopped
```

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    environment:
      NGINX_HOST: localhost
    restart: unless-stopped
```

3. Mapping: Docker Run vs Docker Compose

4. Advantages of Docker Compose

1. Simplifies multi-container applications
2. Provides reproducibility
3. Version controllable configuration
4. Unified lifecycle management
5. Supports service scaling

Example:

```
docker compose up --scale web=3
```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose up
p --scale nginx=3
[+] up 3/3
  ✓ Container docker-lab-nginx-3 Created 1.0s
  ✓ Container docker-lab-nginx-1 Created 1.1s
  ✓ Container docker-lab-nginx-2 Created 1.0s
Attaching to nginx-1, nginx-2, nginx-3
nginx-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
nginx-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
nginx-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
nginx-1 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
nginx-1 | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
nginx-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
nginx-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
nginx-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
nginx-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: using the "epoll" event method
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: nginx/1.29.5
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker processes
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker process 29
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker process 30
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker process 31
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker process 32
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker process 33
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker process 34
nginx-1 | 2026/04/13 08:27:07 [notice] 1#1: start worker process 35

```

PART B – PRACTICAL TASK

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

Execute:

```
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
```

Verify:

```
docker ps
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
d1b65ee13844cc052e7ee26f611d2f1925112550650269139185fad3b0c79f9c
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
d1b65ee13844	nginx:alpine	"/docker-entrypoint..."	10 seconds ago
796ed73fc7d7	nginx	"/docker-entrypoint..."	About a minute ago
ad0520f20bca	nginx	"/docker-entrypoint..."	About a minute ago
f3ce388a0424	nginx	"/docker-entrypoint..."	About a minute ago
1f60224ba459	9ae764d507bc	"python app.py"	18 hours ago
e96fbf887134	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	19 hours ago
be5526f83dfc	wordpress:latest	"docker-entrypoint.s..."	20 hours ago
213c9de46f67	wordpress:latest	"docker-entrypoint.s..."	20 hours ago
257a2c74a705	wordpress:latest	"docker-entrypoint.s..."	20 hours ago

Access:

```
http://localhost:8081
```

Stop and remove container:

```
docker stop lab-nginx
docker rm lab-nginx
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker stop lab-nginx
lab-nginx
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker rm lab-nginx
lab-nginx
```

Step 2: Run Same Setup Using Docker Compose

Create `docker-compose.yml` :

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Run:

```
docker compose up -d
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose up
p -d
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 4/4
✔Container docker-lab-nginx-3 Removed 0.9s
✔Container docker-lab-nginx-2 Removed 1.1s
✔Container docker-lab-nginx-1 Recreated 1.6s
✔Container lab-nginx Started 0.7s
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ |
```

Verify:

```
docker compose ps
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose ps
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED
lab-nginx           nginx:alpine        "/docker-entrypoint..."  nginx      About a minute ago
Up 59 seconds      0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
```

Stop:

```
docker compose down
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose down
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✔Container lab-nginx Removed 0.7s
✔Network docker-lab_default Removed 0.5s
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ |
```

Task 2: Multi-Container Application

Objective:

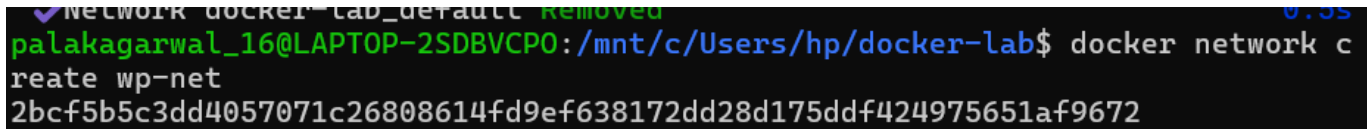
Deploy WordPress with MySQL using:

1. Docker Run (manual way)
2. Docker Compose (structured way)

A. Using Docker Run

1. Create network:

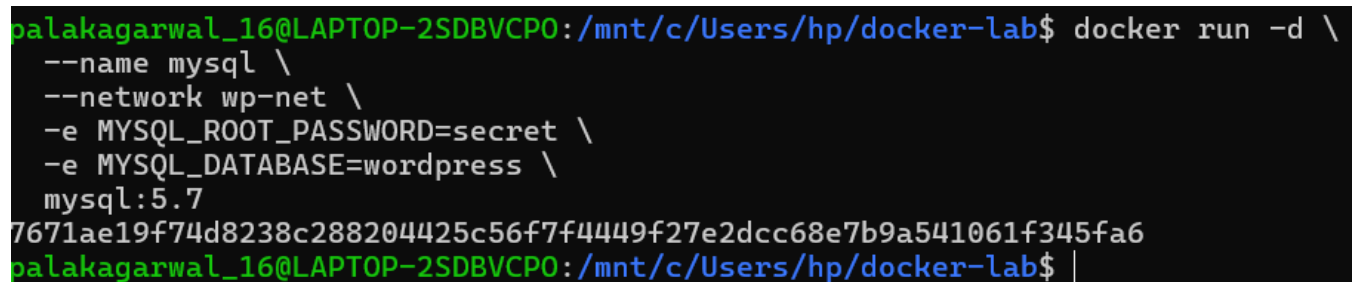
```
docker network create wp-net
```



```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker network create wp-net
2bcf5b5c3dd4057071c26808614fd9ef638172dd28d175ddf424975651af9672
```

1. Run MySQL:

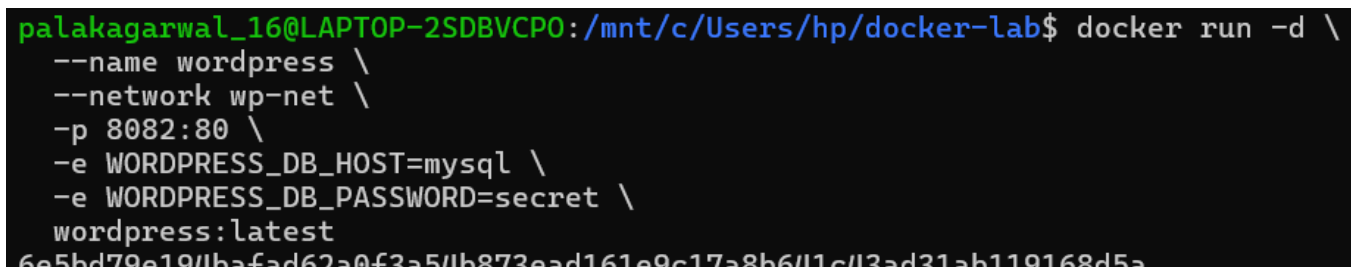
```
docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
```



```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
7671ae19f74d8238c288204425c56f7f4449f27e2dcc68e7b9a541061f345fa6
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$
```

1. Run WordPress:

```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```



```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
6e5bd79e194baefad62a0f3a54b873ead161e9c17a8b641c43ad31ab119168d5a
```

Test:

`http://localhost:8082`

B. Using Docker Compose

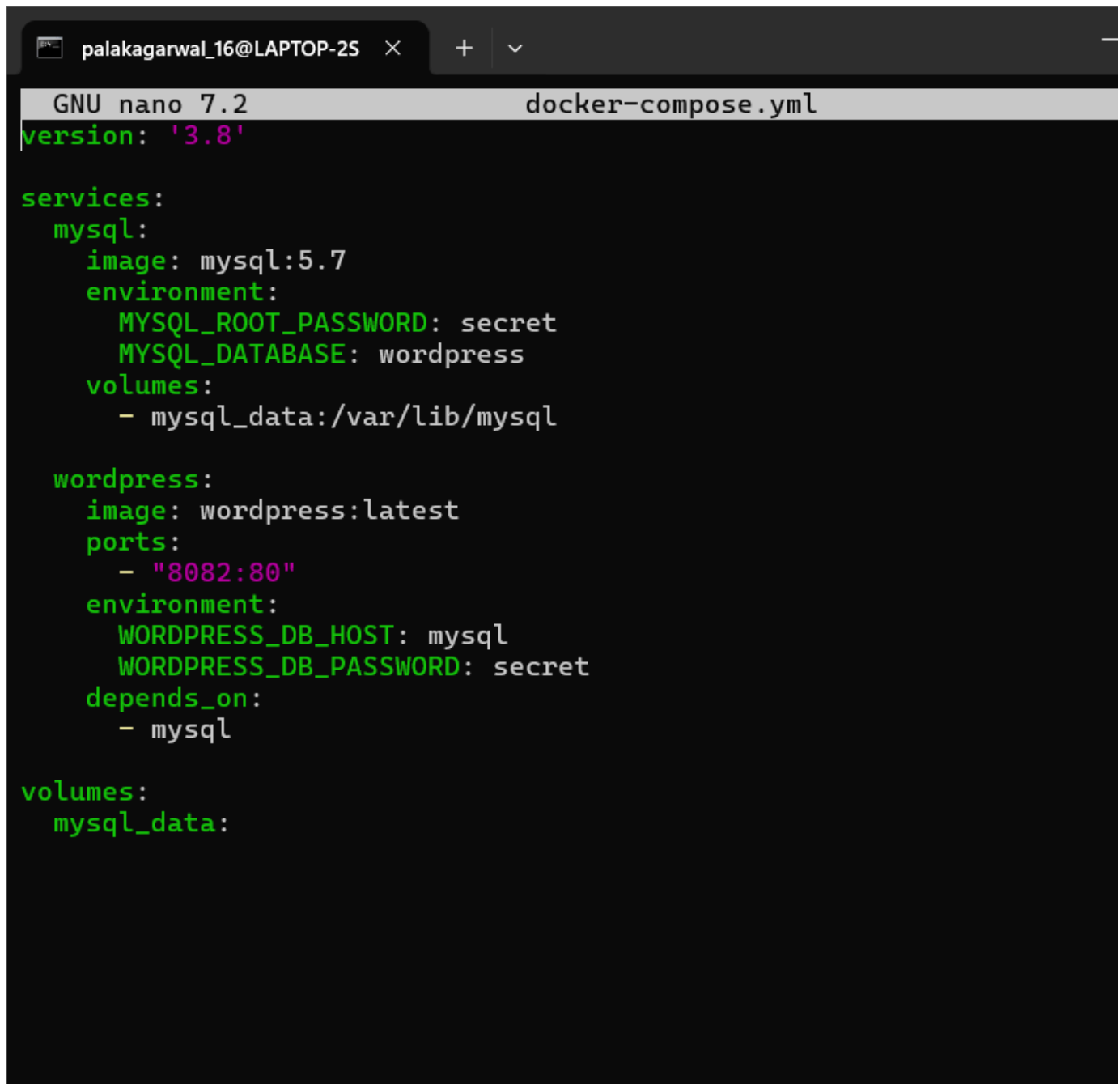
Create `docker-compose.yml` :

```
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:
```



The screenshot shows a terminal window with a dark background. At the top, the browser tab is labeled 'palakagarwal_16@LAPTOP-25'. The terminal title bar indicates 'GNU nano 7.2' and the file being edited is 'docker-compose.yml'. The content of the file is as follows:

```
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:
```

Run:

```
docker compose up -d
```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose up
p -d
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `ver
sion` is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
[+] up 2/3
✓Network docker-lab_default      Created      0.1s
✓Container docker-lab-mysql-1    Started     1.4s
✎ Container docker-lab-wordpress-1 Starting      1.8s
Error response from daemon: failed to set up container networking: driver fa
iled programming external connectivity on endpoint docker-lab-wordpress-1 (4
b24f8da1de3d1e70188a5233e49bd9293fbfa1b3840f1f3f9acc00a5b09066d): failed to
bind host port 0.0.0.0:8082/tcp: address already in use
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ |

```

Stop:

```
docker compose down -v
```

PART C – CONVERSION & BUILD-BASED TASKS

Task 3: Convert Docker Run to Docker Compose

Problem 1: Basic Web Application

Given Docker Run Command:

```

docker run -d \
  --name webapp \
  -p 5000:5000 \
  -e APP_ENV=production \
  -e DEBUG=false \
  --restart unless-stopped \
  node:18-alpine

```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker run -d \
  --name webapp \
  -p 5001:5000 \
  -e APP_ENV=production \
  -e DEBUG=false \
  --restart unless-stopped \
  node:18-alpine
70a34793dd7b38ec8c4f476d72ed8e435576b93e0ff7b86e2ce2482fa417ca7a
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ |

```

Student Task:

1. Write an equivalent `docker-compose.yml`
2. Ensure:
 - o Same container name
 - o Same port mapping
 - o Same environment variables
 - o Same restart policy

3. Run using:

```
docker compose up -d
```

4. Verify using:

```
docker compose ps
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker compose up
p -d
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `ver
sion` is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
[+] up 3/4
✓Network docker-lab_default          Created          0.1s
✓Volume docker-lab_mysql_data        Created          0.0s
✓Container docker-lab-mysql-1        Started          1.3s
❗ Container docker-lab-wordpress-1 Starting          1.8s
Error response from daemon: failed to set up container networking: driver fa
iled programming external connectivity on endpoint docker-lab-wordpress-1 (3
b57598b53d475e3be50430d1e50484e1c429fcac7aab3ddc976d6648fb5c257): failed to
bind host port 0.0.0.0:8082/tcp: address already in use
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker compose p
s
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute `ver
sion` is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
NAME                STATUS              IMAGE                COMMAND              SERVICE    CREATED
docker-lab-mysql-1  Up 6 seconds       mysql:5.7            "docker-entrypoint.s..."  mysql      7 second
s ago
ports: 3306/tcp, 33060/tcp
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ |
```

Problem 2: Volume + Network Configuration

Given Docker Run Commands:

```
docker network create app-net
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker network create app-net
d29b38c518fa0911588bd9d639a60f2075d985d0b87fde45355775ac4ecd9a90
```

```
docker run -d \
  --name postgres-db \
  --network app-net \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker run -d \
  --name postgres-db \
  --network app-net \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15
42d292bfc555e2ab22aa6f0d7fdd7f97f65e76c65e9d50b75957ab133024ad76
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$
```

```
docker run -d \
  --name backend \
  --network app-net \
  -p 8000:8000 \
  -e DB_HOST=postgres-db \
  -e DB_USER=admin \
  -e DB_PASS=secret \
  python:3.11-slim
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker run -d \
  --name backend \
  --network app-net \
  -p 8000:8000 \
  -e DB_HOST=postgres-db \
  -e DB_USER=admin \
  -e DB_PASS=secret \
  python:3.11-slim
Unable to find image 'python:3.11-slim' locally
3.11-slim: Pulling from library/python
37ffa6577b04: Pull complete
d98f36ba2159: Pull complete
0816ee351b60: Pull complete
6f70bfdcb954: Download complete
79505fc62d4b: Download complete
Digest: sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
Status: Downloaded newer image for python:3.11-slim
8a73ea8486bf99cec05d97c51f93c854e84e8ed3cba8a8e9d6cc11afb19ca02a
```

Student Task:

1. Create a single `docker-compose.yml` file that:
 - Defines both services
 - Creates named volume `pgdata`
 - Creates custom network `app-net`
 - Uses `depends_on`
2. Bring up services using one command.
3. Stop and remove everything properly.

```
GNU nano 7.2 docker-compose.yml *
services:
  app:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5001:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
      DB_HOST: db
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASSWORD: postgres
      DB_NAME: mydb
    depends_on:
      - db
    networks:
      - app-net
    command: >
      node -e "require('http').createServer((req,res)=>res.end('Hello from >
    restart: unless-stopped

  db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net
    restart: unless-stopped

volumes:
  pgdata:
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose up
p -d
WARN[0000] Found orphan containers ([docker-lab-wordpress-1 docker-lab-mysql-1]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
[+] up 2/2
✔Container webapp Started 1.4s
✔Container postgres-db Started 0.5s
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$
```

Task 4: Resource Limits Conversion

Given Docker Run Command:

```
docker run -d \
  --name limited-app \
  -p 9000:9000 \
  --memory="256m" \
  --cpus="0.5" \
  --restart always \
  nginx:alpine
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker run -d \
  --name limited-app \
  -p 9000:9000 \
  --memory="256m" \
  --cpus="0.5" \
  --restart always \
  nginx:alpine
00cf4b448598bdf1e53575282cd1f99e502d90f71ca8fddb95923be546455df7

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ nano docker-compose.yml
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose up
p -d
WARN[0000] Found orphan containers ([webapp postgres-db docker-lab-wordpress-1 docker-lab-mysql-1]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
[+] up 1/1
✔Container limited-app Started 1.0s
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$
```

Student Task:

1. Convert this to Docker Compose.
2. Add resource limits using:


```
deploy:  
  resources:  
    limits:
```

3. Explain:

- When deploy works
- Difference between normal Compose mode and Swarm mode

```
palakagarwal_16@LAPTOP-2S  ×  +  ∨
GNU nano 7.2                docker-compose.yml *
services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:80"
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 256M
    restart: always|

palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker stack dep
loy -c docker-compose.yml mystack
Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the backgro
und.
In a future release, --detach=false will become the default.
Creating network mystack_default
Creating service mystack_limited-app
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker stack ser
vices mystack
ID                NAME                MODE                REPLICAS    IMAGE
PORTS
uu76sork4cye     mystack_limited-app  replicated         1/1          nginx:alpine
*:9000->80/tcp
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ |
```

PART D – USING DOCKERFILE INSTEAD OF STANDARD IMAGE

image: node:18-alpine

Students must:

- Create their own Dockerfile
- Build image using Compose
- Run container from that build

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker compose up
p --build -d
WARN[0000] Found orphan containers ([webapp postgres-db docker-lab-wordpress
-1 docker-lab-mysql-1]) for this project. If you removed or renamed this ser
vice in your compose file, you can run this command with the --remove-orphan
s flag to clean it up.
[+] up 1/1
✓Container limited-app Started 1.9s
```

Task 5: Replace Standard Image with Dockerfile (Node App)

Scenario

You are given:

```
docker run -d -p 3000:3000 node:18-alpine
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker run -d -p
3000:3000 node:18-alpine
5c8ef958bd030683217f47a0c5e34b455e1a2932e1a295d2349679c32b8c2999
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$
```

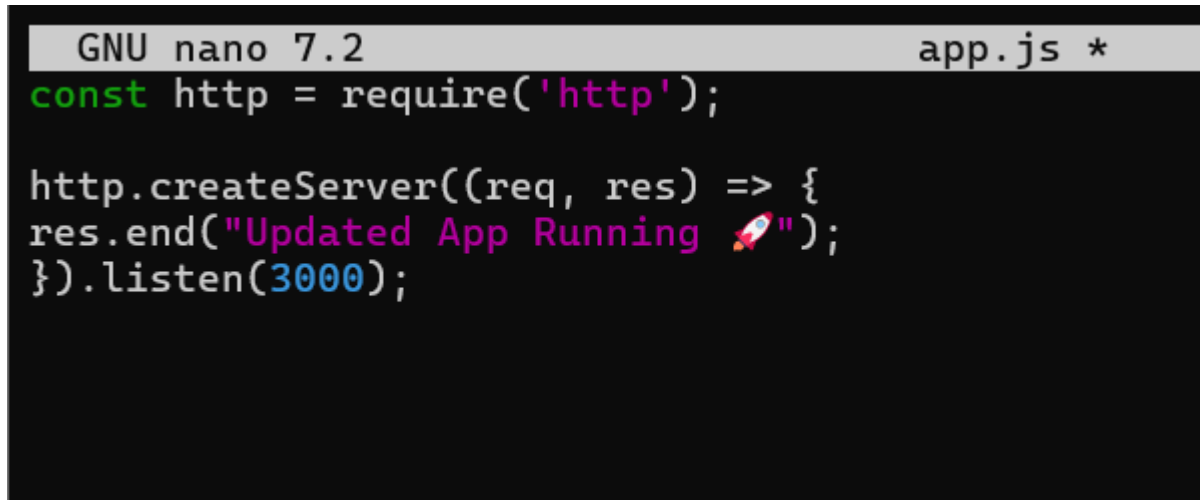
Now instead of directly using node:18-alpine , you must:

1. Create a simple Node.js app
2. Write a Dockerfile
3. Use Docker Compose build: option

Step 1: Create app.js

```
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```



```
GNU nano 7.2 app.js *
const http = require('http');

http.createServer((req, res) => {
  res.end("Updated App Running 🚀");
}).listen(3000);
```

Step 2: Create Dockerfile

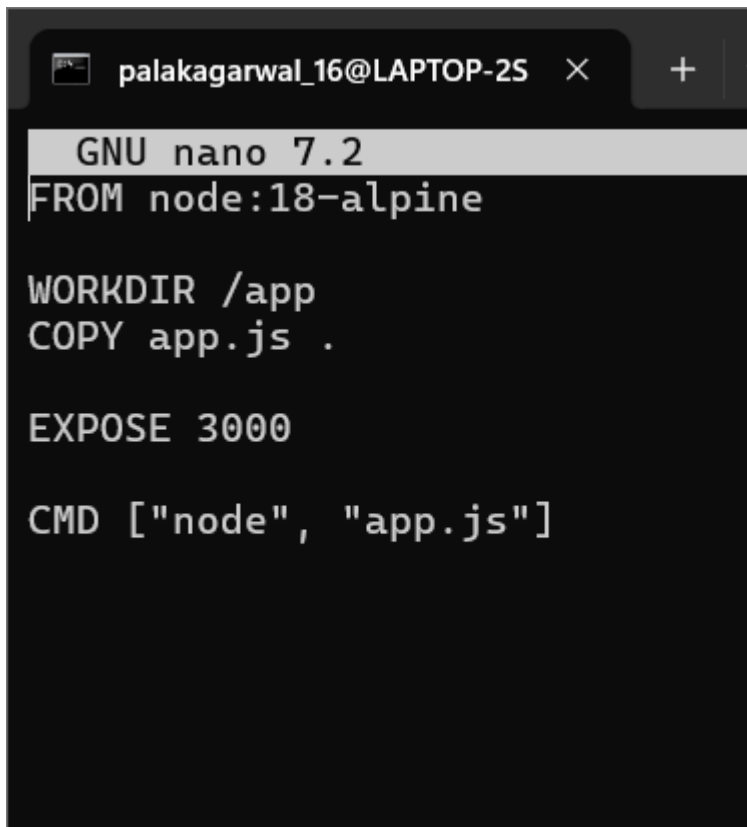
```
FROM node:18-alpine

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```



```
GNU nano 7.2
FROM node:18-alpine

WORKDIR /app
COPY app.js .

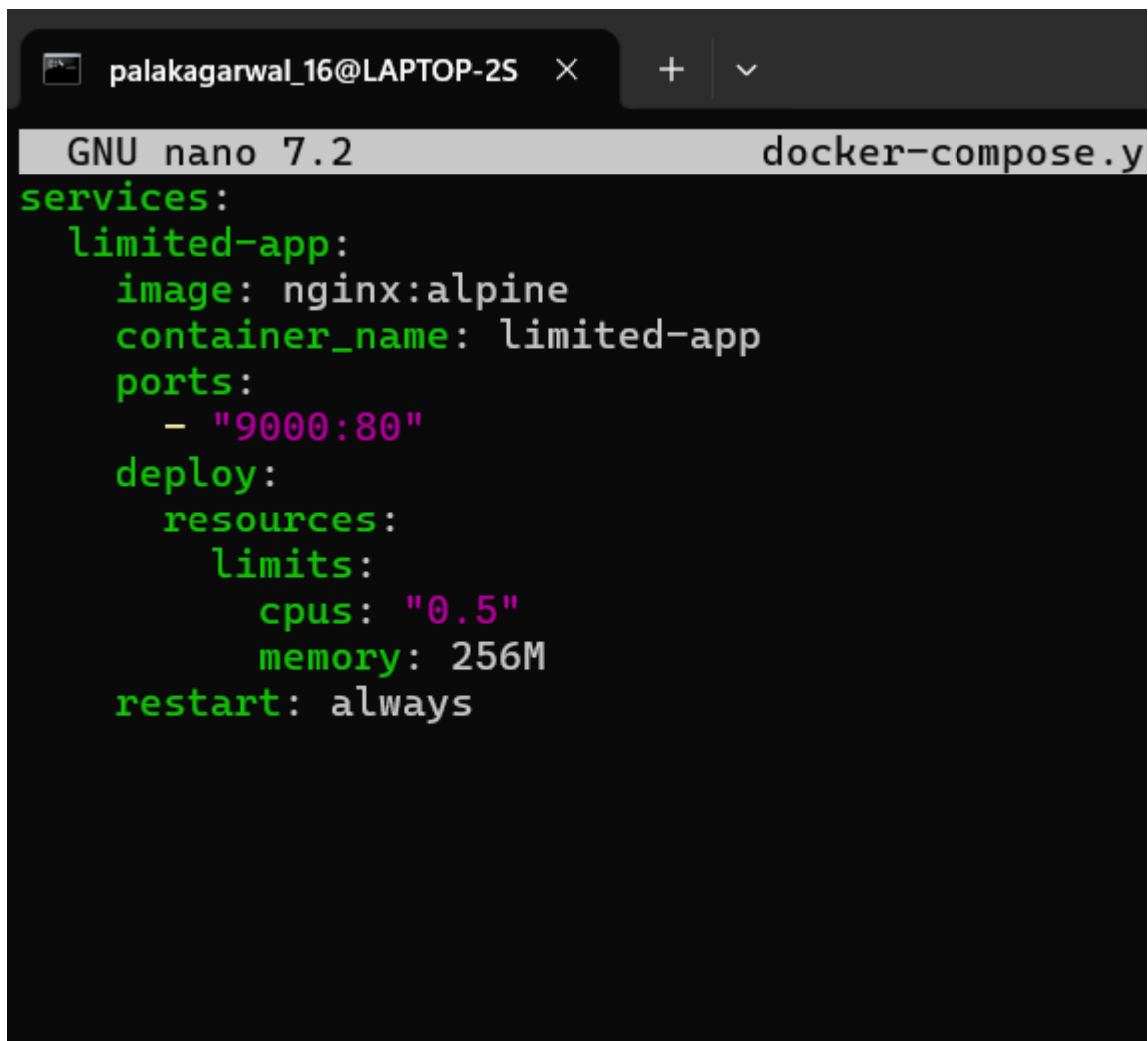
EXPOSE 3000

CMD ["node", "app.js"]
```

Step 3: Create `docker-compose.yml`

```
version: '3.8'

services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

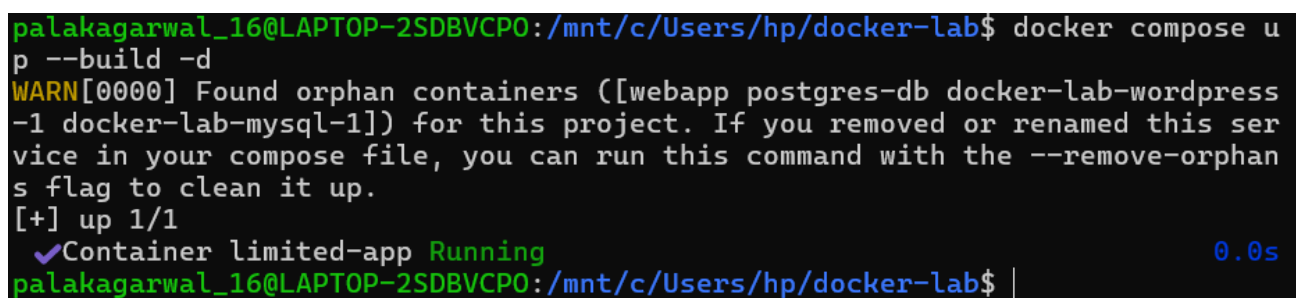


```
GNU nano 7.2 docker-compose.y
services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:80"
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 256M
    restart: always
```

Student Task:

1. Build and run using:

```
docker compose up --build -d
```



```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ docker compose u
p --build -d
WARN[0000] Found orphan containers ([webapp postgres-db docker-lab-wordpress
-1 docker-lab-mysql-1]) for this project. If you removed or renamed this ser
vice in your compose file, you can run this command with the --remove-orphan
s flag to clean it up.
[+] up 1/1
✔Container limited-app Running 0.0s
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/docker-lab$ |
```

2. Verify in browser:

```
http://localhost:3000
```

3. Modify app.js message.
4. Rebuild and observe changes.

5. Explain difference between:

- image:
- build:

image: image: node:18-alpine Meaning: Uses pre-built image from Docker Hub No Dockerfile needed Faster startup Example use: nginx mysql

build: context: . dockerfile: Dockerfile Meaning: Builds custom image from Dockerfile You control app setup Used for real applications

Advanced Build Challenge

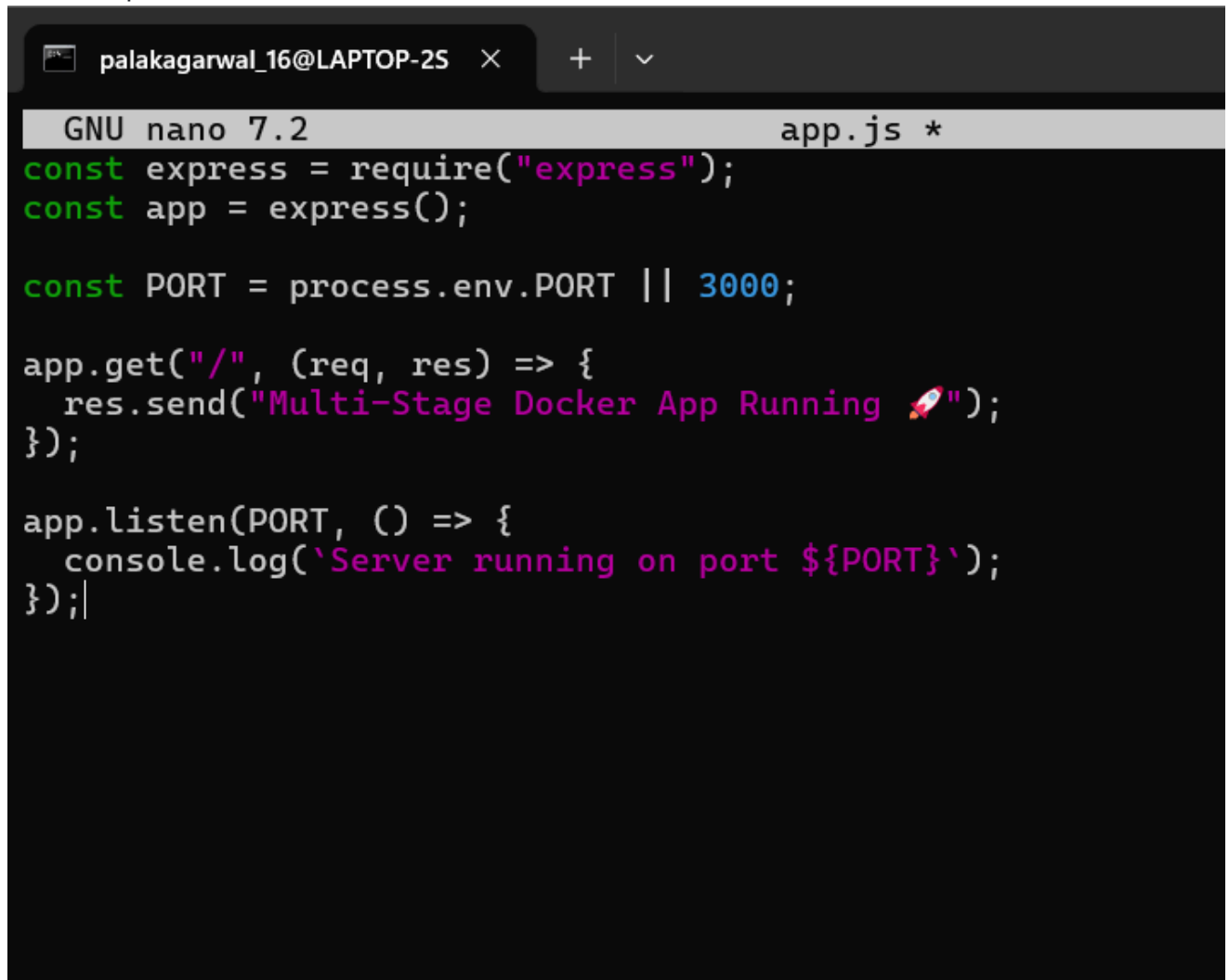
Task 6: Multi-Stage Dockerfile with Compose

Requirement:

Create a simple Python FastAPI or Node production-ready app using:

- Multi-stage Dockerfile
- Smaller final image

- Use Compose to build it

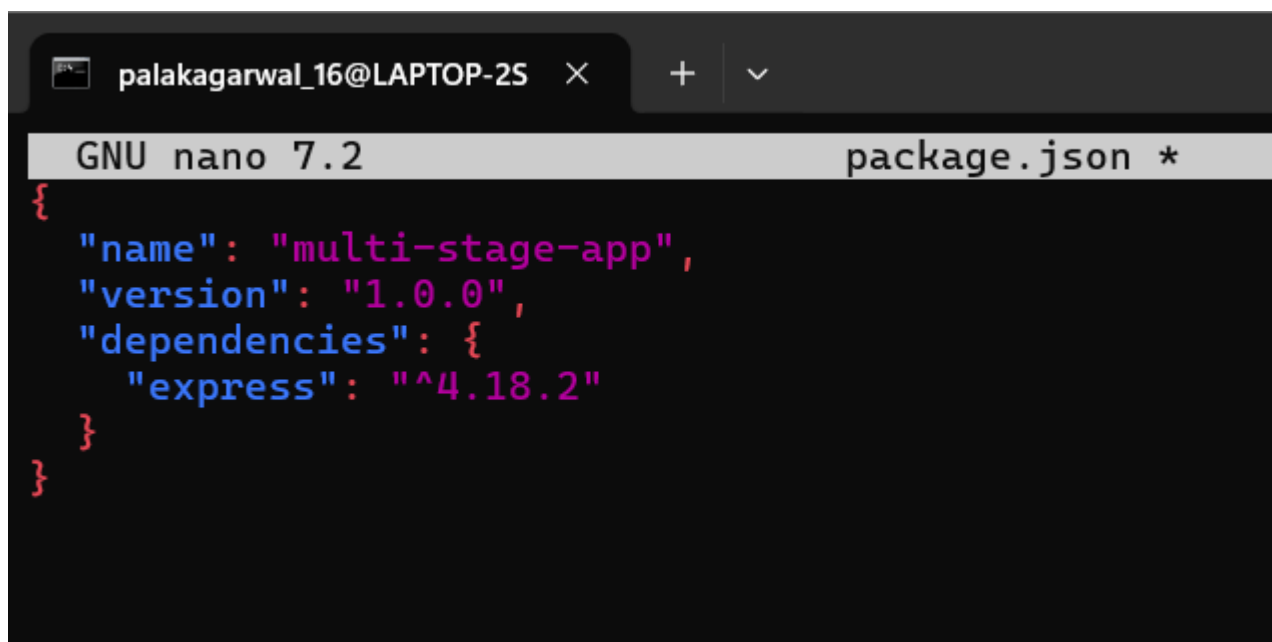


```
GNU nano 7.2 app.js *
const express = require("express");
const app = express();

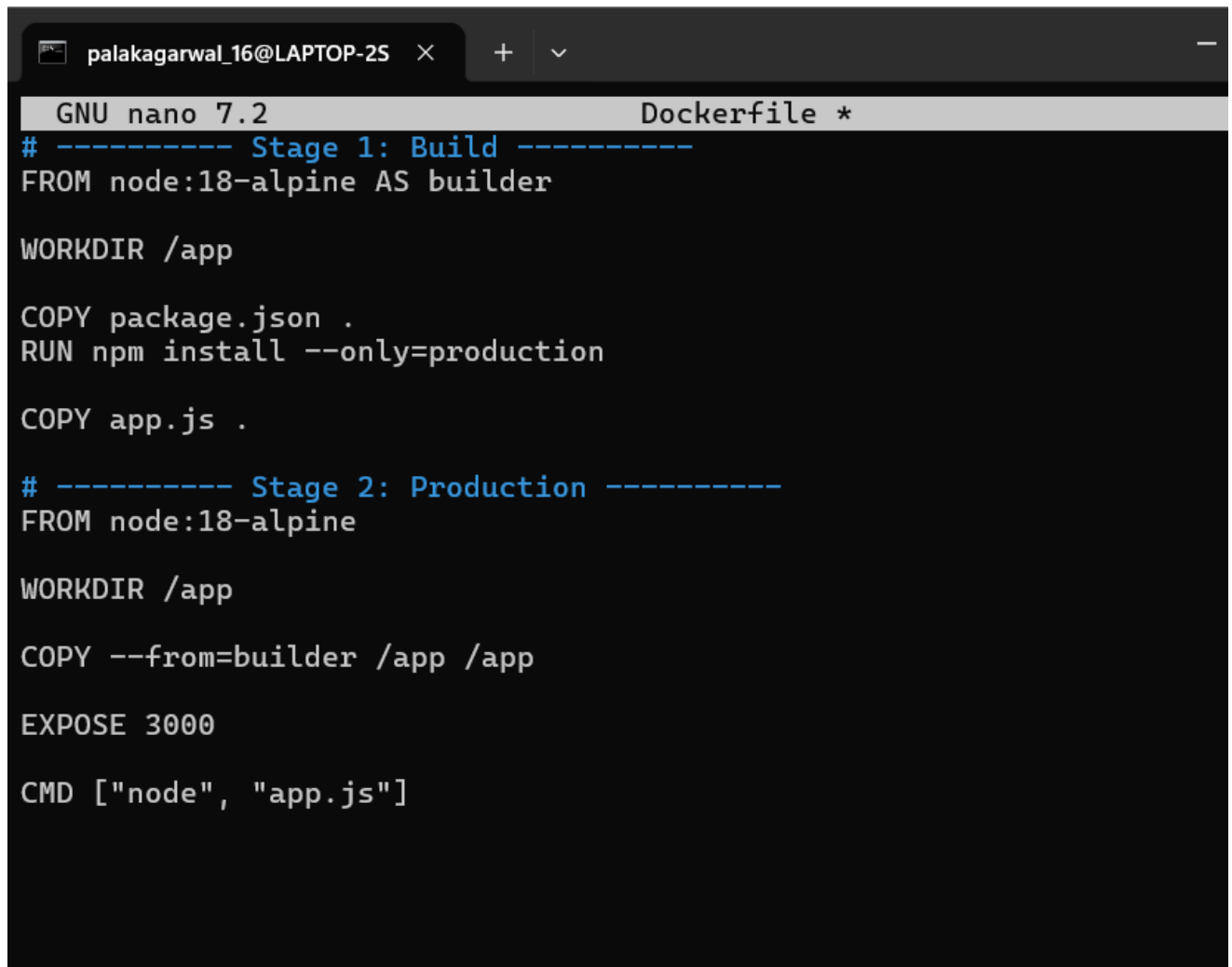
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.send("Multi-Stage Docker App Running 🚀");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});|
```



```
GNU nano 7.2 package.json *
{
  "name": "multi-stage-app",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```



```
palakagarwal_16@LAPTOP-2S x + v
GNU nano 7.2 Dockerfile *
# ----- Stage 1: Build -----
FROM node:18-alpine AS builder

WORKDIR /app

COPY package.json .
RUN npm install --only=production

COPY app.js .

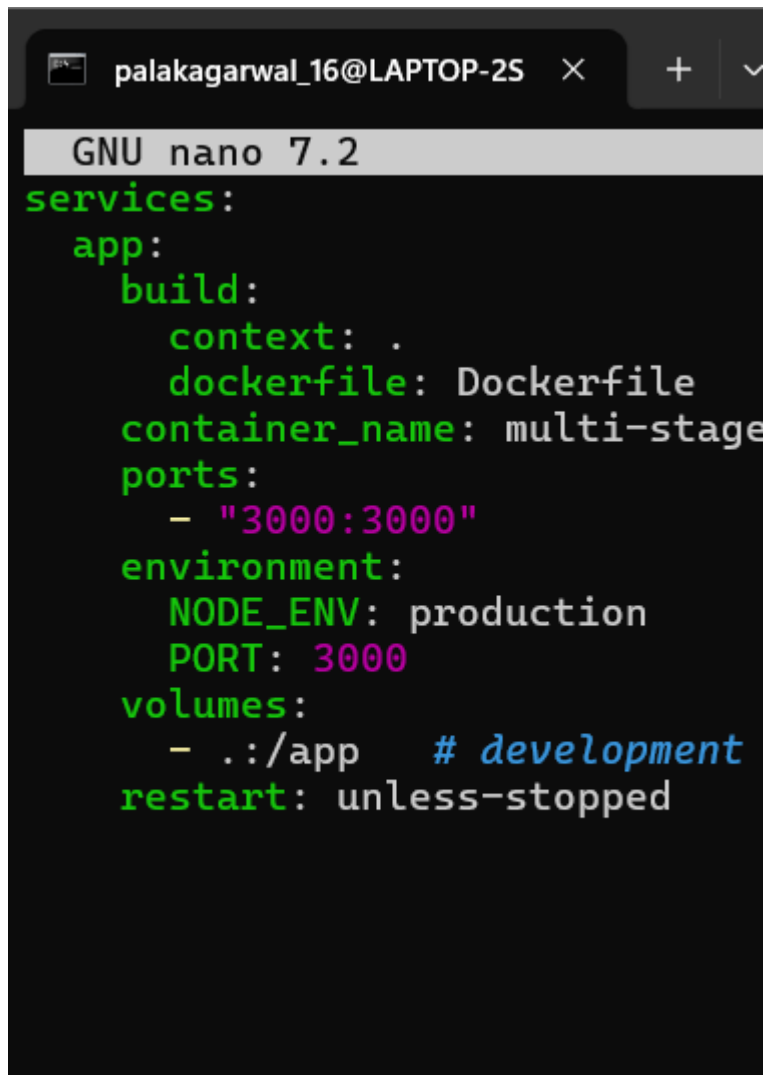
# ----- Stage 2: Production -----
FROM node:18-alpine

WORKDIR /app

COPY --from=builder /app /app

EXPOSE 3000

CMD ["node", "app.js"]
```



The image shows a terminal window with a dark background. At the top, the window title bar displays 'palakagarwal_16@LAPTOP-25' with standard window controls. Below the title bar, the terminal header indicates 'GNU nano 7.2'. The main content of the terminal is a Docker Compose file being edited. The file is structured as follows: 'services:' is followed by an indented 'app:' section. Under 'app:', there is a 'build:' section with 'context: .' and 'dockerfile: Dockerfile'. Below 'build:' is 'container_name: multi-stage'. Then comes 'ports:' with a list containing '- "3000:3000"'. This is followed by 'environment:' with 'NODE_ENV: production' and 'PORT: 3000'. Next is 'volumes:' with a list containing '- ./app # development'. Finally, 'restart: unless-stopped' is at the bottom. The text is color-coded: 'services:', 'app:', 'build:', 'context:', 'dockerfile:', 'container_name:', 'ports:', 'environment:', 'volumes:', and 'restart:' are in green. '3000:3000' and '3000' are in pink. 'development' is in blue.

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: multi-stage
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
      PORT: 3000
    volumes:
      - ./app # development
    restart: unless-stopped
```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker compose u
p --build -d
[+] Building 16.3s (13/13) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 498B 0.0s
=> [internal] load build definition from Dockerfile 0.2s
=> => transferring dockerfile: 340B 0.1s
=> [internal] load metadata for docker.io/library/node:18-alpine 0.2s
=> [internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [builder 1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421 0.1s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c2 0.1s
=> [internal] load build context 0.2s
=> => transferring context: 439B 0.1s
=> CACHED [builder 2/5] WORKDIR /app 0.0s
=> [builder 3/5] COPY package.json . 0.1s
=> [builder 4/5] RUN npm install --only=production 10.0s
=> [builder 5/5] COPY app.js . 0.2s
=> [stage-1 3/3] COPY --from=builder /app /app 0.6s
=> exporting to image 1.4s
=> => exporting layers 0.5s
=> => exporting manifest sha256:40ac6df8049ad155085eb78970e211d660ac 0.0s
=> => exporting config sha256:bca5a8cc61b2ef617107c0c98c95d0c5dd1eac 0.0s
=> => exporting attestation manifest sha256:ec44b63d761902b6778e3950 0.1s
=> => exporting manifest list sha256:afc75d8e983b6842ab9c29332b6dd24 0.0s
=> => naming to docker.io/library/docker-lab-app:latest 0.0s
=> => unpacking to docker.io/library/docker-lab-app:latest 0.6s
=> resolving provenance for metadata file 0.1s
[+] up 1/1
✔ Image docker-lab-app Built 16.8s
WARN[0018] Found orphan containers ([limited-app postgres-db docker-lab-word
press-1 docker-lab-mysql-1]) for this project. If you removed or renamed thi
[+] up 3/3in your compose file, you can run this command with the --remove-o
✔ Image docker-lab-app Built 16.8s
✔ Container webapp Recreated 2.3s
✔ Container multi-stage-app Started 0.7s
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker images

```

Must:

1. Write multi-stage Dockerfile
2. Use build: in Compose
3. Add environment variables
4. Add volume mount for development mode
5. Compare image size:

docker images

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	IN USE
alpine:latest	25109184c71b	13.1MB	3.95MB	
assignment-1-node_backend:latest	764bb93e88ed	1.58GB	397MB	
assignment-1-postgres_db:latest	13bc0843540e	622MB	157MB	
c-app:latest	e1372e0245d2	2.17GB	552MB	
docker-lab-app:latest	afc75d8e983b	186MB	45.6MB	U
docker-lab-nodeapp:latest	617714270281	181MB	44.9MB	
flask-app:latest	3efb7b4a43b0	1.6GB	410MB	
ghcr.io/k3d-io/k3d-proxy:5.8.3	d43e6ffd27c2	93.1MB	24.7MB	
ghcr.io/k3d-io/k3d-tools:5.8.3	c5ceb2b9e8ff	31.9MB	10.3MB	
hello-world:latest	85404b3c5395	25.9kB	9.52kB	
jenkins/jenkins:lts	c4098086090c	806MB	288MB	U
my-node-app:latest	c4dc10dbc46d	2.17GB	552MB	
mysql:5.7	4bc6bc963e6d	700MB	149MB	U
mysql:8.0	64756cc92f70	1.08GB	247MB	
nginx:alpine	582c496ccf79	93.5MB	26.9MB	U
nginx:latest	341bf0f3ce6c	240MB	65.8MB	
nginx:latest@sha256:dec7a90bd0973b076832dc56933fe876bc014929e14b4ec49923951405370112	dec7a90bd097	240MB	65.8MB	
node:18-alpine	8d6421d663b4	181MB	45.3MB	U
node_backend:latest	554343da74d2	1.58GB	397MB	
palakagarwal16/myapp:latest	3854f7237163	203MB	49.7MB	
postgres:15	31224c60946e	633MB	164MB	U
python:3.11-slim	233de06753d3	188MB	47.7MB	U
rancher/k3s:v1.31.5-k3s1	53cf744fe2fa	298MB	74.8MB	
redis:7-alpine	8b81dd37ff02	61.2MB	17.7MB	
ubuntu:latest	186072bba1b2	119MB	31.7MB	
wordpress:latest	1a758de2f424	1.08GB	275MB	U

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$
```

Experiment 6 B

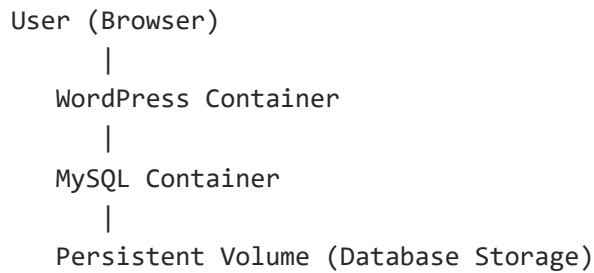
Multi-Container Application using Docker Compose (WordPress + Database)**

1. Objective

To deploy a multi-container application using **Docker Compose**, consisting of:

- WordPress (frontend + PHP)
- MySQL database (backend)

3. Architecture Overview



Steps

Step 1: Create Project Directory

```
mkdir wp-compose-lab
cd wp-compose-lab
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ mkdir wp-compose-lab
mkdir: cannot create directory 'wp-compose-lab': File exists
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ cd wp-compose-lab
```

Step 2: Create docker-compose.yml

```
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
```

```
environment:
  WORDPRESS_DB_HOST: db:3306
  WORDPRESS_DB_USER: wpuser
  WORDPRESS_DB_PASSWORD: wppass
  WORDPRESS_DB_NAME: wordpress
volumes:
  - wp_data:/var/www/html
```

```
volumes:
  db_data:
  wp_data:
```

```
palakagarwal_16@LAPTOP-2S  ×  +  ▾
GNU nano 7.2                docker-compose.yml
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

depends_on

- Ensures DB starts before WordPress

Step 3: Start Application

```
docker-compose up -d
```

What happens:

- Images are pulled
- Network is created
- Containers are started
- DNS-based service discovery enabled

Step 4: Verify Containers

```
docker ps
```

```

palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/wp-compose-lab$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS
NAMES
df11e98e471f   mysql:5.7                          "docker-entrypoint.s..." 53 seconds ago
Up 51 seconds  3306/tcp, 33060/tcp
wordpress_db
2d80294c3c52   docker-lab-app                     "docker-entrypoint.s..." 27 minutes ago
multi-stage-app
c74435a729c7   nginx:alpine                       "/docker-entrypoint...." 42 minutes ago
Up 42 minutes  0.0.0.0:9000->80/tcp, [::]:9000->80/tc
limited-app
p
f79d7bd2e88d   nginx:alpine                       "/docker-entrypoint...." 44 minutes ago
Up 44 minutes  80/tcp
mystack_limited-app
.1.lid6nan339zjlgek2ilu453mt
84697fc912c5   postgres:15                        "docker-entrypoint.s..." 53 minutes ago
Up 53 minutes  5432/tcp
postgres-db
62e68111e9d8   mysql:5.7                          "docker-entrypoint.s..." About an hour
ago Up About an hour 3306/tcp, 33060/tcp
docker-lab-mysql-1
7671ae19f74d   mysql:5.7                          "docker-entrypoint.s..." About an hour
ago Up About an hour 3306/tcp, 33060/tcp
mysql
e96fbf887134   jenkins/jenkins:lts               "/usr/bin/tini -- /u..." 20 hours ago
Up 20 hours 0.0.0.0:8080->8080/tcp, [::]:8080->808
0/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
jenkins
be5526f83dfc   wordpress:latest                  "docker-entrypoint.s..." 21 hours ago
Up 21 hours 80/tcp
wpstack_wordpress.3
.tenaieoi9vkrcbw8wc5fhomgy
213c9de46f67   wordpress:latest                  "docker-entrypoint.s..." 21 hours ago
Up 21 hours 80/tcp
wpstack_wordpress.1
.u515dya7dah76xtisyih6ymv6
257a2c74a705   wordpress:latest                  "docker-entrypoint.s..." 21 hours ago
Up 21 hours 80/tcp
wpstack_wordpress.2

```

Expected:

- wordpress_app
- wordpress_db

Step 5: Access WordPress

Open browser:

<http://localhost:8080>

Step 6: Check Volumes

```
docker volume ls
```

```
local    e01d744779592e25d905db776d448d3e9be7560a47b1c146e5de34694f4bb00b
local    ef46124ef5b529213e1776c43d859e2e3b77b2d7fc91e805962ffd11a518543c
local    hp_db_data
local    hp_mysql_data
local    hp_wp_content
local    jenkins_home
local    k3d-mycluster-images
local    mydata
local    mysql-data
local    myvolume
local    pgdata
local    postgres-data
local    redis-data
local    test
local    wp-compose-lab_db_data
local    wp-compose-lab_wp_data
local    wpstack_db_data
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wp-compose-lab$ |
```

- db_data → database persistence
- wp_data → WordPress files

Step 7: Stop Application

```
docker-compose down
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wp-compose-lab$ docker-compose down
WARN[0000] /mnt/c/Users/hp/wp-compose-lab/docker-compose.yml: the attribute
`version` is obsolete, it will be ignored, please remove it to avoid potenti
al confusion
[+] down 3/3
✓Container wordpress_app      Removed      0.1s
✓Container wordpress_db      Removed      1.7s
✓Network wp-compose-lab_default Removed      0.5s
```

- Containers removed
- Volumes remain intact

5. Scaling in Docker Compose

Method 1: Scale WordPress Containers

```
docker-compose up --scale wordpress=3
```

```
✓ Network wp-compose-lab_default Removed 0.3s
alalakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wp-compose-lab$ docker-compose
se up --scale wordpress=3
WARN[0000] /mnt/c/Users/hp/wp-compose-lab/docker-compose.yml: the attribute
version` is obsolete, it will be ignored, please remove it to avoid potenti
al confusion
+ ] up 2/2
✓ Network wp-compose-lab_default Created 0.2s
✓ Container wordpress_db Created 0.3s
WARNING: The "wordpress" service is using the custom container name "wordpre
s_app". Docker requires each container to have a unique name. Remove the cu
stom name to scale the service
alalakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wp-compose-lab$
```

Result:

- 3 WordPress containers running

Problem:

- All try to use same port (8080)
- No load balancing

Solution: Use Reverse Proxy (Nginx)

Add another service:

```
nginx:
  image: nginx:latest
  ports:
    - "8080:80"
```

Then configure load balancing manually.

Limitations of Compose Scaling

- No built-in load balancing
- No auto-healing
- Single host only
- Not production-ready for scaling

6. Running Same Setup with Docker Swarm

Step 1: Initialize Swarm

```
docker swarm init
```

```
compose.yml
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/wp-compose-lab$ docker swarm
init
Error response from daemon: This node is already part of a swarm. Use "docke
r swarm leave" to leave this swarm and join another one.
```

Step 2: Deploy Stack

```
docker stack deploy -c docker-compose.yml wpstack
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/wp-compose-lab$ docker stack
deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the backgro
und.
In a future release, --detach=false will become the default.
Updating service wpstack_db (id: kwcerh5zsmc7o1t85n3bsctoi)
Updating service wpstack_wordpress (id: uunbgglv53ty7a5ryd0xk9arc)
```

Step 3: Scale Service

```
docker service scale wpstack_wordpress=3
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/wp-compose-lab$ docker servi
ce scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/wp-compose-lab$ |
```

4. Benefits of Docker Swarm

- Built-in load balancing
- Automatic container restart (self-healing)
- Horizontal scaling across nodes
- Rolling updates without downtime
- Service abstraction (not individual containers)

5. Key Learning Outcomes

- Multi-container apps require orchestration
- Docker Compose is ideal for:
 - Development
 - Testing
 - Learning
- Docker Swarm is useful for:
 - Simple production clusters
 - Easy scaling without Kubernetes complexity

6. Conclusion

This experiment demonstrated:

- How to deploy **WordPress + MySQL** using Docker Compose
- How containers communicate using internal networking
- Importance of **volumes for persistence**
- Scaling limitations of Compose
- Advantages of using **Docker Swarm for production-ready deployments**

Cloud-containerization-and-DevOps

Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

1. Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

2. Introduction

CI/CD stands for Continuous Integration and Continuous Deployment. It automates the process of building, testing, and deploying applications. Tools like Jenkins, GitHub, and Docker Hub help in creating a complete pipeline for efficient software delivery.

3. CI/CD Pipeline Architecture

Developer → GitHub → Jenkins → Docker Build → Docker Hub

4. Part A: GitHub Repository Setup (Source Code + Build Definition)

4.1 Create Repository

Create a repository on GitHub: my-app

The screenshot shows the GitHub interface for a repository named 'my-app' owned by 'Palak-agarwal16'. The repository is public and has 0 stars, 0 forks, 0 watchers, 1 branch, and 0 tags. The main branch is 'main'. The repository contains four files: Dockerfile, Jenkinsfile, app.py, and requirements.txt. The commit history shows a recent commit 'fix jenkinsfile final' by Palak-agarwal16, 40 minutes ago. The README section is currently empty, with a prompt to 'Add a README'.

File	Commit	Time
Dockerfile	Create Dockerfile	1 hour ago
Jenkinsfile	fix jenkinsfile final	40 minutes ago
app.py	app.py	1 hour ago
requirements.txt	requirements.txt	1 hour ago

4.2 Project Structure

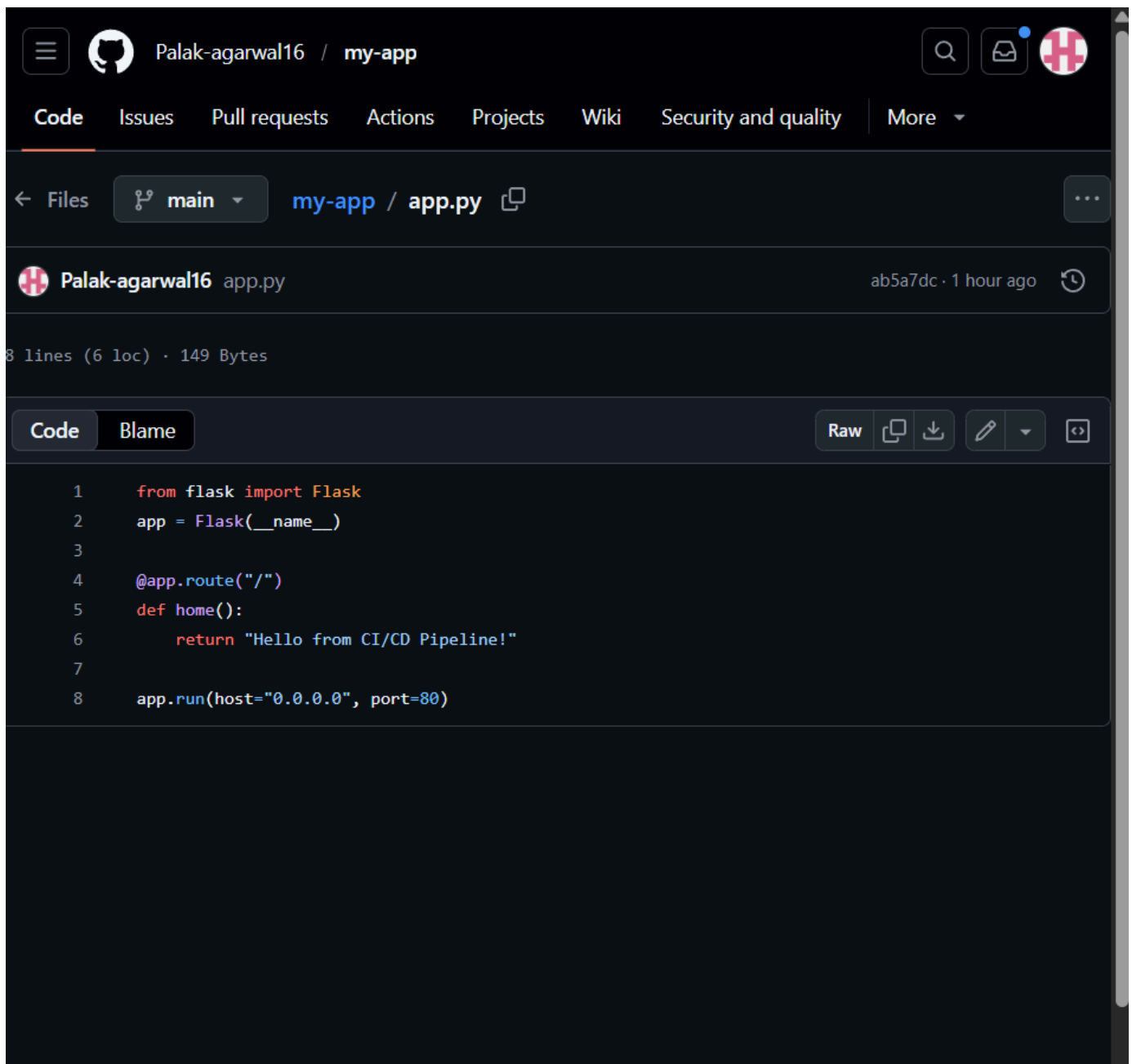
my-app/ |— app.py |— requirements.txt |— Dockerfile |— Jenkinsfile

4.3 Application Code

app.py from flask import Flask app = Flask(name)

@app.route("/") def home(): return "Hello from CI/CD Pipeline!" #return "Hello from CI/CD Pipeline!, my sapid is 123456"

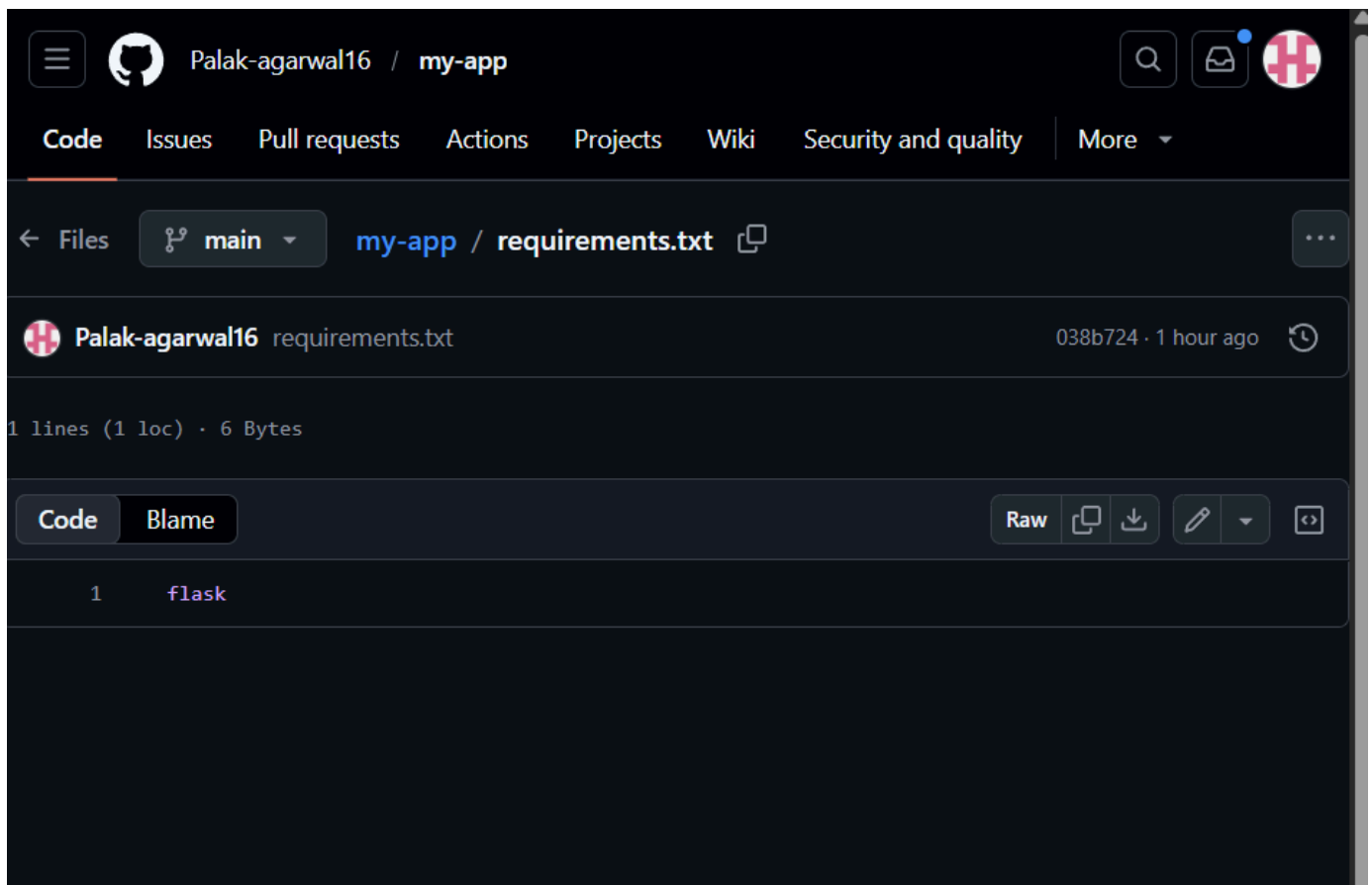
app.run(host="0.0.0.0", port=80)



The screenshot shows a GitHub repository page for a user named 'Palak-agarwal16' and a repository named 'my-app'. The 'Code' tab is selected, and the file 'app.py' is displayed. The file is 8 lines long, 6 lines of code, and 149 bytes. The code is a simple Flask application that returns 'Hello from CI/CD Pipeline!' when accessed at the root URL. The commit hash 'ab5a7dc' is shown, along with the time '1 hour ago'.

```
1  from flask import Flask
2  app = Flask(__name__)
3
4  @app.route("/")
5  def home():
6      return "Hello from CI/CD Pipeline!"
7
8  app.run(host="0.0.0.0", port=80)
```

requirements.txt flask



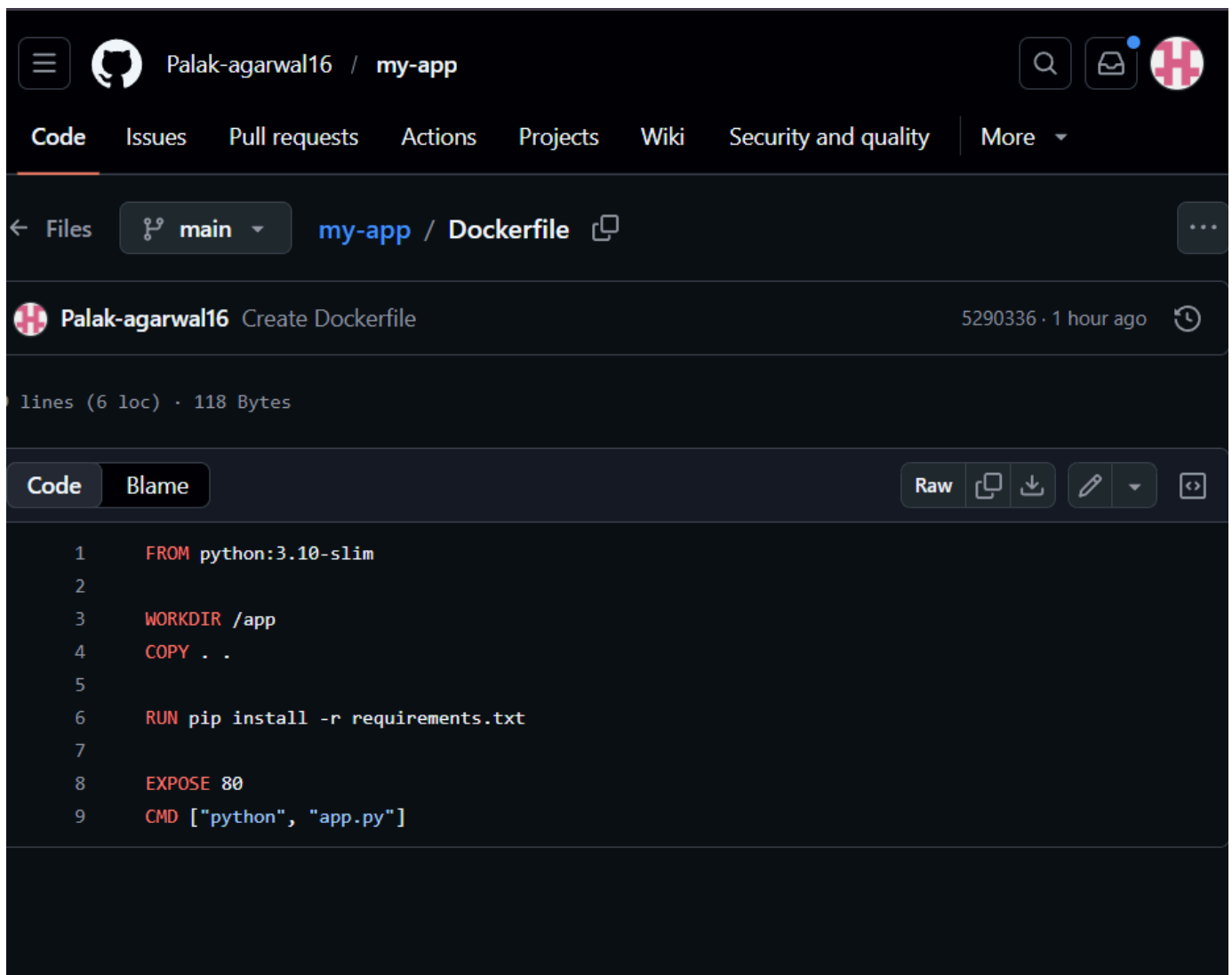
4.4 Dockerfile (Build Process)

FROM python:3.10-slim

WORKDIR /app COPY . .

RUN pip install -r requirements.txt

EXPOSE 80 CMD ["python", "app.py"]



4.5 Jenkinsfile (Pipeline Definition in GitHub)

pipeline { agent any

```
environment {
    IMAGE_NAME = "your-dockerhub-username/myapp"
}

stages {

    stage('Clone Source') {
        steps {
            git 'https://github.com/your-username/my-app.git'
        }
    }

    stage('Build Docker Image') {
        steps {
            sh 'docker build -t $IMAGE_NAME:latest .'
        }
    }

    stage('Login to Docker Hub') {
```

```

    steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
            sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --password $DOCKER_TOKEN'
        }
    }
}

stage('Push to Docker Hub') {
    steps {
        sh 'docker push $IMAGE_NAME:latest'
    }
}
}
}

```

```

1  pipeline {
2      agent any
3
4      environment {
5          IMAGE_NAME = "palakagarwal16/myapp"
6      }
7
8      stages {
9
10         stage('Clone Source') {
11             steps {
12                 git branch: 'main', url: 'https://github.com/Palak-agarwal16/my-app.git'
13             }
14         }
15
16         stage('Build Docker Image') {
17             steps {
18                 sh 'docker build -t $IMAGE_NAME:latest .'
19             }
20         }
21
22         stage('Login to Docker Hub') {
23             steps {
24                 withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {

```

5. Part B: Jenkins Setup using Docker (Persistent Configuration)

5.1 Create Docker Compose File

version: '3.8'

services: jenkins: image: jenkins/jenkins:lts container_name: jenkins restart: always ports: - "8080:8080" - "50000:50000" volumes: - jenkins_home:/var/jenkins_home - /var/run/docker.sock:/var/run/docker.sock user: root

volumes: jenkins_home:

```
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

5.2 Start Jenkins

docker-compose up -d

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/docker-lab$ docker-compose up
-d
WARN[0000] /mnt/c/Users/hp/docker-lab/docker-compose.yml: the attribute 'ver
sion' is obsolete, it will be ignored, please remove it to avoid potential c
onfusion
[+] up 16/16
✔Image jenkins/jenkins:lts      Pulled      85.1s
✔Volume docker-lab_jenkins_home Created    0.0s
✔Container jenkins              Started   1.3s
```

Access: <http://localhost:8080>

5.3 Unlock Jenkins

`docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword`

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/my-app$ docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword  
642decc4bb0a4feb99ce6da8f41d700f
```

5.4 Initial Setup

- Install suggested plugins
- Create admin user

6. Part C: Jenkins Configuration

6.1 Add Docker Hub Credentials

- Type: Secret Text
- ID: dockerhub-token
- Value: Docker Hub Access Token

< Add Secret text ×

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ✓

Secret
.....

ID ?
dockerhub-token

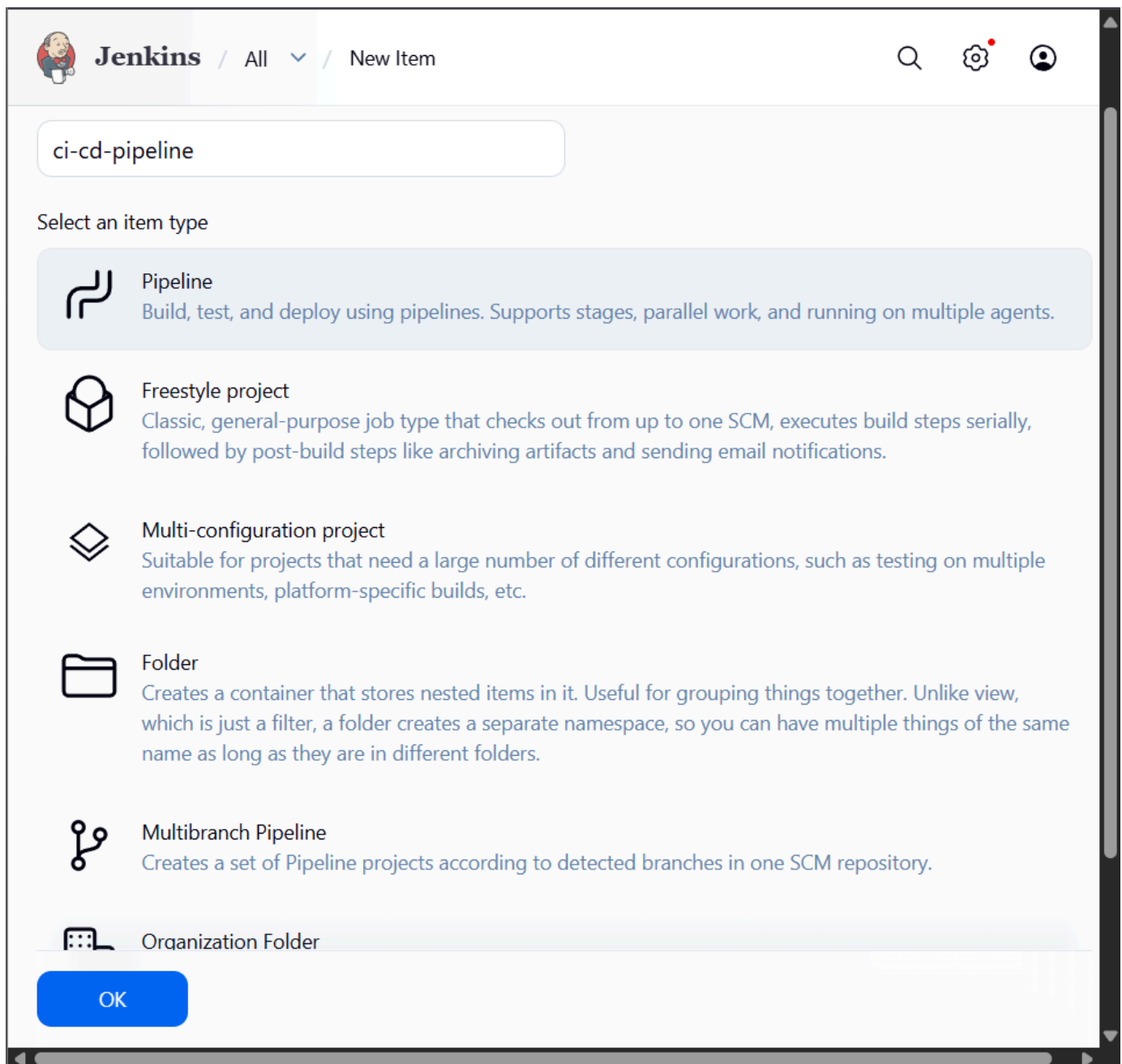
Description ?

Create

6.2 Create Pipeline Job

Pipeline script from SCM

- SCM: Git
- Repo URL: your GitHub repo
- Script Path: Jenkinsfile



7. Part D: Trigger Mechanism (Manual Execution)

In this experiment, the pipeline was triggered manually from Jenkins dashboard.

Steps:

1. Open Jenkins Dashboard
2. Select pipeline job
3. Click Build Now
4. Pipeline executes

The screenshot shows the Jenkins web interface for a pipeline named 'ci-cd-pipeline'. The top navigation bar includes the Jenkins logo, the pipeline name, and search, settings, and user icons. A left sidebar contains links for Status, Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, and Pipeline Syntax. The main content area shows the 'Builds' section with a filter input and a list of recent builds. Two builds are listed under 'Today': build #2 at 14:47 and build #1 at 14:06, both marked as successful with green checkmarks. Below the build list, the pipeline name 'ci-cd-pipeline' is displayed with a green checkmark icon, followed by an 'Add description' button. The bottom of the screen shows a browser address bar with the URL 'localhost:8080/job/ci-cd-pipeline/' and a system taskbar with various icons and the time '20:17'.

Jenkins / ci-cd-pipeline

Status

Changes

Build Now

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

Builds >

Filter

Today

#2 14:47

#1 14:06

ci-cd-pipeline

Add description

localhost:8080/job/ci-cd-pipeline/

8. Part E: Execution Flow (Stage-wise)

Stage 1: Code Push Developer updates code in GitHub

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/my-app$ git add .
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/my-app$ git commit -m "fix jenkinsfile final"
[main 75ee769] fix jenkinsfile final
 1 file changed, 1 insertion(+), 1 deletion(-)
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/my-app$ git push
Username for 'https://github.com': Palak-agarwal16
Password for 'https://Palak-agarwal16@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 322 bytes | 16.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Palak-agarwal16/my-app.git
   17c6ddf..75ee769  main -> main
```

Stage 2: Triggering GitHub sends event to Jenkins



ci-cd-pipeline

Permalinks

- [Last build \(#1\), 41 min ago](#)
- [Last stable build \(#1\), 41 min ago](#)
- [Last successful build \(#1\), 41 min ago](#)
- [Last completed build \(#1\), 41 min ago](#)

Stage 3: Jenkins Pipeline Execution

Clone Stage: Pulls latest code from GitHub

```

Started by user Palak Agarwal
Obtained Jenkinsfile from git https://github.com/Palak-agarwal16/my-app.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/ci-cd-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/ci-cd-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Palak-agarwal16/my-app.git # timeout=10
Fetching upstream changes from https://github.com/Palak-agarwal16/my-app.git

```

Build Stage: Docker builds image

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/my-app$ docker build -t palakagarwal16/myapp:latest .
[+] Building 24.6s (8/9)                                docker:default
=> [internal] load build definition from Dockerfile      0.3s
=> => transferring dockerfile: 155B                     0.1s
=> [internal] load metadata for docker.io/library/python:3.10-slim 2.8s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                        0.1s
=> => transferring context: 2B                          0.0s
=> [1/4] FROM docker.io/library/python:3.10-slim@sha256:26fc00cf5fd6 0.1s
=> => resolve docker.io/library/python:3.10-slim@sha256:26fc00cf5fd6 0.1s
=> [internal] load build context                        0.7s
=> => transferring context: 40.15kB                     0.7s
=> CACHED [2/4] WORKDIR /app                          0.0s
=> [3/4] COPY . .                                       0.2s
=> [4/4] RUN pip install -r requirements.txt           20.3s
=> => # Collecting click>=8.1.3
=> => #   Downloading click-8.3.2-py3-none-any.whl (108 kB)
=> => #   _____ 108.4/108.4 kB 132.0 kB
=> => # /s eta 0:00:00
=> => # Collecting itsdangerous>=2.2.0
=> => #   Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)

```

Auth Stage: Jenkins logs into Docker Hub

```

sha256:80e951644ad6fdf4ec08d77b06d3d3314f3c1692d1bb12a5465606218651d23b 0.1s done
#10 exporting manifest list
sha256:a5f5a44524466caedab1b18c7d9cc6b5450ff3f44c4b1d8f287e8179011705c7 0.0s done
#10 naming to docker.io/palakagarwal16/myapp:latest 0.0s done
#10 unpacking to docker.io/palakagarwal16/myapp:latest
#10 unpacking to docker.io/palakagarwal16/myapp:latest 2.2s done
#10 DONE 4.7s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Login to Docker Hub)
[Pipeline] withCredentials
Masking supported pattern matches of $DOCKER_TOKEN
[Pipeline] {
[Pipeline] sh
+ echo ****
+ docker login -u palakagarwal16 --password-stdin
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded

```

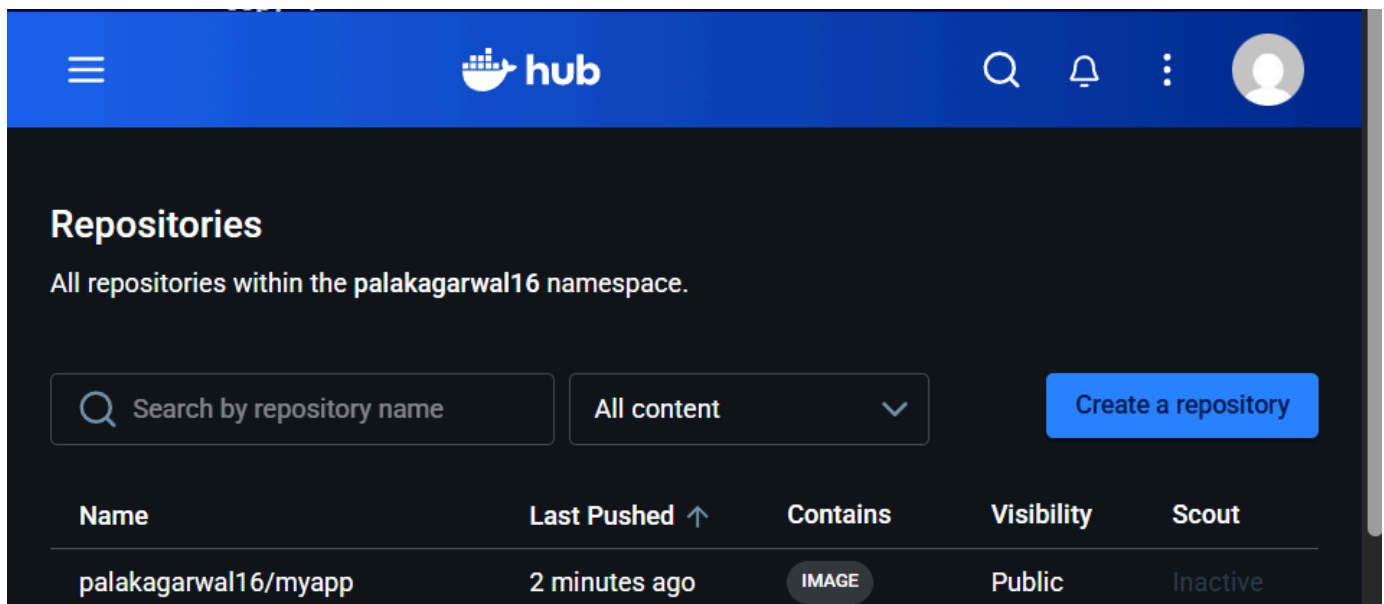
Push Stage: Image pushed to Docker Hub

```

[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Push to Docker Hub)
[Pipeline] sh
+ docker push palakagarwal16/myapp:latest
The push refers to repository [docker.io/palakagarwal16/myapp]
ddf3b231f567: Waiting
bdd96403af7b: Waiting
6f466d9741b7: Waiting
bed04db721b4: Waiting
5435b2dcdf5c: Waiting
8a40b8b01719: Waiting

```

Stage 4: Artifact Ready Docker image available globally



The screenshot shows the Docker Hub interface for the namespace **palakagarwal16**. At the top, there's a blue header with the Docker Hub logo and navigation icons. Below the header, the title "Repositories" is displayed, followed by the text "All repositories within the palakagarwal16 namespace." A search bar with the placeholder "Search by repository name" and a dropdown menu set to "All content" are visible. A blue button labeled "Create a repository" is on the right. Below these elements is a table listing repositories. The table has columns: Name, Last Pushed, Contains, Visibility, and Scout. One repository is listed: **palakagarwal16/myapp**, pushed 2 minutes ago, containing an **IMAGE**, with **Public** visibility and an **Inactive** Scout status.

Name	Last Pushed ↑	Contains	Visibility	Scout
palakagarwal16/myapp	2 minutes ago	IMAGE	Public	Inactive

/var/run/docker.sock

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/my-app$ cat docker-compose.yml
ml
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

9. Basic Pipeline Structure

```
pipeline { agent any
```

```
    stages {
        stage('Build') {
            steps {
                sh 'echo Hello'
            }
        }
    }
}
```

 **Jenkins** / ci-cd-pipeline / #2 / Stages

Search

✓

Checkout SCM

1s

✓

Clone Source

1s

✓

Build Docker Image

33s

✓

Login to Docker Hub

3s

✓

Push to Docker Hub

9s

✓

docker push \$IMAGE_NAME:latest

9s

13

bed04db721b4: Waiting

14

5435b2dcdf5c: Waiting

15

8a40b8b01719: Waiting

16

935d60c8480a: Waiting

17

9833b846f421: Waiting

18

935d60c8480a: Waiting

19

9833b846f421: Waiting

20

ddf3b231f567: Waiting

21

bdd96403af7b: Waiting

22

6f466d9741b7: Waiting

23

bed04db721b4: Waiting

24

5435b2dcdf5c: Waiting

25

8a40b8b01719: Waiting

26

5435b2dcdf5c: Waiting

27

8a40b8b01719: Waiting

28

935d60c8480a: Waiting

29

9833b846f421: Waiting

30

ddf3b231f567: Waiting

<https://palak-agarwal16.github.io/Cloud-containerization-and-DevOps/LAB/Experiment-7/>

16/18

```
Code Blame Raw [Icons]
4     environment {
5         IMAGE_NAME = "palakagarwal16/myapp"
6     }
7
8     stages {
9
10        stage('Clone Source') {
11            steps {
12                git branch: 'main', url: 'https://github.com/Palak-agarwal16/my-app.git'
13            }
14        }
15
16        stage('Build Docker Image') {
17            steps {
18                sh 'docker build -t $IMAGE_NAME:latest .'
19            }
20        }
21
22        stage('Login to Docker Hub') {
23            steps {
24                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')])
25                sh 'echo $DOCKER_TOKEN | docker login -u palakagarwal16 --password-stdin'
26            }
27        }
28    }
29
30    stage('Push to Docker Hub') {
31        steps {
32            sh 'docker push $IMAGE_NAME:latest'
33        }
34    }
```

 **Jenkins** / Manage Jenkins ▾ / Credentials 🔍 ⚙️ 👤

Credentials

[+ Add Credentials](#)

[?](#)

 **dockerhub-token**
 System - Global ✎️ ⋮

Stores scoped to Jenkins

 **System** Domains: [Global](#)

10. Key Takeaways

- pipeline → stages → stage → steps defines structure
- sh executes shell commands
- git fetches code
- withCredentials secures secrets
- secrets are protected and temporary

11. Result

CI/CD pipeline executed successfully. Docker image built and pushed to Docker Hub.

12. Conclusion

Successfully implemented CI/CD pipeline using Jenkins, GitHub, and Docker Hub. Automation improved efficiency and reduced manual effort.

Cloud-containerization-and-DevOps

Experiment 9: Working with Ansible

Introduction

What is Ansible?

- Ansible is an open-source **automation tool** for **configuration management**, **application deployment**, and **orchestration**.
- It follows an **agentless** architecture, using SSH for Linux and WinRM for Windows.
- Uses YAML-based **playbooks** to define automation tasks.

Ansible is software that enables cross-platform automation and orchestration at scale and has become the standard choice among enterprise automation solutions.

Key Concepts

Component	Description
Control Node	Machine with Ansible installed
Managed Nodes	Target servers (no Ansible agent needed)
Inventory	Defines the list of managed nodes (EC2 instances, servers, etc.). <code>inventory.ini</code>
Playbooks	YAML files containing a sequence of automation steps.
Tasks	Individual actions in playbooks (e.g., installing a package).
Modules	Built-in functionality to perform tasks (e.g., <code>yum</code> , <code>apt</code> , <code>service</code>).
Roles	Pre-defined reusable automation scripts.

Benefits of using Ansible

- A free and open-source community project with a huge audience.
- Battle-tested over many years as the preferred tool of IT wizards.
- Easy to start and use from day one, without the need for any special coding skills.
- Simple deployment workflow without any extra agents.
- Includes some sophisticated features around modularity and reusability that come in handy as users become more proficient.
- Extensive and comprehensive official documentation that is complemented by a plethora of online material produced by its community.

To sum up, Ansible is simple yet powerful, agentless, community-powered, predictable, and secure.

Hands-On

Step-1:- Install Ansible in WSL

```
sudo apt install -y
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ansible-core ieee-data python3-argcomplete python3-dnspython
  python3-jmespath python3-kerberos python3-libcloud python3-lockfile
  python3-netaddr python3-ntlm-auth python3-packaging python3-passlib
  python3-requests-ntlm python3-resolvelib python3-selinux
  python3-simplejson python3-winrm python3-xlrd python3-xlsxwriter
Suggested packages:
  cowsay sshpass python3-trio python3-aioquic python3-h2 python3-httpx
  python3-httpcore python-lockfile-doc ipython3 python-netaddr-docs
The following NEW packages will be installed:
  ansible ansible-core ieee-data python3-argcomplete python3-dnspython
  python3-jmespath python3-kerberos python3-libcloud python3-lockfile
  python3-netaddr python3-ntlm-auth python3-packaging python3-passlib
  python3-requests-ntlm python3-resolvelib python3-selinux
  python3-simplejson python3-winrm python3-xlrd python3-xlsxwriter
0 upgraded, 19 newly installed, 0 to remove and 91 not upgraded.
Need to get 22.0 MB of archives.
After this operation, 330 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-packaging al
l 24.0-1 [41.1 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-resolve
lib all 1.0.1-1 [25.7 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-dnspython al
l 2.6.1-1ubuntu1 [163 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 ieee-data all 202208
27.1 [2113 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-netaddr all
0.8.0-2ubuntu1 [319 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 ansible-core all
```

Step-2:- Verify Installation

```
ansible --version
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/palakagarwal_16/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/palakagarwal_16/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Jan 22 2026, 20:57:42) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

Step-3:- Ping Localhost

```
ansible localhost -m ping
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ ansible localhost -m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
```

Step-4:- Generate ssh Keys

```
ssh-keygen -t rsa -b 4096
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/palakagarwal_16/.ssh/id_rsa): Sem6
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in Sem6
Your public key has been saved in Sem6.pub
The key's fingerprint is:
SHA256:oFx17tPdckMM+BiN7SyFn8TM2FbA5Evj7Sz8uqcVZ2c palakagarwal_16@LAPTOP-2SDBVCP0
The key's randomart image is:
+---[RSA 4096]---+
|      . . #+o. |
|      . o *.%o |
|     o   . @+.o |
|    . o . . +oB=. |
|    o   S o ooo+E |
|      .. o== |
|      o + |
|      +. |
|      +=. |
+-----[SHA256]-----+
```

Step-5:- Copy keys in id_rsa

#Copy in current Directory

```
cp ~/.ssh/id_rsa.pub .
```

```
cp ~/.ssh/id_rsa .
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/ansible-lab$ cp ~/.ssh/id_rsa
a.pub .
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/ansible-lab$ cp ~/.ssh/id_rsa
a .
```

Step-6:- Create Dockerfile

```
FROM ubuntu
```

```
RUN apt update -y
```

```
RUN apt install -y python3 python3-pip openssh-server
```

```
RUN mkdir -p /var/run/ssh
```

Configure SSH

```
RUN mkdir -p /run/ssh && \
  echo 'root:password' | chpasswd && \
  sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
  sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
  sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config
```

Create .ssh directory and set proper permissions

```
RUN mkdir -p /root/.ssh && \
  chmod 700 /root/.ssh
```

Copy SSH keys (note: this is not secure for production!)

```
COPY id_rsa /root/.ssh/id_rsa
```

```
COPY id_rsa.pub /root/.ssh/authorized_keys
```

Set proper permissions for keys

```
RUN chmod 600 /root/.ssh/id_rsa && \
  chmod 644 /root/.ssh/authorized_keys
```

Fix for SSH Login

```
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc
```

Expose SSH port

```
EXPOSE 22
```

Start SSH service when container starts

```
CMD ["/usr/sbin/sshd", "-D"]
```

```

GNU nano 7.2 Dockerfile
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

# Configure SSH
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

# Create .ssh directory
RUN mkdir -p /root/.ssh && \
    chmod 700 /root/.ssh

# Copy SSH keys
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

# Set permissions
RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

# SSH login fix
RUN sed -i 's@session\\s*required\\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]

```

Step-7:- Build Image

```
docker build -t ubuntu-server
```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ docker build -t
ubuntu-server .
[+] Building 36.7s (15/15) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 947B                                0.1s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.6s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [ 1/10] FROM docker.io/library/ubuntu:latest@sha256:186072bba1b2f 0.1s
=> => resolve docker.io/library/ubuntu:latest@sha256:186072bba1b2f43 0.1s
=> [internal] load build context                                    0.0s
=> => transferring context: 58B                                       0.0s
=> CACHED [ 2/10] RUN apt update -y                                0.0s
=> CACHED [ 3/10] RUN apt install -y python3 python3-pip openssh-ser 0.0s
=> CACHED [ 4/10] RUN mkdir -p /var/run/sshd                        0.0s
=> CACHED [ 5/10] RUN mkdir -p /run/sshd && echo 'root:password'    0.0s
=> CACHED [ 6/10] RUN mkdir -p /root/.ssh && chmod 700 /root/.ss    0.0s
=> CACHED [ 7/10] COPY id_rsa /root/.ssh/id_rsa                    0.0s
=> CACHED [ 8/10] COPY id_rsa.pub /root/.ssh/authorized_keys        0.0s
=> CACHED [ 9/10] RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /    0.0s
=> CACHED [10/10] RUN sed -i 's@session\\s*required\\s*pam_loginuid. 0.0s
=> exporting to image                                              33.9s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:840ae1782a62f55e094ffa4873746f023d01 0.0s
=> => exporting config sha256:ac809663b70999e5c3704934c5bb120e44ffda 0.0s
=> => exporting attestation manifest sha256:605cec12d055eaf93373c5b9 0.1s
=> => exporting manifest list sha256:63c3421f070f7291323fb9dafd10c6f 0.0s
=> => naming to docker.io/library/ubuntu-server:latest            0.0s
=> => unpacking to docker.io/library/ubuntu-server:latest          33.6s

```

Step-8:- Run container from build Image

```
docker run -d -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ docker run -d -  
-rm -p 2222:22 --name ssh-test-server ubuntu-server  
f9b11aaeaacecfa636be3b7c41b0e63c7f21217699e08471b3c5fab3b636c6d6
```

Step-9:- Find Container IP Address

```
docker inspect -f '{{.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ docker inspect  
-f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server  
172.17.0.3
```

Step-10:- Test SSH connection via Container IP

```
# with key  
ssh -i ~/.ssh/id_rsa root@172.17.0.2
```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ ssh -i ~/.ssh/id_rsa -p 2222 root@localhost
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:QpwuqXurAQGdQi7hTNChbtgkebuz0VzTL60Pq2zXiK.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
Enter passphrase for key '/home/palakagarwal_16/.ssh/id_rsa':
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@69b9d4a2aea0:~# for i in {1..4}; do
    echo "Creating server${i}"
    docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
    echo "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})"
done
Creating server1
-bash: docker: command not found
-bash: docker: command not found
IP of server1 is

```

Step-11:- Start Multiple Conatiners

```

for i in {1..4}; do
    echo -e "\n Creating server${i}\n"
    docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
    echo -e "IP of server${i} is $(docker inspect -f '' server${i})"
done

```



```

palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/ansible-lab$ for i in {1..4}
; do
echo "Creating server${i}"
docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
echo "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Net
works}}{{.IPAddress}}{{end}}' server${i})"
done
Creating server1
e13b4852dc3ea7e7cb3aaca3778804dd44762c70e601880f1794d08a4f358bf
IP of server1 is 172.17.0.4
Creating server2
96f1627e29b73c3c73e5743811476dce1b085e128b91366dfa876cc8c4326f2d
IP of server2 is 172.17.0.5
Creating server3
1f720a4c0b912a8881fd49bb3e0f6c8895dec2691ab50d785bf4648e91ad7957
IP of server3 is 172.17.0.6
Creating server4
76ec9d4514811caeb2cd64cbc71e5548c25b1dcbbdb1ea407430d276b9e1edf6
IP of server4 is 172.17.0.7

```

Step-12:- Create inventory.ini

```

# Get container IPs
echo "[servers]" > inventory.ini
for i in {1..4}; do
    docker inspect -f '{{.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done

# Add inventory variables
cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF

```

```

palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/ansible-lab$ echo "[servers]
" > inventory.ini

for i in {1..4}; do
    port=220${i}
    echo "server${i} ansible_host=127.0.0.1 ansible_port=${port}" >> inventory
.ini
done

cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF

```

Step-13:- Verify inventory.ini


```
cat inventory.ini
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ cat inventory.i
ni
[servers]
server1 ansible_host=127.0.0.1 ansible_port=2201
server2 ansible_host=127.0.0.1 ansible_port=2202
server3 ansible_host=127.0.0.1 ansible_port=2203
server4 ansible_host=127.0.0.1 ansible_port=2204

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

Step-14:- Test Connectivity

```
ssh -i ~/.ssh/id_rsa root@172.17.0.3
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ ssh -i ~/.ssh/i
d_rsa -p 2201 root@localhost
The authenticity of host '[localhost]:2201 ([127.0.0.1]:2201)' can't be esta
blished.
ED25519 key fingerprint is SHA256:QpwuqXurAQGdQi7hTNChbtgkebuz0VzTLe60Pq2zXi
k.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:4: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2201' (ED25519) to the list of known
hosts.
Enter passphrase for key '/home/palakagarwal_16/.ssh/id_rsa':
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x8
6_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@e13b4852dc3e:~# exit
logout
Connection to localhost closed.
```

**

Step-15:- Ansible Ping Test

```
ansible all -i inventory.ini -m ping
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/ansible-lab$ ANSIBLE_HOST_KEY_CHECKING=False ansible all -i inventory.ini -m ping
[WARNING]: Ansible is being run in a world writable directory (/mnt/c/Users/hp/ansible-lab), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir
server2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server4 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/ansible-lab$ |
```

Step-16:- Create Playbook `playbook1.yml`

```
--- # it should start with three dash only
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install required packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on "
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ nano playbook1.yml
```

Step-17:- Run Playbook

```
ansible-playbook -i inventory.ini playbook1.yml
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ ansible-playbook -i inventory.ini playbook1.yml
[WARNING]: Ansible is being run in a world writable directory (/mnt/c/Users/hp/ansible-lab), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir

PLAY [Update and configure servers] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server3]
ok: [server1]
ok: [server4]

TASK [Update apt packages] *****
changed: [server3]
changed: [server1]
changed: [server4]
changed: [server2]

TASK [Install required packages] *****
changed: [server1]
changed: [server3]
changed: [server4]
changed: [server2]

TASK [Create test file] *****
changed: [server4]
changed: [server3]
changed: [server1]
```

Step-18:- Verify Changes

```
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ ansible all -i
inventory.ini -m command -a "cat /root/ansible_test.txt"
[WARNING]: Ansible is being run in a world writable directory
(/mnt/c/Users/hp/ansible-lab), ignoring it as an ansible.cfg source. For more
information see
https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-
in-
world-writable-dir
server1 | CHANGED | rc=0 >>
Configured by Ansible on server1
server3 | CHANGED | rc=0 >>
Configured by Ansible on server3
server4 | CHANGED | rc=0 >>
Configured by Ansible on server4
server2 | CHANGED | rc=0 >>
Configured by Ansible on server2
```

Step-19:- CleanUp

```
for i in {1..4}; do docker rm -f server${i}; done
```

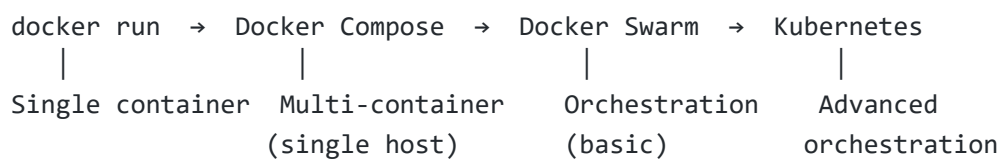
```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/ansible-lab$ for i in {1..4}
; do docker rm -f server${i}; done
server1
server2
server3
server4
```

Cloud-containerization-and-DevOps

Experiment 10: Orchestration via Docker Swarm

Introduction

The Progression Path [↗](#)



This experiment focuses on: Moving from Compose → Swarm

Hands-On

Step-1:- Create a `docker-compose.yml`

```
nano docker-compose.yml
```

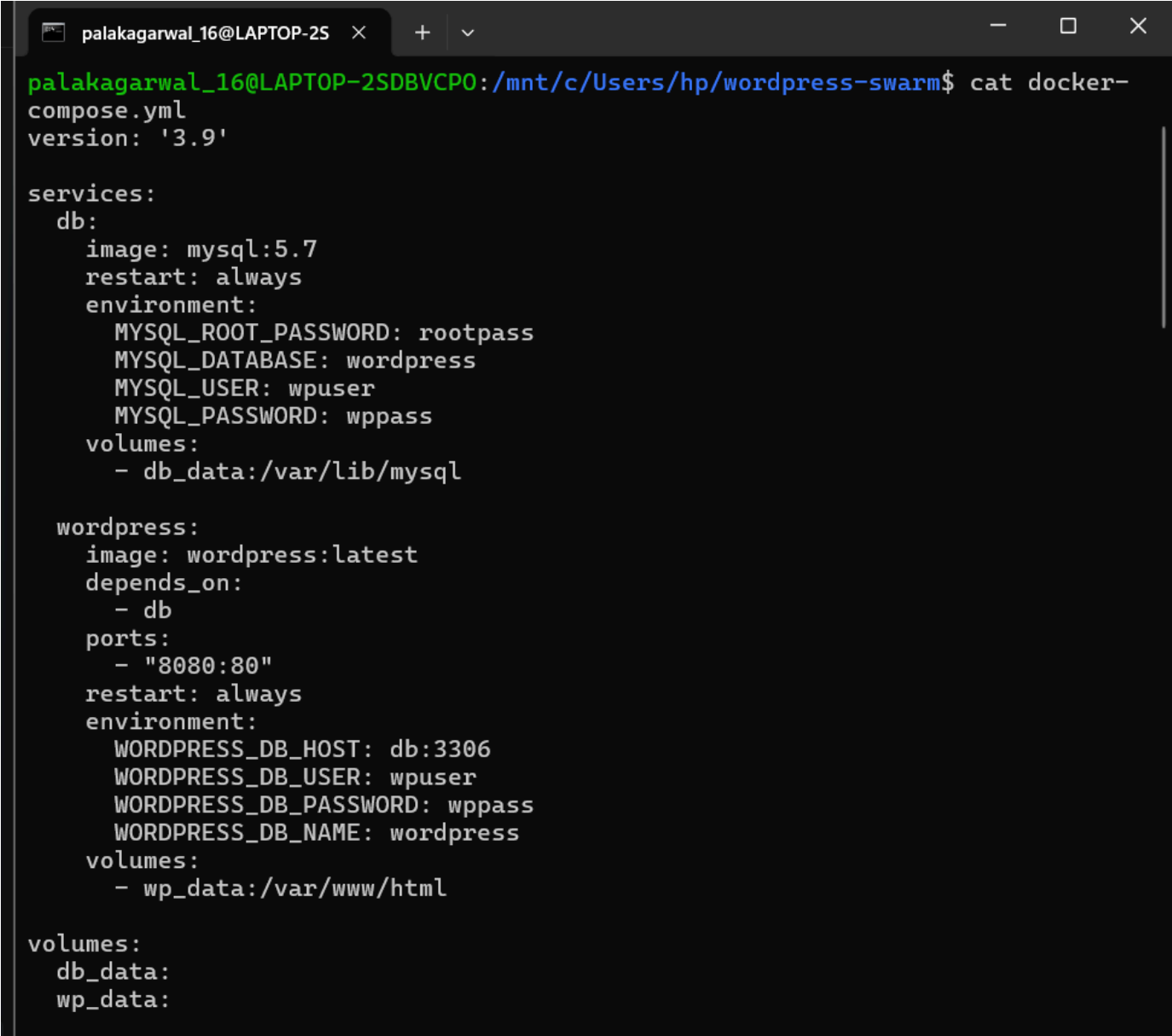
Paste This:

```
version: '3.9'

services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql
```

```
wordpress:
  image: wordpress:latest
  depends_on:
    - db
  ports:
    - "8080:80"
  restart: always
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wppass
    WORDPRESS_DB_NAME: wordpress
  volumes:
    - wp_data:/var/www/html
```

```
volumes:
  db_data:
  wp_data:
```



The screenshot shows a terminal window with a dark background. The title bar indicates the user is 'palakagarwal_16@LAPTOP-25'. The prompt shows the user is in the directory '/mnt/c/Users/hp/wordpress-swarm'. The command 'cat docker-compose.yml' has been executed, displaying the following content:

```
palakagarwal_16@LAPTOP-25DBVCP0:/mnt/c/Users/hp/wordpress-swarm$ cat docker-
compose.yml
version: '3.9'

services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Step-2:- Initialize Swarm

```
docker swarm init
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker swarm
init
Swarm initialized: current node (vun5tkvap6dlkk6blb8c3l21n) is now a manager
.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2wf4ugl3r42j2knlulxvc0igz8apjvd5azq9l
3l0rpg174q5fm-3bgwurkbv87gln9fmhc5xyfx0 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and fo
llow the instructions.
```

Step-3:- Verify Swarm is Active

```
docker node ls
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker node
ls
ID                                HOSTNAME                STATUS    AVAILABILITY    MANA
GER STATUS    ENGINE VERSION
vun5tkvap6dlkk6blb8c3l21n *    docker-desktop    Ready    Active           Lead
er                                29.2.1
```

Step-4:- Deploy as a stack

```
docker stack deploy -c docker-compose.yml wpstack
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker stac
k deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in the backgro
und.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
```

Step-5:- Verify by listing Services

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker serv
ice ls
ID                                NAME                MODE                REPLICAS    IMAGE
PORTS
ybka4kvту2e5    wpstack_db          replicated          1/1          mysql:5.7
on3fj4rpmkxz    wpstack_wordpress   replicated          0/1          wordpress:latest
*:8080->80/tcp
```

Step-6:- Details for container service

```
docker service ps wpstack_wordpress
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker service ps wpstack_wordpress
ID                NAME                IMAGE                ERROR                PORTS                NODE                DES
IREED STATE      CURRENT STATE      ERROR                PORTS                docker-desktop      Run
qn0kyckh0au4     wpstack_wordpress.1  wordpress:latest     Preparing 20 seconds ago
```

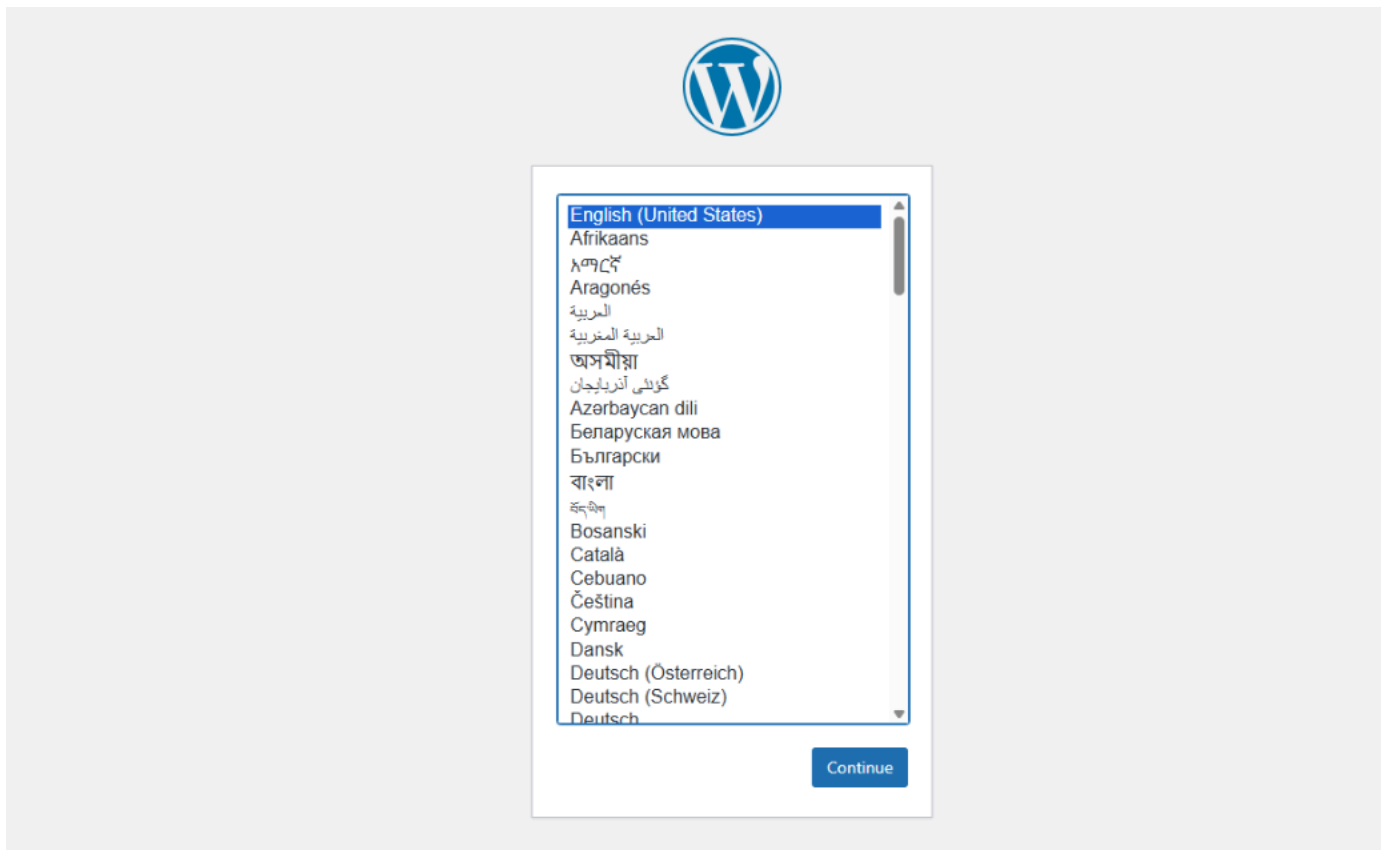
Step-7:- List Containers

```
docker ps
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker ps
CONTAINER ID   IMAGE                                COMMAND              CREATED
STATUS        PORTS              NAMES
6c1ffef4a395   mysql:5.7                "docker-entrypoint.s..." 28 seconds ago
Up 28 seconds  3306/tcp, 33060/tcp wpstack_db.1.z7898s
85girsyfmjlmm0i11l7
3d5a2f94a432   sonarqube:lts-community    "/opt/sonarqube/dock..." 5 days ago
Up 3 minutes  0.0.0.0:9001->9000/tcp, [::]:9001->9000/tcp sonarqube
de3235249e19   postgres:13                "docker-entrypoint.s..." 5 days ago
Up 3 minutes  5432/tcp      sonar-db
16caf86b6b3a   portainer/portainer-ce:lts  "/portainer"          2 weeks ago
Up 3 minutes  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp portainer
2d80294c3c52   docker-lab-app             "docker-entrypoint.s..." 2 weeks ago
Restarting (1) 40 seconds ago multi-stage-app
84697fc912c5   postgres:15                "docker-entrypoint.s..." 2 weeks ago
Up 3 minutes  5432/tcp      postgres-db
```

Step-8:- Verify application on Containers

```
http://localhost:8080/
```

Step-9:- Scale application Containers

```
docker service scale wpstack_wordpress=3
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
```

Step-10:- Verify Scalling

```
docker service ls
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker service ls
ID                NAME                MODE                REPLICAS            IMAGE
ybka4kvту2e5      wpstack_db           replicated          1/1                  mysql:5.7
on3fj4rpmkxz      wpstack_wordpress    replicated          3/3                  wordpress:latest
*:8080->80/tcp
```

Step-11:- List Wordpress Containers

```
docker ps | grep wordpress
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker ps |
grep wordpress
981cec7eeda0    wordpress:latest    "docker-entrypoint.s..."    About a
minute ago    Up About a minute    80/tcp    wpstack_wordpre

ss.3.tf0cz1ja5uxytaww799ev3vya
72238897389d    wordpress:latest    "docker-entrypoint.s..."    About a
minute ago    Up About a minute    80/tcp    wpstack_wordpre

ss.2.vdq51w30fiqgv07js0zia0wyo
bb9d8ab0e61d    wordpress:latest    "docker-entrypoint.s..."    About a
minute ago    Up About a minute    80/tcp    wpstack_wordpre

ss.1.qn0kyckh0au4mxd1ebro1647z
```

Step-12:- Kill any Container

```
docker kill <container-id>
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker kill
981cec7eeda0
981cec7eeda0
```

Step-13:- Watch Swarm Recreating Contrainer

```
docker service ps wpstack_wordpress
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker serv
ice ps wpstack_wordpress
ID                NAME                IMAGE                NODE
DESIRED STATE    CURRENT STATE      ERROR                PORTS
qn0kyckh0au4     wpstack_wordpress.1    wordpress:latest    docker-desktop
Running          Running about a minute ago
vdq51w30fiqg     wpstack_wordpress.2     wordpress:latest    docker-desktop
Running          Running about a minute ago
rgk0u908g65n     wpstack_wordpress.3     wordpress:latest    docker-desktop
Running          Running 12 seconds ago
tf0cz1ja5uxy     \_ wpstack_wordpress.3    wordpress:latest    docker-desktop
Shutdown        Failed 18 seconds ago    "task: non-zero exit (137)"
```

Step-14:- Verify Container running

```
docker ps | grep wordpress
```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker ps |
grep wordpress
15eb0540dfaf    wordpress:latest    "docker-entrypoint.s..."  23 seco
nds ago    Up 17 seconds    80/tcp    wpstack_wordpress.3

.rgk0u908g65nsk21fmuxlr5r5
72238897389d    wordpress:latest    "docker-entrypoint.s..."  2 minut
es ago    Up 2 minutes    80/tcp    wpstack_wordpress.2

.vdq51w30fiqgv07js0zia0wyo
bb9d8ab0e61d    wordpress:latest    "docker-entrypoint.s..."  2 minut
es ago    Up 2 minutes    80/tcp    wpstack_wordpress.1

.qn0kyckh0au4mxd1ebroi647z

```

Step-15:- Remove Stack

```
docker stack rm wpstack
```

```

palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker stac
k rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default

```

Step-16:- Verify Service Removal

```

docker service ls
docker ps

```

```

docker ps
ID                NAME                MODE                REPLICAS            IMAGE                PORTS                CREATED
CONTAINER ID    IMAGE                STATUS                COMMAND                PORTS                NAMES
15eb0540dfaf    wordpress:latest    Up 27 seconds        "docker-entrypoint.s... 80/tcp                wpstack_wordpress.3
.rgk0u908g65nsk21fmuxlr5r5
72238897389d    wordpress:latest    Up 2 minutes         "docker-entrypoint.s... 80/tcp                wpstack_wordpress.2
.vdq51w30fiqgv07js0zia0wyo
bb9d8ab0e61d    wordpress:latest    Up 2 minutes         "docker-entrypoint.s... 80/tcp                wpstack_wordpress.1
.qn0kyckh0au4mxd1ebroi647z
6c1ffef4a395    mysql:5.7           Up 2 minutes         "docker-entrypoint.s... 3306/tcp, 33060/tcp    wpstack_db.1.z7898s
85girsyfmj1mm0i11l7
3d5a2f94a432    sonarqube:lts-community    Up 6 minutes         "/opt/sonarqube/dock... 0.0.0.0:9001->9000/tcp, [::]:9001->9000/tcp    sonarqube
de3235249e19    postgres:13         Up 6 minutes         "docker-entrypoint.s... 5432/tcp                sonar-db
16caf86b6b3a    portainer/portainer-ce:lts    Up 6 minutes         "/portainer"            0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp    portainer
2d80294c3c52    docker-lab-app       Restarting (1) 55 seconds ago    "docker-entrypoint.s..."                multi-stage-app
84697fc912c5    postgres:15         Up 6 minutes         "docker-entrypoint.s..." 5432/tcp                postgres-db
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/wordpress-swarm$ docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS

```

Conclusion

Summary

You started with	You can now do
Single container (<code>docker run</code>)	Multi-container (Compose)
Manual scaling	One-command scaling (<code>scale</code>)
Manual recovery	Automatic self-healing
Single host	Multi-host cluster ready

Final Takeaway

Compose defines the application. Swarm runs it reliably.

Quick Reference Card

Initialize Swarm

```
docker swarm init
```

Deploy stack

```
docker stack deploy -c docker-compose.yml <stack-name>
```

List services

```
docker service ls
```

Scale service

```
docker service scale <stack-name_service-name>=<replicas>
```

See service tasks

```
docker service ps <service-name>
```

Remove stack

```
docker stack rm <stack-name>
```

Leave Swarm (if needed)

```
docker swarm leave --force
```

Cloud-containerization-and-DevOps

Experiment 10: Working with SonarQube

Introduction

What is SonarQube?

SonarQube is an open-source platform for continuous inspection of code quality. It performs automatic reviews with static analysis to detect bugs, code smells &/ security vulnerabilities.

How SonarQube Solves the Problem:

- **Continuous Inspection:** Scans code with every commit, providing immediate feedback.
- **Quality Gates:** Defines pass/fail criteria for code quality.
- **Technical Debt Quantification:** Measures effort needed to fix issues.
- **Multi-language Support:** Supports 20+ programming languages.
- **Visual Analytics:** Dashboard showing code quality metrics and trends.

Hands-On

Step-1:- Create a `docker-compose.yml`

```
nano docker-compose.yml
```

Paste This:

```
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
```

```
volumes:  
  - sonar-db-data:/var/lib/postgresql/data  
networks:  
  - sonarqube-lab
```

```
sonarqube:  
  image: sonarqube:lts-community  
  container_name: sonarqube  
  ports:  
    - "9000:9000"  
  environment:  
    SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube  
    SONAR_JDBC_USERNAME: sonar  
    SONAR_JDBC_PASSWORD: sonar  
  volumes:  
    - sonar-data:/opt/sonarqube/data  
    - sonar-extensions:/opt/sonarqube/extensions  
  depends_on:  
    - sonar-db  
  networks:  
    - sonarqube-lab
```

```
volumes:  
  sonar-db-data:  
  sonar-data:  
  sonar-extensions:
```

```
networks:  
  sonarqube-lab:  
    driver: bridge
```

```

GNU nano 7.2 docker-compose.yml
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    restart: unless-stopped
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
    networks:
      - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    restart: unless-stopped
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - sonarqube-lab

volumes:
  sonar-db-data:
  sonar-data:

```

Step-2:- Start Compose

```
docker-compose up -d
```

```

palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/sample-java-app$ docker comp
ose up -d
WARN[0000] /mnt/c/Users/hp/docker-compose.yml: the attribute `version` is ob
solete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] Found orphan containers ([ba15d4b7600d]) for this project. If you
removed or renamed this service in your compose file, you can run this comm
and with the --remove-orphans flag to clean it up.
[+] up 2/2
✔Container sonar-db   Running      0.0s
✔Container sonarqube  Running      0.0s
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp/sample-java-app$

```

Step-3:- Verify SonarQube is running via Logs


```
docker-compose logs -f sonarqube
```

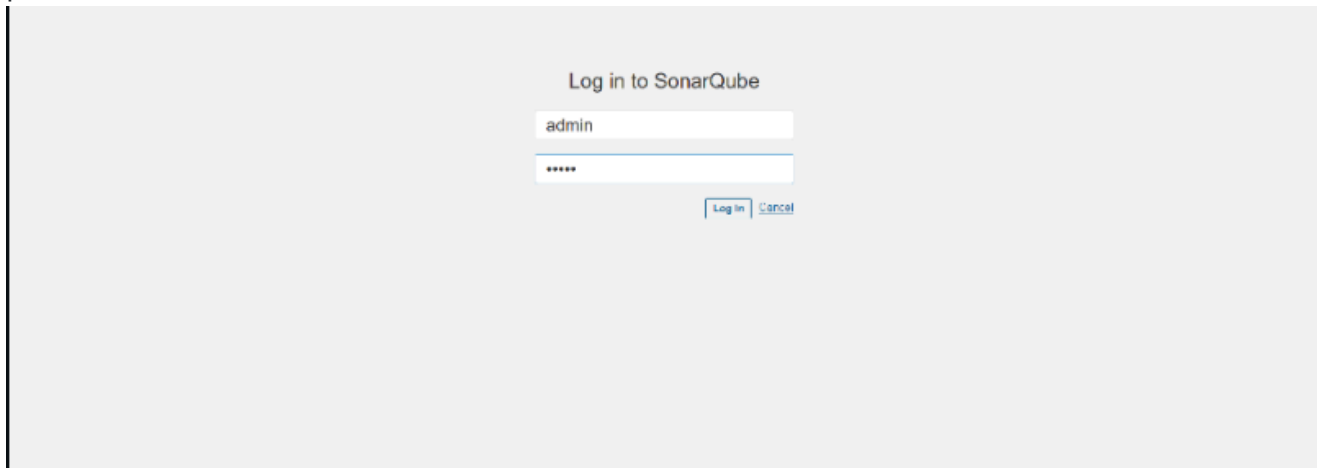
```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ docker logs -f sonarqube
2026.04.18 04:22:35 INFO app[][o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
2026.04.18 04:22:35 INFO app[][o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:35219]
2026.04.18 04:22:35 INFO app[][o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]: /opt/sonarqube/elasticsearch/bin/elasticsearch
2026.04.18 04:22:35 INFO app[][o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
2026.04.18 04:22:44 INFO es[][o.e.n.Node] version[7.17.15], pid[25], build[default/tar/0b8ecfb4378335f4689c4223d1f1115f16bef3ba/2023-11-10T22:03:46.987399016Z], OS[Linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8]
2026.04.18 04:22:44 INFO es[][o.e.n.Node] JVM home [/opt/java/openjdk]
2026.04.18 04:22:44 INFO es[][o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/sonarqube/logs/es_hs_err_pid%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.headless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimization=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog4j.shutdownHookEnabled=false, -Dlog4j2.disable.jmx=true, -Dlog4j2.formatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -Des.enforce.bootstrap.checks=true, -Xmx512m, -Xms512m, -XX:MaxDirectMemorySize=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqube/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distribution.flavor=default, -Des.distribution.type=tar, -Des.bundled_jdk=false]
2026.04.18 04:22:45 INFO es[][o.e.p.PluginsService] loaded module [analysis-common]
2026.04.18 04:22:45 INFO es[][o.e.p.PluginsService] loaded module [lang-painless]
2026.04.18 04:22:45 INFO es[][o.e.p.PluginsService] loaded module [parent-join]
2026.04.18 04:22:45 INFO es[][o.e.p.PluginsService] loaded module [reindex]
2026.04.18 04:22:45 INFO es[][o.e.p.PluginsService] loaded module [transport-netty4]
2026.04.18 04:22:45 INFO es[][o.e.p.PluginsService] no plugins loaded
```

Step-4:- Login SonarQube

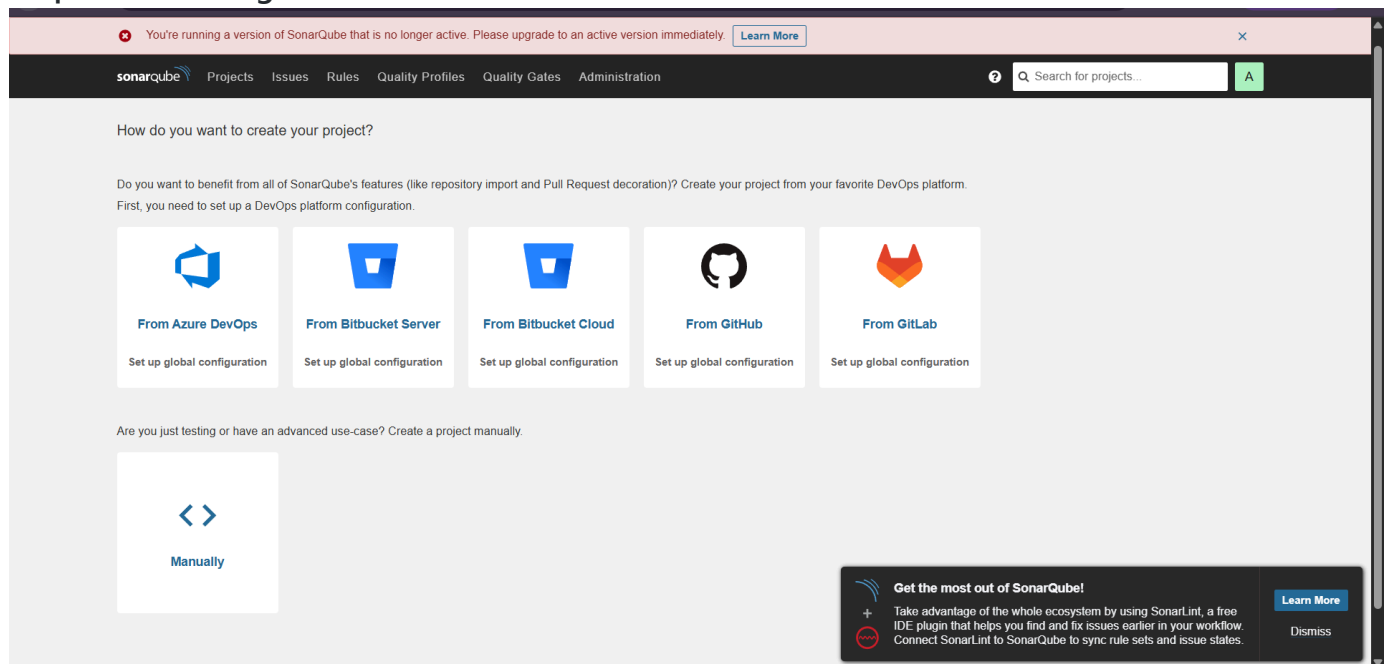
<http://localhost:9001>

- username: admin

- password: admin



Step-5:- HomePage



Step-6:- Create Sample Java App

```
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

Step-7:- Create Calculator.java

```
nano src/main/java/com/example/Calculator.java
```

```
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }
}
```

```
public int add(int a, int b) {
    int result = a + b;
    int unused = 100;
    return result;
}

public String getUser(String userId) {
    String query = "SELECT * FROM users WHERE id = " + userId;
    return query;
}

public int multiply(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a;
    }
    return result;
}

public int multiplyAlt(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a;
    }
    return result;
}

public String getName(String name) {
    return name.toUpperCase();
}

public void riskyOperation() {
    try {
        int x = 10 / 0;
    } catch (Exception e) {
    }
}
}
```

```
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public String getName(String name) {
        return name.toUpperCase();
    }
}
```

Step-8:- Create pom.xml

nano pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN</sonar.login>
  </properties>
```

```

<build>
  <plugins>
    <plugin>
      <groupId>org.sonarsource.scanner.maven</groupId>
      <artifactId>sonar-maven-plugin</artifactId>
      <version>3.9.1.2184</version>
    </plugin>
  </plugins>
</build>
</project>

```

```

<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN</sonar.login>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>

```

Step-9:- Create Tokens Profile -> My Account -> Security -> TokenName: scanner-token -> Generate

Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name	Type	Expires in	
Enter Token Name	Select Token Type	30 days	Generate

 New token "palaktoken" has been created. Make sure you copy it now, you won't be able to see it again!

 Copy `squ_4b67cf84f133e4b7e2cee2e1ee096626522ea301`

Name	Type	Project	Last use	Created	Expiration	
palaktoken	User		Never	April 18, 2026	–	Revoke

Step-10:- Run Scanner

```
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

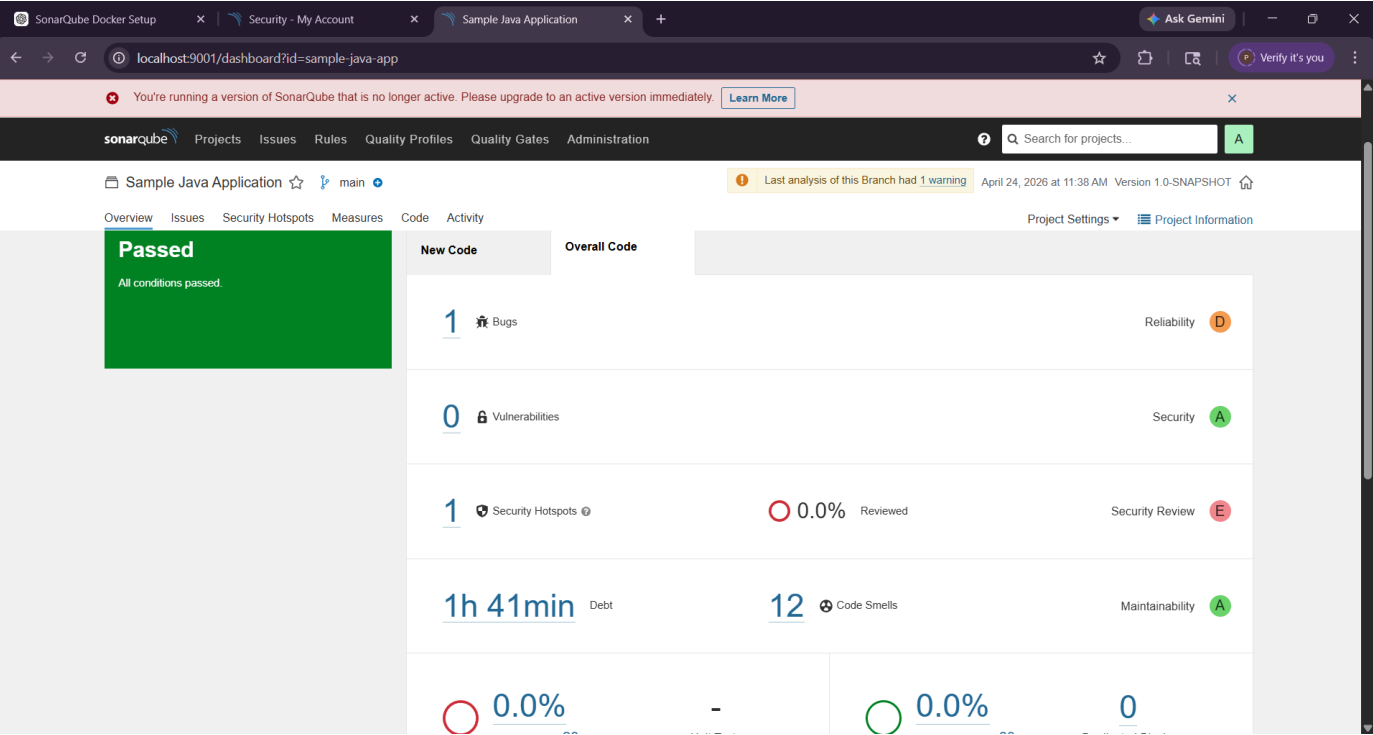
```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/sample-java-app$ mvn sonar:sonar -Dsonar.login=squ_2e95439e05120daf211c16a731ece5545447a39f
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- sonar-maven-plugin:4.0.0.4121:sonar (default-cli) @ sample-app ---
[INFO] 06:08:09.532 Java 21.0.10 Ubuntu (64-bit)
[INFO] 06:08:09.533 Linux 6.6.87.2-microsoft-standard-WSL2 (amd64)
[INFO] 06:08:10.646 User cache: /home/palakagarwal_16/.sonar/cache
[INFO] 06:08:11.249 Communicating with SonarQube Server 9.9.8.100196
[INFO] 06:08:11.394 Default locale: "en", source code encoding: "UTF-8" (analysis is platform dependent)
[INFO] 06:08:13.167 Load global settings
[INFO] 06:08:13.678 Load global settings (done) | time=515ms
[INFO] 06:08:13.689 Server id: 74C15348-AZ2-FeoTJ4JgJc_ZF7TJ
[INFO] 06:08:13.708 User cache: /home/palakagarwal_16/.sonar/cache
[INFO] 06:08:13.728 Load/download plugins
[INFO] 06:08:13.728 Load plugins index
[INFO] 06:08:13.928 Load plugins index (done) | time=200ms
[INFO] 06:08:20.989 Load/download plugins (done) | time=7261ms
[INFO] 06:08:22.617 Process project properties
[INFO] 06:08:22.644 Process project properties (done) | time=27ms
[INFO] 06:08:22.653 Execute project builders
[INFO] 06:08:22.656 Execute project builders (done) | time=3ms
[INFO] 06:08:22.671 Project key: sample-java-app
[INFO] 06:08:22.672 Base dir: /mnt/c/Users/hp/sample-java-app
[INFO] 06:08:22.672 Working dir: /mnt/c/Users/hp/sample-java-app/target/sonar
[INFO] 06:08:22.704 Load project settings for component key: 'sample-java-app'
[INFO] 06:08:23.525 Load quality profiles
[INFO] 06:08:23.853 Load quality profiles (done) | time=329ms
[INFO] 06:08:23.891 Load active rules
[INFO] 06:08:29.076 Load active rules (done) | time=5185ms
```

Step-11:- Scroll down to view Build Success

```
INFO] 06:08:42.267 Sensor Analysis Warnings import [csharp] (done) | time=2
IS
INFO] 06:08:42.268 Sensor Zero Coverage Sensor
INFO] 06:08:42.303 Sensor Zero Coverage Sensor (done) | time=35ms
INFO] 06:08:42.304 Sensor Java CPD Block Indexer
INFO] 06:08:42.355 Sensor Java CPD Block Indexer (done) | time=51ms
INFO] 06:08:42.365 SCM Publisher SCM provider for this project is: git
INFO] 06:08:42.368 SCM Publisher 2 source files to be analyzed
WARNING] 06:08:42.613 Thread[#50,JGit-FileStoreAttributeReader-1,5,main]: g
t smaller file timestamp on /mnt/c (C:\), /mnt/c/Users/hp/.git: 2026-04-24T
06:08:42Z < 2026-04-24T06:08:42.1555933Z. Aborting measurement at resolution
PT0.8444067S.
INFO] 06:08:42.867 SCM Publisher 0/2 source files have been analyzed (done)
| time=497ms
WARNING] 06:08:42.867 Missing blame information for the following files:
WARNING] 06:08:42.868 * src/main/java/com/example/Calculator.java
WARNING] 06:08:42.868 * pom.xml
WARNING] 06:08:42.869 This may lead to missing/broken features in SonarQube
INFO] 06:08:42.876 CPD Executor Calculating CPD for 1 file
INFO] 06:08:42.897 CPD Executor CPD calculation finished (done) | time=20ms
INFO] 06:08:43.570 Analysis report generated in 365ms, dir size=130.7 kB
INFO] 06:08:45.208 Analysis report compressed in 1624ms, zip size=20.4 kB
INFO] 06:08:46.220 Analysis report uploaded in 1011ms
INFO] 06:08:46.232 ANALYSIS SUCCESSFUL, you can find the results at: http:/
localhost:9001/dashboard?id=sample-java-app
INFO] 06:08:46.232 Note that you will be able to access the updated dashboa
rd once the server has processed the submitted analysis report
INFO] 06:08:46.233 More about the report processing at http://localhost:900
/api/ce/task?id=AZ2-GuJeJ4JgJc_ZGFMj
INFO] 06:08:46.494 Analysis total time: 25.115 s
INFO] -----
--
INFO] BUILD SUCCESS
INFO] -----
--
INFO] Total time: 37.802 s
INFO] Finished at: 2026-04-24T06:08:46Z
INFO] -----
--
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp/sample-java-app$ docker ps
```

Step-12:- On Browser test will be ListedThe image shows a web browser window displaying the SonarQube login page. The page has a light gray background. At the top, the text "Log in to SonarQube" is centered. Below this, there are two input fields: the first is labeled "admin" and the second is labeled "****" (representing a password). To the right of the password field, there are two buttons: "Log in" and "Cancel". The "Log in" button is highlighted with a blue border and a blue shadow.

Step-13:- View Report



Cloud-containerization-and-DevOps

Experiment 12: Kubernetes

Introduction

Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

Core Kubernetes Concepts (Simple Explanation)

Docker Concept	Kubernetes Equivalent	What it means
Container	Pod	A pod is a group of one or more containers. Smallest unit in K8s.
Compose service	Deployment	Describes how your app should run (e.g., 2 copies, which image to use)
Load balancing	Service	Exposes your app to the outside world or other pods
Scaling	ReplicaSet	Ensures a certain number of pod copies are always running

Hands-On

Step-1:- Create a `wordpress-deployment.yaml`

nano wordpress-deployment.yaml

Paste This:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

```
palakagarwal_16@LAPTOP-2SDBVCPO:/mnt/c/Users/hp$ cat wordpress-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

Step-2:- Apply Deployment

```
kubectl apply -f wordpress-deployment.yaml
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ kubectl apply -f wordpress-  
deployment.yaml  
deployment.apps/wordpress created
```

Step-3:- Create wordpress-service.yaml

```
nano wordpress-service.yaml
```

Paste this:-

```
apiVersion: v1  
kind: Service  
metadata:  
  name: wordpress-service  
spec:  
  type: NodePort  
  selector:  
    app: wordpress  
  ports:  
    - port: 80  
      targetPort: 80  
      nodePort: 30007
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ cat wordpress-service.yaml  
apiVersion: v1  
kind: Service  
metadata:  
  name: wordpress-service  
spec:  
  type: NodePort  
  selector:  
    app: wordpress  
  ports:  
    - port: 80  
      targetPort: 80  
      nodePort: 30007
```

Step-4:- Deploy Service

```
kubectl apply -f wordpress-service.yaml
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ kubectl apply -f wordpress-  
service.yaml  
service/wordpress-service created
```

Step-5:- Get pods

```
kubectl get pods
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-6t6cp	0/1	ContainerCreating	0	65s
wordpress-6698dd7d66-x5bh9	0/1	ContainerCreating	0	65s

Step-6:- Check service

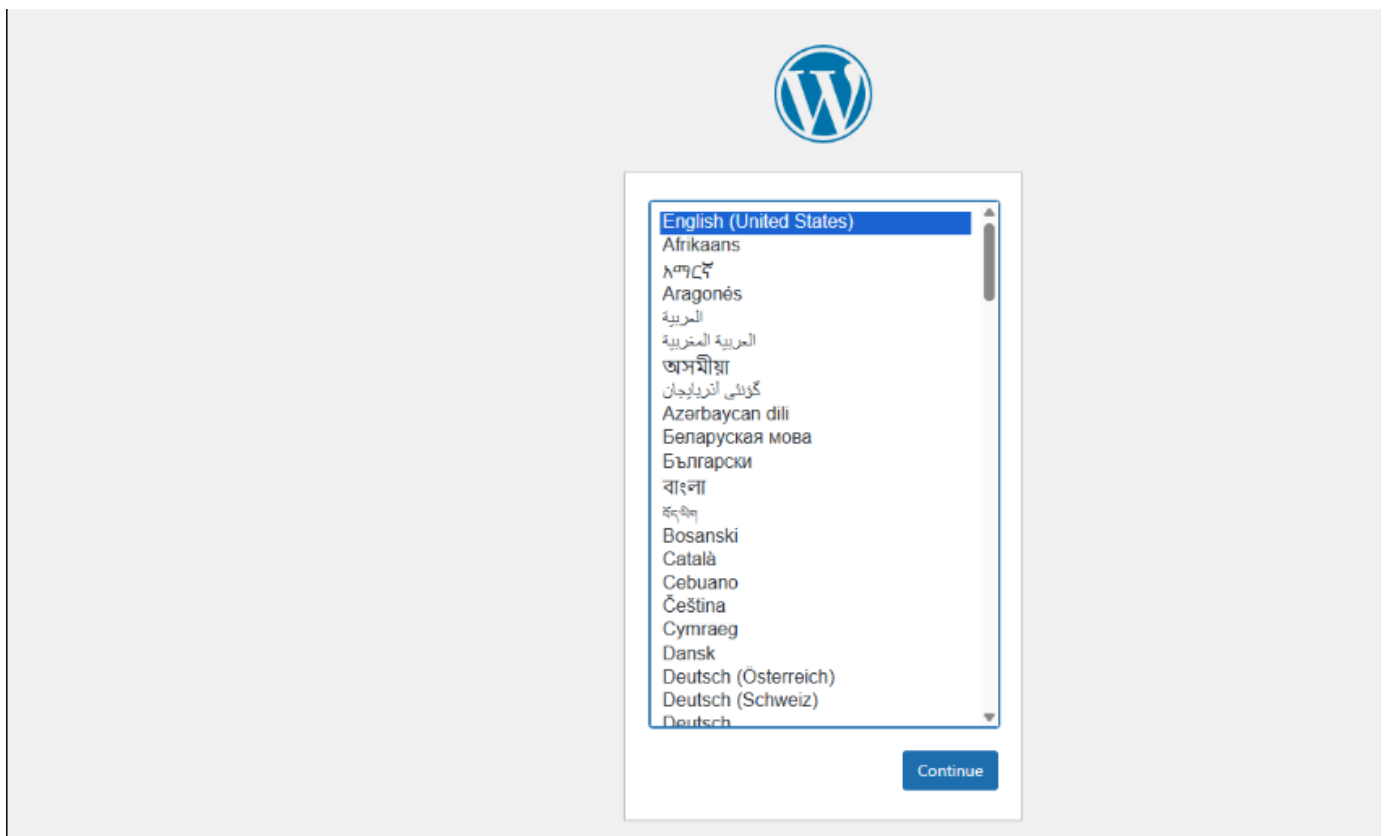
```
kubectl get svc
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP
wordpress-service	NodePort	10.109.20.189	<none>	80:30007/TCP

Step-7:- Verify On Browser

```
http://localhost:8080/
```



Step-8:- Scale Deployment

```
kubectl scale deployment wordpress --replicas=4
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ kubectl scale deployment wo  
rdpress --replicas=4  
deployment.apps/wordpress scaled
```

Step-9:- Get Pods

```
kubectl get pods
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ kubectl get pods  
NAME                                READY   STATUS             RESTARTS   AGE  
wordpress-6698dd7d66-4gn62         0/1     ContainerCreating   0           6s  
wordpress-6698dd7d66-6t6cp         0/1     ContainerCreating   0           92s  
wordpress-6698dd7d66-vfw7d         0/1     ContainerCreating   0           6s  
wordpress-6698dd7d66-x5bh9         0/1     ContainerCreating   0           92s
```

Step-10:- Delete Pods to check Self-healing

```
kubectl delete pod <pod-name>  
kubectl get pods
```

```
palakagarwal_16@LAPTOP-2SDBVCP0:/mnt/c/Users/hp$ kubectl delete pod wordpres  
s-6698dd7d66-4gn62
```

Insights

When to Use Which Tool

Tool	Best for
k3d	Quick learning on your laptop
Minikube	Single-node cluster testing
kubeadm	Real, production-style cluster

Summary of Commands (Cheat Sheet)

Goal	Command
Apply a YAML file	<code>kubectl apply -f file.yaml</code>
See all pods	<code>kubectl get pods</code>
See all services	<code>kubectl get svc</code>

Goal	Command
Scale a deployment	<code>kubectl scale deployment <name> --replicas=N</code>
Delete a pod	<code>kubectl delete pod <pod-name></code>
See all nodes	<code>kubectl get nodes</code>